Artificial Intelligence

Mihir Rao

Contents

1 Introduction to AI and Rational Agents **6**
1.1 Agents and their environments . . . 6
1.2 Reflex and planning agents . . . 6
1.3 Specifying a task with PEAS . . . 7
1.4 Properties of the environment . . . 7

2 Search: State Spaces and Uninformed Search **8**
2.1 Formulating a search problem . . . 8
2.2 State-space graphs and search trees . . . 9
2.3 A common tree-search procedure . . . 10
2.4 Depth-first search . . . 10
2.5 Breadth-first search . . . 11
2.6 Iterative deepening . . . 11
2.7 Uniform-cost search . . . 12

3 Search: Informed Search, A* and Heuristics **13**
3.1 Heuristics and remaining cost . . . 13
3.2 Greedy search and A* . . . 13
3.3 Admissibility and A* tree search . . . 14
3.4 Constructing heuristics by relaxation . . . 15
3.5 Dominance and combining estimates . . . 16
3.6 Graph search and repeated states . . . 17
3.7 Consistency and A* graph search . . . 18

4 Search: Local Search **19**
4.1 A* heuristics and memory . . . 19
4.2 Local search problems . . . 19
4.3 Hill climbing . . . 20
4.4 Simulated annealing . . . 21
4.5 Local beam search . . . 22
4.6 Genetic algorithms . . . 23
4.7 Continuous state spaces . . . 24

5 Games: Trees, Minimax and Pruning **26**
5.1 Games and adversarial search . . . 26
5.2 State values and minimax . . . 26
5.3 Multiple players and general-sum games . . . 28
5.4 Alpha-beta pruning . . . 28
5.5 Depth-limited search and evaluation functions . . . 30

6 Games: Expectimax and Monte Carlo Tree Search **32**
6.1 Uncertain outcomes and expectimax . . . 32
6.2 Pruning and depth limits . . . 33

6.3 Choosing the probabilities . . . 33
6.4 Mixed layer types . . . 34
6.5 Monte Carlo tree search . . . 34

7 Logic: Propositional Logic and Planning **37**
7.1 Knowledge-based agents . . . 37
7.2 Syntax, semantics, and entailment . . . 37
7.3 Propositional logic . . . 38
7.4 A knowledge base for partially observable Pacman . . . 38
7.5 Entailment, implication, and reasoning tasks . . . 39

8 Logic: Logical Inference, Theorem Proving, Boolean Satisfiability and DPLL **41**
8.1 Validity, satisfiability, and entailment . . . 41
8.2 Logical equivalences and conjunctive normal form . . . 41
8.3 Forward chaining . . . 42
8.4 DPLL satisfiability . . . 43
8.5 Resolution . . . 45
8.6 Logical agents and planning as satisfiability . . . 46
8.7 State estimation, localization, and mapping . . . 46

9 Logic: First-Order Logic **48**
9.1 Fluents and successor-state axioms . . . 48
9.2 Representations and expressive power . . . 48
9.3 Possible worlds . . . 49
9.4 Syntax and semantics . . . 49
9.5 Writing sentences . . . 50
9.6 Inference in first-order logic . . . 50
9.7 Propositionalization . . . 51
9.8 Lifted inference . . . 51

10 Introduction to Probability **52**
10.1 Uncertainty and probability models . . . 52
10.2 Joint and marginal distributions . . . 53
10.3 Conditional probability and normalization . . . 53
10.4 The product and chain rules . . . 54
10.5 Inference by enumeration . . . 54
10.6 Bayes' rule . . . 55
10.7 Independence . . . 55
10.8 Conditional independence . . . 56

11 Bayes Nets: Bayesian Networks **58**
11.1 From conditional independence to compact models . . . 58
11.2 Bayes net syntax . . . 58
11.3 Global semantics . . . 59
11.4 Conditional independence semantics . . . 60
11.5 Node ordering and causality . . . 61

12 Bayes Nets: Inference **63**
12.1 Inference by enumeration in Bayes nets . . . 63
12.2 Factors . . . 63
12.3 Joining and eliminating . . . 64
12.4 General variable elimination . . . 65
12.5 Elimination order and complexity . . . 66

13 Bayes Nets: Sampling **68**
13.1 Why sample . . . 68
13.2 Prior sampling . . . 69
13.3 Rejection sampling . . . 69
13.4 Likelihood weighting . . . 70
13.5 Markov chain Monte Carlo . . . 71
13.6 Gibbs sampling . . . 71

14 HMMs: Markov Chains and Hidden Markov Models **74**
14.1 Markov models . . . 74
14.2 Prediction with the mini-forward algorithm . . . 75
14.3 Stationary distributions . . . 75
14.4 Hidden Markov models . . . 76
14.5 Inference tasks . . . 77

15 HMMs: Forward and Viterbi Algorithms, Dynamic Bayes Nets and Particle Filtering **79**
15.1 Recursive filtering . . . 79
15.2 Most likely explanation and the Viterbi algorithm . . . 80
15.3 Dynamic Bayes nets . . . 82
15.4 Particle filtering . . . 82

16 Utility Theory, Rationality, Decision Networks and Value of Perfect Information **84**
16.1 Utilities and maximum expected utility . . . 84
16.2 Rational preferences . . . 84
16.3 Human utilities and money . . . 85
16.4 Utilities of sequences . . . 86
16.5 Decision networks . . . 86
16.6 Value of perfect information . . . 87
16.7 The optimizer's curse . . . 89

17 MDPs: States, Values, Policies and Q-values **90**
17.1 Markov decision processes . . . 90
17.2 Policies . . . 91
17.3 Utilities of reward sequences . . . 92
17.4 Optimal quantities and the Bellman equations . . . 93
17.5 Time-limited values and value iteration . . . 94

18 MDPs: Dynamic Programming **96**
18.1 Value iteration as a fixed-point method . . . 96
18.2 Policy evaluation . . . 97
18.3 Policy extraction . . . 98
18.4 Policy iteration . . . 98
18.5 Planning versus learning . . . 100

19 Machine Learning I **101**
19.1 Classification . . . 101
19.2 The Naive Bayes model . . . 101
19.3 Naive Bayes for digits and text . . . 103
19.4 Maximum likelihood estimation . . . 104
19.5 Overfitting and generalization . . . 105
19.6 Laplace smoothing . . . 106
19.7 Features and baselines . . . 107

20 Machine Learning II **108**
20.1 Linear classifiers . . . 108

20.2 The binary perceptron . . . 109
20.3 Bias . . . 110
20.4 The multiclass perceptron . . . 110
20.5 Properties and problems of the perceptron . . . 111
20.6 Logistic regression . . . 111
20.7 Multiclass logistic regression . . . 112

21 Machine Learning III **113**
21.1 Linear regression . . . 113
21.2 Logistic regression . . . 114
21.3 Multiclass logistic regression . . . 115
21.4 Gradient ascent . . . 115

22 Machine Learning: Neural Networks **117**
22.1 Learning the features . . . 117
22.2 Two-layer networks . . . 117
22.3 Deep networks and activations . . . 118
22.4 Training with the log loss . . . 119
22.5 Backpropagation . . . 120
22.6 Properties of neural networks . . . 122

23 Reinforcement Learning: Bandits and Recommendation Systems **123**
23.1 Multi-armed bandits and regret . . . 123
23.2 Greedy and ϵ-greedy algorithms . . . 124
23.3 Lower bound . . . 125
23.4 Upper confidence bounds . . . 125
23.5 Bayesian bandits . . . 127
23.6 Value of information . . . 127
23.7 Contextual bandits . . . 128
23.8 Exploration in MDPs . . . 129

24 Reinforcement Learning I **130**
24.1 Learning in an unknown MDP . . . 130
24.2 Model-based learning . . . 131
24.3 Model-free learning and direct evaluation . . . 132
24.4 Temporal difference learning . . . 133
24.5 Q-learning . . . 134

25 Reinforcement Learning II **135**
25.1 TD value estimation in practice . . . 135
25.2 Exploration and exploitation . . . 136
25.3 Approximate Q-learning . . . 137
25.4 Policy search . . . 138

26 AI Existential Safety **139**
26.1 Reward maximization and control . . . 139
26.2 A magic box and two world-models . . . 139
26.3 Testing the two models . . . 141
26.4 Whether intervention is possible . . . 141
26.5 Consequences and proposed approaches . . . 142

27 Large Language Models **144**
27.1 Tokens and embeddings . . . 144
27.2 Autoregressive generation . . . 144
27.3 Self-attention and transformers . . . 145

27.4 Pre-training, fine-tuning, and instruction tuning 145
27.5 Reinforcement learning from human feedback 146
27.6 Decoding with beam search 147
27.7 Multimodal models and progress 147

Lecture 1: Introduction to AI and Rational Agents

1.1 Agents and their environments

An AGENT observes an ENVIRONMENT through sensors and changes it through actuators. Its observations inform the actions it takes. The environment together with its agents forms the WORLD.

A RATIONAL AGENT chooses actions to obtain the best expected outcome according to its goals or preferences. The criterion concerns what can be expected from the available information, rather than a guarantee that every realized outcome will be favorable.

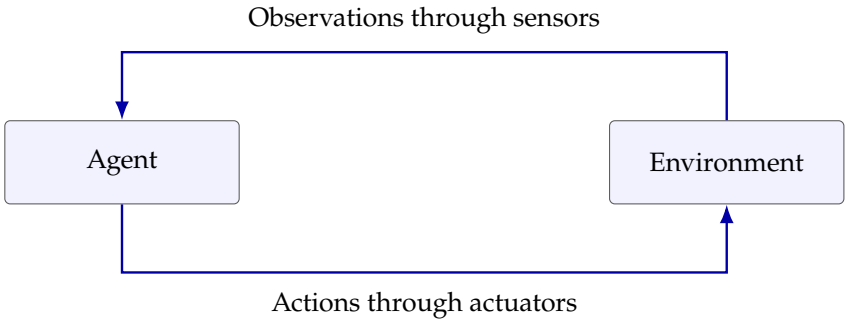


Figure 1: The interaction between an agent and its environment. Sensors supply observations; actuators carry out actions.

For a checkers-playing agent, the environment is the board and opposing player, and an action is a legal piece move. The agent's preferences determine which outcomes it should seek.

1.2 Reflex and planning agents

A REFLEX AGENT selects an action from its current percept, possibly using memory or an estimate of the current state. It does not compare the future consequences of candidate actions.

A PLANNING AGENT uses a model of how actions change the world to reason about possible futures. It predicts the consequences of actions and chooses among them according to its objective. The distinction is between reacting to the present and explicitly looking ahead; retaining memory alone does not make an agent a planner.

1.3 Specifying a task with PEAS

A PEAS DESCRIPTION fixes four components of the task:

Component	What it specifies
Performance measure	How outcomes are evaluated.
Environment	Where the agent acts and what affects it.
Actuators	How the agent changes the environment.
Sensors	How the agent obtains information.

The performance measure supplies the objective; the remaining components describe the circumstances and means of achieving it.

1.4 Properties of the environment

The information available to an agent and the effects of its actions determine what its design must handle.

- FULLY OBSERVABLE versus PARTIALLY OBSERVABLE: does the agent observe the relevant state completely? With partial observations, it must maintain an internal estimate of the state.
- DETERMINISTIC versus STOCHASTIC: does a state and action determine one next state, or a distribution of possible next states? Stochastic transitions require reasoning about uncertain outcomes.
- SINGLE-AGENT versus MULTI-AGENT: do other agents also act in the environment? Their behavior affects the outcome, and randomizing actions can prevent predictability.
- STATIC versus DYNAMIC: can the environment change while the agent is deciding what to do? In a dynamic environment, deliberation may leave observations outdated.
- KNOWN versus UNKNOWN dynamics: is the transition model available to the agent? Unknown dynamics require learning how actions affect the world, potentially through deliberate exploration.

Knowing the dynamics does not remove randomness: an agent can know a stochastic transition model while remaining uncertain about the next outcome.

Source. Nikhil Sharma, *CS 188: Introduction to Artificial Intelligence*, Spring 2024, Note 1: Agents.

Lecture 2: Search: State Spaces and Uninformed Search

Planning requires a model of how actions change the world and a test for whether a goal has been reached. A complete planning method finds a plan whenever one exists; an optimal method finds a plan of minimum cost. An agent can plan once or replan as new observations arrive. A reflex rule can also be rational when its chosen action already maximizes the relevant objective.

2.1 Formulating a search problem

Definition 2.1 [Search problem]. A search problem consists of a state space $\mathcal{S}$, available actions $A(s)$, a transition model $T(s, a)$, action costs $c(s, a)$, a start state s_0, and a goal test Goal(s). A plan is a sequence of actions that takes s_0 to a goal state. Its cost is the sum of its action costs.

The successor function combines the actions, their resulting states, and their costs. In route planning through Romania, states are cities and successors are adjacent cities, with road distances as costs. The start is Arad and the goal test asks whether the current city is Bucharest.

A world state describes the full environment. A search state retains the information needed for the chosen planning problem. This abstraction depends on the objective.

	Pathing	Eat-all-dots
State	Position (x, y)	Position and remaining-dot booleans
Actions	Legal north, south, east, west moves	The same legal moves
Transition	Update position	Update position and consumed dots
Goal test	Reach the destination	No dots remain

For 120 agent positions, four orientations, two ghosts with 12 positions each, and 30 independently represented dot booleans, the full encoding has

$$120 \cdot 4 \cdot 12^2 \cdot 2^{30}$$

possible configurations. The pathing abstraction has only 120 states; eat-all-dots has $120 \cdot 2^{30}$. These are counts of combinations in the encoding; restrictions on reachability can reduce the reachable subset.

To eat all dots while keeping ghosts continuously scared, the search state must also retain the remaining power pellets and the remaining scared time. Information can be omitted only when it is irrelevant to future decisions.

2.2 State-space graphs and search trees

A STATE-SPACE GRAPH has one vertex per state and directed edges for action results. A SEARCH TREE instead unfolds possible plans from the start state: its nodes represent paths, and their labels give the states reached by those paths. Different paths can therefore create different tree nodes with the same state label.

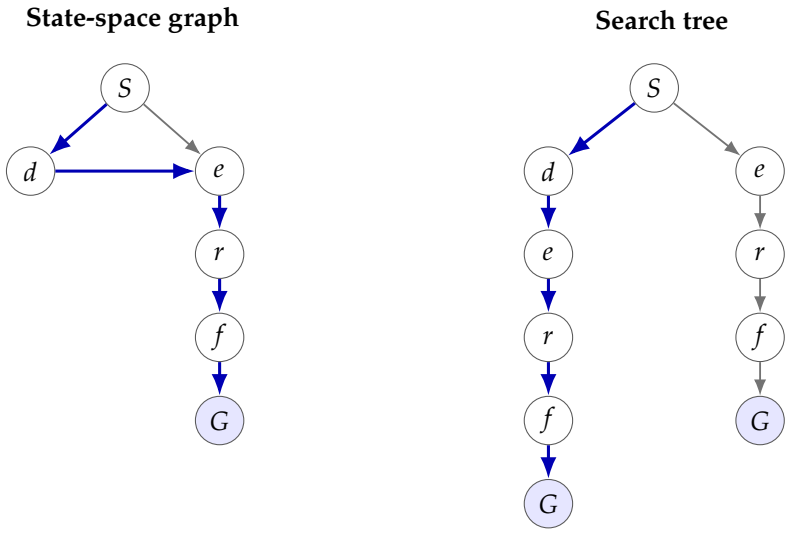


Figure 2: Selected transitions and their search tree. The state e occurs once in the graph but twice in the tree, reached by $S \to e$ and $S \to d \to e$. Blue marks $S \to d \to e \to r \to f \to G$.

Cycles in a finite graph can produce an infinite search tree. Neither structure needs to be constructed in full: generate successors only when a node is expanded, and retain the partial plans currently under consideration.

2.3 A common tree-search procedure

The FRONTIER, or FRINGE, contains generated nodes that have not yet been selected for exploration. A node records its state, parent, incoming action, and cumulative path cost g.

Algorithm 2.1: Tree search

Input: Search problem and an initially empty frontier F with a chosen removal rule.
Output: A plan reaching a goal, or failure if the frontier is exhausted.

Insert the root node $(s_0, \varnothing, \varnothing, 0)$ into F
while F is not empty **do**
  Remove a node n from F according to the removal rule
  if Goal(n.state) **then**
    return the plan obtained by following n's parent links
  for each $a \in A(n.\text{state})$ **do**
    $s' \leftarrow T(n.\text{state}, a)$
    $g' \leftarrow n.g + c(n.\text{state}, a)$
    Insert (s', n, a, g') into F
return failure

The removal rule determines the exploration strategy. This is tree search: it does not suppress repeated states. In particular, the goal test occurs when a node is removed, which matters for cost-sensitive search.

UNINFORMED SEARCH uses the problem's transitions and costs without an estimate of how close a state is to a goal. To compare methods, let $b > 1$ bound the branching factor, m be the maximum tree depth, and s be the shallowest goal depth. A full tree through depth m has

$$1 + b + b^2 + \cdots + b^m = O(b^m)$$

nodes. Time bounds below count tree-node operations, apart from the additional cost of implementing a priority queue.

2.4 Depth-first search

DEPTH-FIRST SEARCH (DFS) removes a deepest frontier node first, using a last-in, first-out STACK. Newly generated children are explored before older siblings.

With finite maximum depth m, DFS may explore the entire tree, taking $O(b^m)$ time. It retains the current branch and pending siblings along it, requiring $O(bm)$ space. It can fail to return when a branch is infinite, even if another branch contains a goal. Thus unrestricted tree-search DFS is not complete. It is also not optimal: the first goal in its exploration order need not be shallow or inexpensive.

2.5 Breadth-first search

Breadth-first search (BFS) removes a shallowest frontier node first, using a first-in, first-out queue. It finishes each depth before exploring the next one.

For finite b and a goal at finite depth s, BFS is complete and finds a shallowest goal. This is a least-cost plan when all action costs are equal and positive; unequal costs break the connection between depth and cost.

There are $1 + b + \cdots + b^s = O(b^s)$ nodes through the goal depth. With the goal test on removal, other nodes at depth s can generate children at depth $s + 1$ before a goal is selected. A bound on generated nodes and frontier storage is therefore $O(b^{s+1})$. The exponential frontier is BFS's main space limitation.

DFS can reach a deep goal quickly if its early branches are favorable; BFS is useful when a shallow solution exists or infinite branches would trap DFS. Their performance depends on the tree and successor order.

2.6 Iterative deepening

Iterative deepening search repeatedly runs depth-limited DFS, increasing the limit until a goal is found. A limit of zero checks only the start; subsequent runs allow depths $1, 2, \ldots$.

Although upper levels are revisited, the deepest level dominates for $b > 1$. To reach depth s, the work is bounded by

$$\sum_{i=0}^{s}(s - i + 1)b^i = b^s \sum_{j=0}^{s} \frac{j + 1}{b^j} = O(b^s).$$

Each run stores only its DFS frontier, giving $O(bs)$ space at the final limit. Finite branching makes it complete, and equal positive action costs make its first solution optimal.

For $b = 10$ and $s = 5$, counting full levels and omitting the root,

$$\begin{aligned} \text{BFS:} \quad & 10 + 100 + 1000 + 10000 + 100000 = 111110, \\ \text{iterative deepening:} \quad & 5 \cdot 10 + 4 \cdot 100 + 3 \cdot 1000 + 2 \cdot 10000 + 100000 = 123450. \end{aligned}$$

The repeated work is modest compared with the storage saved.

2.7 Uniform-cost search

Uniform-cost search (UCS) removes a node with minimum cumulative path cost $g(n)$, using a priority queue. It explores cost contours rather than depth levels. With equal positive action costs, this ordering agrees with BFS's depth ordering, up to ties.

Proposition 2.2 [UCS optimality]. *With nonnegative action costs, a goal returned by UCS has minimum plan cost.*

Proof. Suppose UCS removes a goal of cost C, but a cheaper goal path exists. There is still a frontier node on that path: initially its root is present, and expanding a prefix inserts its next prefix. Nonnegative costs make every prefix of the cheaper path cost at most its final cost, which is less than C. UCS would remove such a prefix before the goal of cost C, a contradiction. □

Optimality on return does not alone guarantee termination. If every action cost is at least $\epsilon > 0$, branching is finite, and an optimal plan has finite cost C^*, only finitely many tree nodes can precede that plan. UCS is then complete. A positive lower bound rules out endlessly exploring ever-longer paths whose costs stay below the goal cost.

Nodes expanded before termination have cost at most C^* and depth at most $d = \lfloor C^*/\epsilon \rfloor$. There are $O(b^d)$ such nodes; their generated children give time and space bounds $O(b^{d+1})$. Thus the effective depth is set by C^*/ϵ. Maintaining a heap adds a logarithmic factor per priority-queue operation.

UCS uses no information about the goal's location, so it can explore many cheap paths in unhelpful directions. DFS, BFS, and UCS share the same tree-search procedure; their frontier removal rules account for their different behavior.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Spring 2024, Lecture 2: Search; Nikhil Sharma, Note 2.

Lecture 3: Search: Informed Search, A* and Heuristics

Informed search uses an estimate of the remaining cost to guide which partial plan to explore. The search problem and successor function stay the same; the frontier ordering changes.

3.1 Heuristics and remaining cost

A heuristic $h(s)$ estimates the cheapest cost from state s to a goal. Let $g(n)$ be the actual cost of the partial plan represented by search node n. We abbreviate $h(n) = h(n.\text{state})$: the heuristic depends on the state, while g depends on the path used to reach it.

For unit-cost movement on a four-neighbor grid, two estimates are

$$h_{\mathrm{M}} = |x - x_G| + |y - y_G|, \qquad h_{\mathrm{E}} = \sqrt{(x - x_G)^2 + (y - y_G)^2}.$$

Manhattan distance ignores walls; Euclidean distance also permits a straight-line route. For horizontal and vertical displacements 10 and 5, these give 15 and $\sqrt{125} \approx 11.2$. In road routing, straight-line distance is similarly a lower bound on driving distance; Arad is 366 km from Bucharest by this estimate.

Example: *Pancake sorting.* A state records pancake sizes from top to bottom. An action reverses the top k pancakes and costs k, rather than one. The goal is increasing order. For example,

$$(4, 1, 2, 3) \xrightarrow{\text{flip 4}} (3, 2, 1, 4) \xrightarrow{\text{flip 3}} (1, 2, 3, 4)$$

has cost $4 + 3 = 7$. The largest pancake still out of place gives a heuristic: 4, then 3, then 0 along this plan. If pancake k is out of place, some future flip must reach its target position k, costing at least k. Thus this estimate never exceeds the remaining sorting cost.

3.2 Greedy search and A*

These methods use the same priority-queue search procedure with different priorities:

Method	Remove a node minimizing	Information used
Uniform-cost search	$g(n)$	Cost already incurred
Greedy search	$h(n)$	Estimated remaining cost
A* search	$f(n) = g(n) + h(n)$	Estimated total plan cost

Greedy search can reach a goal quickly, but ignores what it has already spent. It is not optimal, and on infinite search trees it need not be complete. With straight-line distance, the route Arad–Sibiu–Fagaras–Bucharest costs 450 km, while the route through Rimnicu Vilcea and Pitesti costs 418 km.

For the following illustration, break greedy-search ties in favor of the most recently inserted node.

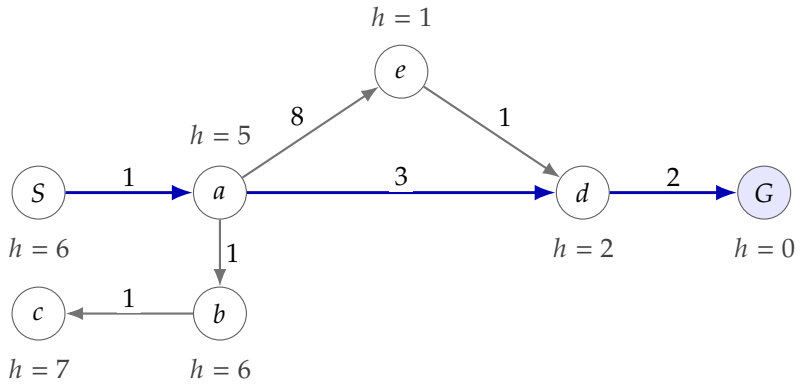


Figure 3: Edge labels are costs; labels beside nodes are heuristic values. The blue route costs 6. Greedy search instead takes S, a, e, d, G, costing 12.

In this graph, UCS explores b and c before reaching d. A* selects a, then d with $f(d) = 4 + 2 = 6$, then the goal of cost 6. The other frontier choices have $f(b) = 2 + 6 = 8$ and $f(e) = 9 + 1 = 10$. Greedy instead chooses e after a because $h(e) = 1$ is smallest.

Note. Test for a goal when removing a node, not when generating it. For paths $S \to A \to G$ with costs $(2, 2)$ and $S \to B \to G$ with costs $(2, 3)$, set $h(A) = 2$, $h(B) = 1$, and $h(G) = 0$. A* explores B first and generates a goal of cost 5, but A remains on the frontier with $f(A) = 4$. Continuing finds the cheaper goal of cost 4.

Arbitrary scores can discard cost information entirely: assigning $h(n) = 1 - g(n)$ makes every priority $f(n) = 1$. Behavior then depends on tie-breaking; FIFO ties produce BFS, which need not minimize unequal costs. This path-dependent score is not a valid nonnegative state heuristic in general.

3.3 Admissibility and A* tree search

Definition 3.1 [Admissible heuristic]. Let $h^*(s)$ be the minimum remaining cost from s to a goal. A heuristic is admissible if

$$0 \le h(s) \le h^*(s) \qquad \text{for every state } s.$$

In particular, $h(G) = 0$ at every goal.

Overestimation can hide a good plan. Suppose $S \to A$ costs 1, $A \to G$ costs 3, and $S \to G$ costs 5. With $h(S) = 7$, $h(A) = 6$, and $h(G) = 0$, A* returns the direct cost-5 goal before exploring A, whose priority is $1 + 6 = 7$. The actual optimum is 4.

Theorem 3.2 [Optimality of A* tree search]. *With an admissible heuristic and nonnegative action costs, a goal returned by A* tree search has minimum plan cost.*

Proof. Let C^* be the cost of an optimal plan. Until an optimal goal is removed, some prefix n of that plan remains on the frontier. Admissibility gives

$$f(n) = g(n) + h(n) \leq g(n) + h^*(n) \leq C^*.$$

Any suboptimal goal B has $f(B) = g(B) > C^*$, since $h(B) = 0$. It cannot be removed while that cheaper prefix remains. Hence a returned goal cannot be suboptimal. □

For completeness on an infinite search space, also assume finite branching, a positive lower bound ϵ on action costs, and a finite-cost optimal plan. Every node selected before that goal has $g(n) \leq f(n) \leq C^*$, so its depth is at most $\lfloor C^*/\epsilon \rfloor$; there are only finitely many such nodes.

UCS explores cost contours in all directions. A* uses its lower bound to concentrate on plausible routes while retaining alternatives needed for optimality. Applications include routing, games, resource and robot-motion planning, translation, and speech recognition.

3.4 Constructing heuristics by relaxation

A relaxed problem removes constraints or permits additional actions without increasing the costs of existing actions. Every original plan is still available, so the exact relaxed optimum is an admissible heuristic for the original problem. Manhattan distance on a maze is the exact distance after removing its walls.

In the 8-puzzle, a state places eight labeled tiles and one blank on a 3×3 board. A unit-cost action slides a tile adjacent to the blank into it. The full encoding has 9! arrangements, and the center blank in the displayed start state permits four actions.

Start

7	2	4
5		6
8	3	1

Goal

	1	2
3	4	5
6	7	8

Figure 4: The 8-puzzle example. The blank is excluded from both heuristic sums: the start has eight misplaced tiles and total Manhattan distance 18.

Two admissible estimates are

$$h_{\text{tiles}}(s) = \#\{\text{numbered tiles outside their goal positions}\},$$
$$h_{\text{Manhattan}}(s) = \sum_{t=1}^{8} \left(|x_t - x_t^G| + |y_t - y_t^G| \right).$$

Each misplaced tile must move at least once. Each tile must also travel at least its Manhattan displacement, and one action moves only one tile by one square. These give the two lower bounds. Equivalently, allow each tile to move to its goal directly, or to move freely through other tiles while paying one per grid step.

The following illustrative averages from Andrew Moore count expanded nodes by optimal solution depth:

Search / heuristic	4 steps	8 steps	12 steps
UCS	112	6,300	3.6×10^6
A* / misplaced tiles	13	39	227
A* / Manhattan distance	12	25	73

The exact heuristic h^* is admissible too, but computing it may require solving the original problem. Search effort trades off against the work needed to evaluate the heuristic.

3.5 Dominance and combining estimates

For admissible heuristics, h_a DOMINATES h_b when $h_a(s) \geq h_b(s)$ for every state. Its lower bound is at least as informative. In the 8-puzzle,

$$0 \leq h_{\text{tiles}} \leq h_{\text{Manhattan}} \leq h^*.$$

The zero heuristic reduces A* to UCS. Larger lower bounds can reduce expansions, though evaluation cost and tie-breaking also affect performance.

Proposition 3.3 [MAXIMUM OF ADMISSIBLE HEURISTICS]. *For admissible $h_1, \ldots, h_k$, $h(s) = \max_i h_i(s)$ is admissible and dominates every h_i.*

Proof. Every $h_i(s)$ lies in $[0, h^*(s)]$, so their maximum does too. By definition, it is at least each individual value. □

Thus pointwise maximum is the least upper bound in the dominance order. The zero heuristic is the bottom element and h^* the top.

3.6 Graph search and repeated states

Different paths can reach the same state, creating repeated work or infinite loops in tree search. GRAPH SEARCH maintains a CLOSED SET of expanded states and skips later copies of them. Use a hash set for expected constant-time membership, rather than a list. Suppressing repeated states preserves reachability because a state's available continuations do not depend on which path reached it.

Algorithm 3.1: A* graph search without reopening closed states

```
Input: Search problem and heuristic h.
Output: A goal plan, or failure if the frontier is exhausted.
C ← ∅; insert the root node with g = 0 into frontier F
while F is not empty do
    Remove a node n minimizing n.g + h(n.state)
    if n.state ∉ C then
        if Goal(n.state) then
            return n's plan
        Add n.state to C
        for each a ∈ A(n.state) do
            s' ← T(n.state, a)
            g' ← n.g + c(n.state, a)
            Insert child (s', n, a, g') into F
return failure
```

The closed set records states on expansion, not on first generation. However, even this rule can lose optimality with an admissible heuristic.

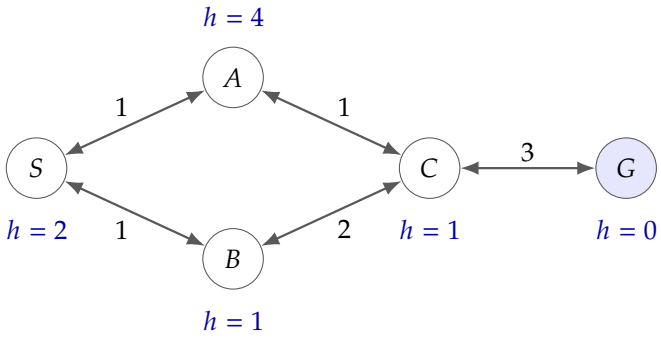


Figure 5: An admissible but inconsistent heuristic. Edge costs apply in both directions. Closing C through B prevents the later, cheaper arrival through A from being expanded.

The expansion order is S, B, C, A, G. The first arrival at C has $g(C) = 3$ and $f(C) = 4$, beating $f(A) = 1 + 4 = 5$. Expanding A later generates a cheaper arrival at C with $g = 2$, but C is already closed. The returned route costs $1 + 2 + 3 = 6$; the optimum $S \to A \to C \to G$ costs $1 + 1 + 3 = 5$.

3.7 Consistency and A* graph search

Definition 3.4 [Consistent heuristic]. A nonnegative heuristic, zero at every goal, is consistent if for every directed edge $u \to v$ of cost $c(u, v)$,

$$h(u) \le c(u, v) + h(v).$$

For a bidirectional edge, the condition must hold in both directions.

Consistency bounds the drop in heuristic value across each action. In the preceding example, $h(A) - h(C) = 4 - 1 = 3 > 1 = c(A, C)$. Replacing $h(A)$ by 2 makes the displayed heuristic consistent.

Consistency implies admissibility: summing its inequalities along any path to a goal G gives $h(s) \le \text{cost}(s \rightsquigarrow G) + h(G)$. Since $h(G) = 0$, taking a cheapest such path yields $h(s) \le h^*(s)$. It also makes f nondecreasing along a path: for a child n' of n,

$$f(n') = g(n) + c(n, n') + h(n') \ge g(n) + h(n) = f(n).$$

Theorem 3.5 [Optimality of A* graph search]. *With a consistent heuristic and nonnegative costs, a state is first expanded along a cheapest path to it. Consequently, a returned goal is optimal even when closed states are never reopened.*

Proof. Suppose v is the first state closed with a suboptimal path cost $g(v) > g^*(v)$, where g^* is shortest-path cost from the start. On a shortest path to v, let u be the first state not yet closed. Its predecessor was closed optimally and inserted a node for u with cost $g^*(u)$, so that node is still on the frontier. Consistency along the remaining path gives

$$g^*(u) + h(u) \le g^*(v) + h(v) < g(v) + h(v).$$

That node must be selected before the suboptimal node for v, a contradiction. A selected goal therefore has optimal path cost. □

Both 8-puzzle heuristics are consistent: one unit-cost move changes each estimate by at most one. Pointwise maximum also preserves consistency. If i maximizes the value at u, then

$$\max_j h_j(u) = h_i(u) \le c(u, v) + h_i(v) \le c(u, v) + \max_j h_j(v).$$

Thus combining consistent heuristics by maximum retains the graph-search guarantee. These guarantees concern the search model; even an optimal modeled plan can fail if its transitions or costs misdescribe the real world.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 3 (Spring 2024); Nikhil Sharma, Note 3.

Lecture 4: Search: Local Search

4.1 A* heuristics and memory

A useful heuristic often combines several simple lower bounds by the pointwise maximum, which stays admissible.

Example: *Knight's moves.* Count the fewest knight moves from S to G, whose displacement is $(\Delta x, \Delta y)$. One move changes the Manhattan distance to G by at most 3, the Euclidean distance by at most $\sqrt{5}$, and $\max(|\Delta x|, |\Delta y|)$ by at most 2. Hence

$$h_1 = \frac{|\Delta x| + |\Delta y|}{3}, \qquad h_2 = \frac{\sqrt{\Delta x^2 + \Delta y^2}}{\sqrt{5}}, \qquad h_3 = \frac{\max(|\Delta x|, |\Delta y|)}{2}$$

are lower bounds. Every move changes the square's color, so the move count is even exactly when S and G have the same color. Rounding each bound up to the next integer of this parity keeps it admissible, and so does the maximum $h = \max(h'_1, h'_2, h'_3)$ of the rounded bounds. Between opposite corners of a 5×10 board, $(\Delta x, \Delta y) = (4, 9)$ gives 4.33, 4.40, and 4.5; all round up to the odd value 5, which is the true minimum: $(1, 2), (1, 2), (1, 2), (-1, 2), (2, 1)$.

Natural admissible heuristics, especially exact costs of relaxed problems, are usually consistent as well. With $h = 0$, A* graph search is UCS, which is therefore optimal.

A* stores every generated node, so it usually runs out of memory before time. Three variants trade time for space:

- IDA* runs iterative deepening with an f-limit instead of a depth limit. The next limit is the smallest f-value that exceeded the current one. This is inefficient when f is real-valued and nearly every node has a distinct value, since each iteration then adds few nodes.
- RBFS is a recursive depth-first search whose f-limit is the best f-value of any alternative path available from an ancestor of the current node. When the limit is exceeded, the recursion unwinds but remembers the best f-value reachable on the abandoned branch.
- SMA* uses all available memory for the queue. When memory is full, it drops the worst node but records its value in the parent, which limits repeated regeneration (thrashing).

4.2 Local search problems

In many problems the path is irrelevant: the goal configuration itself is the solution. A completed Sudoku grid, for instance, tells us how to reach it. The state space is then a set of complete configurations, and we seek one that satisfies constraints, as in n-queens, or optimizes an objective,

as in the traveling salesperson problem.

Local search (iterative improvement) keeps a single current state, or a few, and tries to improve it by moving to neighboring states. It uses constant space, works online as well as offline, and is largely unavoidable when the state being improved is the agent itself, as in learning. Below, $V(s)$ is the objective to maximize; minimizing a cost h is the same as maximizing $V = -h$.

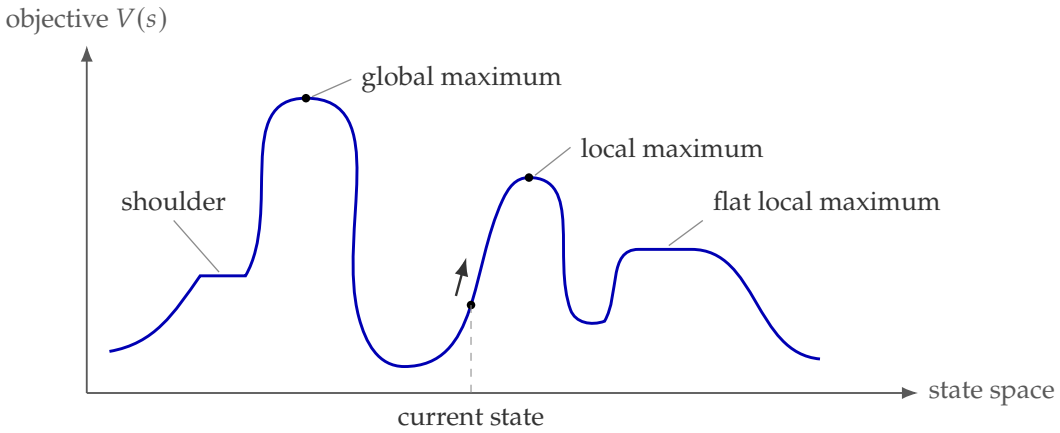


Figure 6: A one-dimensional landscape. Moving uphill from the current state reaches the local maximum, not the global one. Plateaus include flat local maxima and shoulders, from which further progress is possible.

4.3 Hill climbing

Hill climbing (steepest ascent) starts anywhere, repeatedly moves to the best neighbor, and stops when no neighbor is better. It keeps no search tree, only the current state and its value: "like climbing Everest in thick fog with amnesia."

Algorithm 4.1: Hill climbing

Input: Initial state s_0, successors of each state, objective V.
Output: A state with no better successor.

```
s ← s_0
while true do
    s' ← a successor of s maximizing V
    if V(s') ≤ V(s) then
        return s
    s ← s'
```

For n-queens, a state places one queen in each column, an action moves a queen within its column, and h counts attacking pairs: queens sharing a row, column, or diagonal. Hill climbing minimizes h; a goal has $h = 0$.

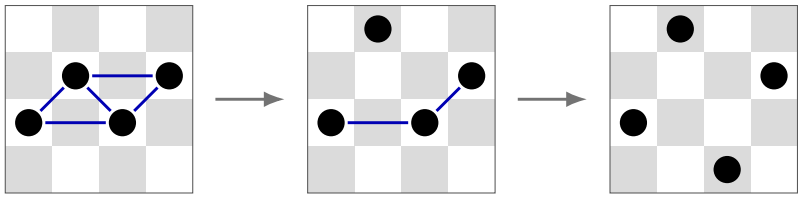


Figure 7: Hill climbing on 4-queens. Lines join attacking pairs. Each arrow moves one queen within its column to a best successor.

Greedy improvement can stop at a LOCAL MAXIMUM, which looks like a global maximum from nearby states, or on a PLATEAU. A plateau is a FLAT LOCAL MAXIMUM if no direction improves, or a SHOULDER if progress resumes after a flat stretch. Hill climbing is therefore incomplete. Common variants are:

- STOCHASTIC HILL CLIMBING: choose randomly among uphill moves. In practice it reaches higher maxima, using more iterations.
- SIDEWAYS MOVES: allow moves of equal value. These escape shoulders but loop forever on flat local maxima, so their number is capped.
- RANDOM-RESTART HILL CLIMBING: rerun from random initial states. If each run succeeds with probability $p > 0$, the number of runs is geometric with mean $1/p$, so a global maximum is found with probability 1.

On 8-queens from random starts, plain hill climbing succeeds with probability $p = 0.14$, averaging 4 moves when it succeeds and 3 when it gets stuck. The expected number of failed runs before success is $(1 - p)/p$, so random restarts need about

$$3 \cdot \frac{1-p}{p} + 4 \approx 22$$

moves in total. Allowing up to 100 sideways moves raises p to 0.94, with averages of 21 and 65 moves, giving $65(1-p)/p + 21 \approx 25$. Each run is far more reliable but longer, and the result depends on a tuning parameter.

4.4 Simulated annealing

SIMULATED ANNEALING combines a random walk with hill climbing, by analogy with slowly cooling a metal into an ordered state. It proposes a random move, always accepts improvements, and accepts a worse move with a probability set by the temperature T. A high temperature permits many bad moves and shakes the state out of local maxima. A schedule lowers T gradually.

Algorithm 4.2: Simulated annealing

Input: Initial state s_0, objective V, schedule $T(t) \geq 0$.
Output: The current state when the temperature reaches zero.

```
s ← s_0
for t = 1, 2, ... do
    if T(t) = 0 then
        return s
    s' ← a random successor of s
    Δ ← V(s') − V(s)
    if Δ > 0 then
        s ← s'
    else
        s ← s' with probability e^{Δ/T(t)}
```

Proposition 4.1 [Boltzmann equilibrium]. *Fix $T > 0$. Suppose the neighbor relation is symmetric, every state has D neighbors, and the proposal is uniform over them. Then*

$$P(x) \propto e^{V(x)/T}$$

is a stationary distribution of simulated annealing.

Proof. Take neighbors x, y with $V(y) \geq V(x)$. The uphill move is always accepted, while the downhill move is accepted with probability $e^{(V(x)-V(y))/T}$:

$$P(x \to y) = \frac{1}{D}, \qquad P(y \to x) = \frac{1}{D} e^{(V(x)-V(y))/T}.$$

With $P(s) = e^{V(s)/T}/Z$, both $P(x)P(x \to y)$ and $P(y)P(y \to x)$ equal $e^{V(x)/T}/(ZD)$. This detailed balance holds on every edge. Rejected proposals leave the state unchanged, so summing over predecessors gives $\sum_y P(y)P(y \to x) = P(x) \sum_y P(x \to y) = P(x)$. □

The ratio $P(y)/P(x) = e^{(V(y)-V(x))/T}$ grows without bound as $T \to 0$, so on a connected state graph the equilibrium concentrates on global maxima. If T decreases slowly enough, the state converges to an optimum with probability approaching 1. The guarantee is weaker than it sounds. Escaping a local maximum may require many consecutive downhill steps, each increasingly unlikely; "slowly enough" can mean exponentially slowly; and random-restart hill climbing has a similar guarantee. Simulated annealing and its relatives are still standard tools for VLSI layout and other configuration problems.

4.5 Local beam search

Local beam search keeps K states, initialized randomly. Each iteration generates all successors of all K states and keeps the best K of the combined list. It stops when any state reaches the optimal value. Stochastic beam search instead samples K successors with a bias toward good ones.

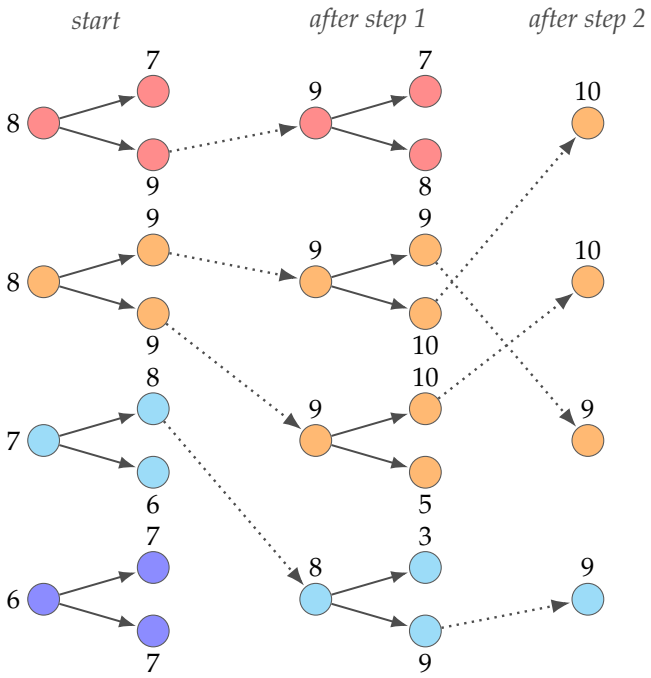


Figure 8: Local beam search with $K = 4$. Numbers are objective values. Colors mark the initial state each state descends from, and dotted arrows show the K best successors kept at each step.

This is not K independent hill climbers. Selection is shared, so productive regions attract the search. In the figure, the lowest initial state is dropped after one step, and after two steps three of the four states descend from one initial state. Beam search can still stall on flat regions; stochastic selection helps, much as in stochastic hill climbing.

4.6 Genetic algorithms

Genetic algorithms are stochastic beam search with a natural selection metaphor. The K states, called the population, are individuals encoded as strings over a finite alphabet. Each step evaluates a nonnegative fitness function, samples parents with probability proportional to fitness, combines pairs by crossover at a random point, and applies occasional random mutation.

For 8-queens, digit i is the row of the queen in column i, from 1 at the bottom. A natural fitness is the number of nonattacking pairs, out of $\binom{8}{2} = 28$, so a solution has fitness 28. In the generation below, the total fitness is 78. Bars mark crossover points, so the first pair yields 327|48552 and 247|52411. Bold digits are mutations.

Population	Fitness	Selection	Parents	Offspring	Mutated
24748552	24	31%	327|52411	32748552	32748**1**52
32752411	23	29%	247|48552	24752411	24752411
24415124	20	26%	32752|411	32752124	32**2**52124
32543213	11	14%	24415|124	24415411	2441541**7**

A mutation moves one queen within its column, a single hill-climbing action. Crossover keeps the left columns of one parent and the right columns of the other. It helps when blocks of the string are good partial solutions, such as nonattacking groups of columns, which evolved separately and can be combined into a high-scoring whole.

Algorithm 4.3: Genetic algorithm with one child per mating

Input: Population P of K strings of length n, fitness $F \geq 0$.
Output: The fittest individual in the final population.

while no individual is fit enough and time remains **do**
  $P' \leftarrow \varnothing$
  for $i = 1$ **to** K **do**
    sample parents x, y from P with probability proportional to F
    choose a crossover point c uniformly from $\{1, \ldots, n\}$
    $z \leftarrow x_1 \cdots x_c \, y_{c+1} \cdots y_n$
    With small probability, mutate a random position of z
    Add z to P'
  $P \leftarrow P'$
return the individual in P maximizing F

The 8-queens example produces two complementary children per pair; this common version keeps one.

4.7 Continuous state spaces

Suppose we place three airports in Romania to minimize the sum of squared distances from each city to its nearest airport. The state is $\mathbf{x} = (x_1, y_1, x_2, y_2, x_3, y_3)$. With city locations (x_c, y_c) and C_a the set of cities nearest airport a,

$$f(\mathbf{x}) = \sum_{a=1}^{3} \sum_{c \in C_a} \big((x_a - x_c)^2 + (y_a - y_c)^2\big).$$

A continuous state or action space can be handled in three ways:

1. *Discretize*: use a grid with increment δ and any discrete local search algorithm.
2. *Random perturbations*: FIRST-CHOICE HILL CLIMBING samples perturbations until one improves the state; simulated annealing also works with random perturbations.
3. *Gradients*: compute $\nabla f(\mathbf{x}) = (\partial f/\partial x_1, \partial f/\partial y_1, \partial f/\partial x_2, \ldots)^\top$ analytically.

Away from ties, small moves do not change the assignments C_a, so

$$\frac{\partial f}{\partial x_1} = \sum_{c \in C_1} 2(x_1 - x_c).$$

An extremum has $\nabla f(\mathbf{x}) = 0$. Here this has a closed form: each airport sits at the centroid of its cities, for example $x_1 = \sum_{c \in C_1} x_c / |C_1|$. For fixed assignments f is a convex quadratic minimized by these centroids. Without ties, the assignments stay fixed under small moves, so this is a local minimum of f. Other assignments can give other local minima, so it need not be global.

When $\nabla f = 0$ has no closed form, GRADIENT DESCENT repeats

$$\mathbf{x} \leftarrow \mathbf{x} - \alpha \nabla f(\mathbf{x})$$

with step size $\alpha > 0$. This is hill climbing that minimizes a cost. Many gradient-based methods refine this idea, and many machine learning algorithms are local searches of this kind.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 4 (Spring 2024); Nikhil Sharma, Note 4.

Lecture 5: Games: Trees, Minimax and Pruning

5.1 Games and adversarial search

A game is a task environment with more than one agent. Games vary along several axes: deterministic or stochastic outcomes, perfect information (full observability) or not, two or more players, teams or individuals, turn-taking or simultaneous moves, and zero-sum or not. Search no longer returns a fixed plan, since the other agents' replies are unknown in advance. Instead we compute a strategy or policy that recommends a move from every possible state.

One formalization of a deterministic game has

- states S with initial state s_0, and players $P = \{1, \ldots, N\}$, usually taking turns;
- actions A, which may depend on the player and state, and a transition function $S \times A \to S$;
- a terminal test $S \to \{\text{true}, \text{false}\}$ and terminal utilities $S \times P \to \mathbb{R}$.

A solution for a player is a policy $S \to A$.

In a zero-sum game the agents have opposite utilities: one maximizes a single value and the other minimizes it, so the competition is pure. In a general-sum game the agents have independent utilities, and cooperation, indifference, competition, and shifting alliances are all possible. A team game gives a common payoff to all members of a team.

Game playing has a long record in AI. Checkers had a computer player in 1950, a self-taught program in 1959, a computer world champion in 1995, and was solved in 2007: every position is known to be a win, loss, or draw under optimal play. In chess, Deep Blue defeated Garry Kasparov in 1997; by 2024 the program Stockfish was rated 3631, against 2847 for Magnus Carlsen. Go, with branching factor above 300, reached strong amateur level with Monte Carlo tree search between 2005 and 2014. AlphaGo defeated human world champions in 2016 and 2017, beating Lee Sedol 4 games to 1 in March 2016, though in 2022 a human exploited a weakness of the neural networks to defeat top Go programs.

5.2 State values and minimax

The value of a state is the best achievable outcome (utility) from that state for the agent controlling it. Terminal states have known terminal utilities. With a single agent, a nonterminal value is the maximum over successors:

$$V(s) = \max_{s' \in \text{successors}(s)} V(s').$$

For example, let Pacman start with 10 points, lose 1 per move, and stop when he eats a pellet two squares to his right. Moving straight to it scores 8; the best continuation after first moving left scores 6. So the right child has value 8, the left has value 6, and Pacman moves right.

Definition 5.1 [MINIMAX VALUE]. In a two-player zero-sum game between MAX and MIN,

$$V(s) = \begin{cases} \text{its utility}, & s \text{ terminal}, \\ \max_{s' \in \text{successors}(s)} V(s'), & \text{MAX to move}, \\ \min_{s' \in \text{successors}(s)} V(s'), & \text{MIN to move}. \end{cases}$$

This is the best utility MAX can guarantee against a rational (optimal) adversary.

Add a ghost to the Pacman example and truncate the tree after one move by each agent, with terminal scores $-8, -5$ after Pacman moves left and $-10, +8$ after he moves right. The ghost nodes have values $\min(-8, -5) = -8$ and $\min(-10, 8) = -10$, so the root has value -8. Pacman moves away from the pellet: the $+8$ outcome exists, but an optimal ghost will not allow it. Simple computation produces this hedging behavior. In tic-tac-toe, the terminal utilities are -1, 0, and $+1$ for a loss, draw, and win by MAX.

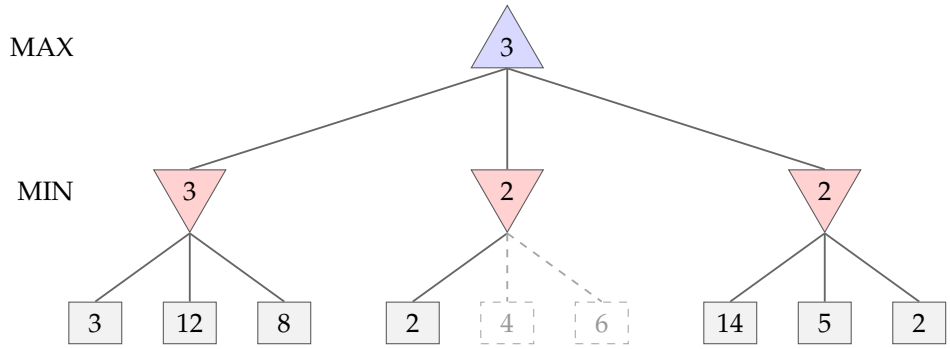


Figure 9: Minimax values. The MIN nodes take $\min(3, 12, 8) = 3$, $\min(2, 4, 6) = 2$, and $\min(14, 5, 2) = 2$, and the root takes the maximum, 3. Alpha-beta pruning never examines the gray leaves.

Minimax evaluates the tree by a postorder depth-first traversal, from the leftmost leaf rightward, and returns the root action whose child supplied the root value. Like exhaustive DFS it takes time $O(b^m)$ and space $O(bm)$ for branching factor b and depth m. Chess has $b \approx 35$ and $m \approx 100$, so an exact solution is infeasible.

Minimax is optimal against a perfect opponent but can be overly cautious otherwise. If MAX chooses between MIN nodes with leaves $(10, 10)$ and $(9, 100)$, minimax picks the left branch with value 10, even when the opponent is unlikely to find the 9.

5.3 Multiple players and general-sum games

Minimax generalizes to games that are not zero-sum or have more than two players. Terminals carry UTILITY TUPLES, with one component per player, and node values are tuples too. The player to move picks the child whose tuple is largest in its own component. Cooperation and competition can then arise dynamically from the computation.

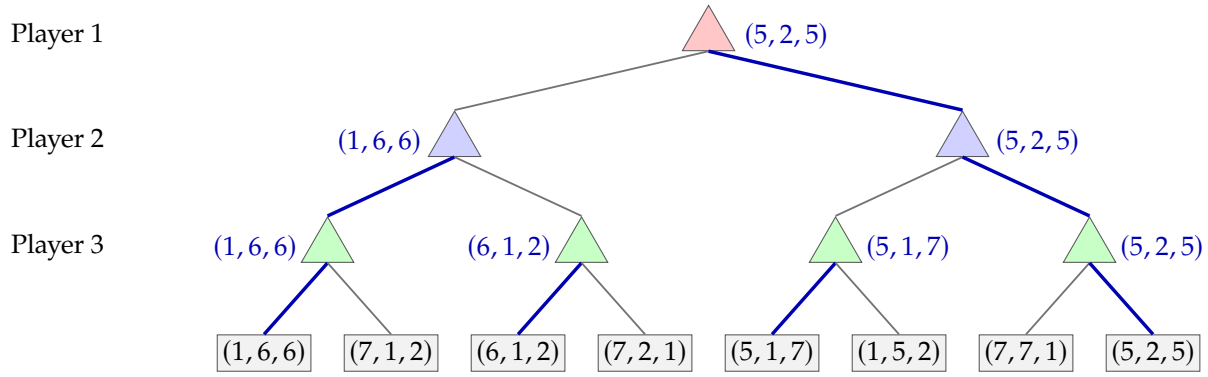


Figure 10: Three players moving in turn. Each node shows the backed-up tuple, and blue edges mark the child chosen by the player to move. The root value is $(5, 2, 5)$.

The selected outcome $(5, 2, 5)$ is reasonable for players 1 and 3 together. Player 3 would prefer $(5, 1, 7)$, but player 2 avoids it.

5.4 Alpha-beta pruning

Minimax need not explore the whole tree. In figure 9, once the middle MIN node sees the leaf 2, its value is at most 2. MAX at the root already has 3 from the left subtree, so it will not choose the middle node whatever its remaining leaves are, and leaves 4 and 6 can be skipped. The value at the root is unchanged.

In general, suppose we are computing the value of a MIN node n by looping over its children, so its running minimum is dropping. Only MAX cares about n's value. Let α be the best value MAX can already obtain at any choice point on the current path from the root. Once n's running minimum is at most α, MAX will avoid n, so its other children can be pruned. Pruning children of a MAX node is symmetric, with β the best value MIN can already obtain on the path.

Algorithm 5.1: Minimax value with alpha-beta pruning

Input: State s; α, MAX's best option on the path to the root; β, MIN's best option on the path.
Output: The value of s (exact when it lies strictly between α and β).

```
if s is terminal then
    return the utility of s
if MAX is to move in s then
    v ← −∞
    for each successor s′ of s do
        v ← max(v, Value(s′, α, β))
        if v ≥ β then
            return v
        α ← max(α, v)
else
    v ← +∞
    for each successor s′ of s do
        v ← min(v, Value(s′, α, β))
        if v ≤ α then
            return v
        β ← min(β, v)
return v
```

The search starts with $\textsc{Value}(s_0, -\infty, +\infty)$. Deleting the two pruning tests gives plain minimax.

Proposition 5.2 [Correctness of alpha-beta]. *Let $v = \textsc{Value}(s, \alpha, \beta)$ with $\alpha < \beta$. If $\alpha < V(s) < \beta$ then $v = V(s)$; if $V(s) \le \alpha$ then $v \le \alpha$; and if $V(s) \ge \beta$ then $v \ge \beta$. In particular, the call at the root returns the minimax value.*

Proof. Induct on the height of s; terminals are exact. Take a MAX node (MIN is symmetric). The kth child is searched with window (α_k, β), where $\alpha_k = \max(\alpha, v_{k-1})$ and v_{k-1} is the running maximum. The loop stops once $v \ge \beta$, so $\alpha_k < \beta$ for every child searched.
If some child has $V(c) \ge \beta$, then either the loop already stopped with $v \ge \beta$, or that child is searched and returns at least β by induction. Either way $v \ge \beta$, which covers $V(s) \ge \beta$.
Otherwise every $V(c_k) < \beta$. By induction, a child returns at most α_k if $V(c_k) \le \alpha_k$, and returns $V(c_k)$ otherwise. In both cases $\max(\alpha_k, r_k) = \max(\alpha_k, V(c_k))$, where r_k is the returned value, and nothing reaches β. By induction on k, the final v satisfies $\max(\alpha, v) = \max(\alpha, V(s))$. If $V(s) > \alpha$ this forces $v = V(s)$; if $V(s) \le \alpha$ it gives $v \le \alpha$. □

Intermediate values can be wrong, including those of the root's children. If the root's left MIN child has the single leaf 10 and its right MIN child has leaves 10 and 0, the right child stops after its first leaf, since $10 \le \alpha = 10$, and reports 10 although its true value is 0. Choosing the root action by comparing reported child values could therefore pick the wrong move. Instead, record the child that last strictly increased v at the root. At the root $\alpha_k = v_{k-1}$, so such a child returned more than α_k, which the proposition shows is its exact value; the last one achieves $V(s_0)$.

The amount of pruning depends on child order: more is possible when good moves come first. With perfect ordering the time drops to $O(b^{m/2})$, doubling the solvable depth, though full search of chess remains hopeless. Deciding what to compute in this way is a simple example of METAREASONING.

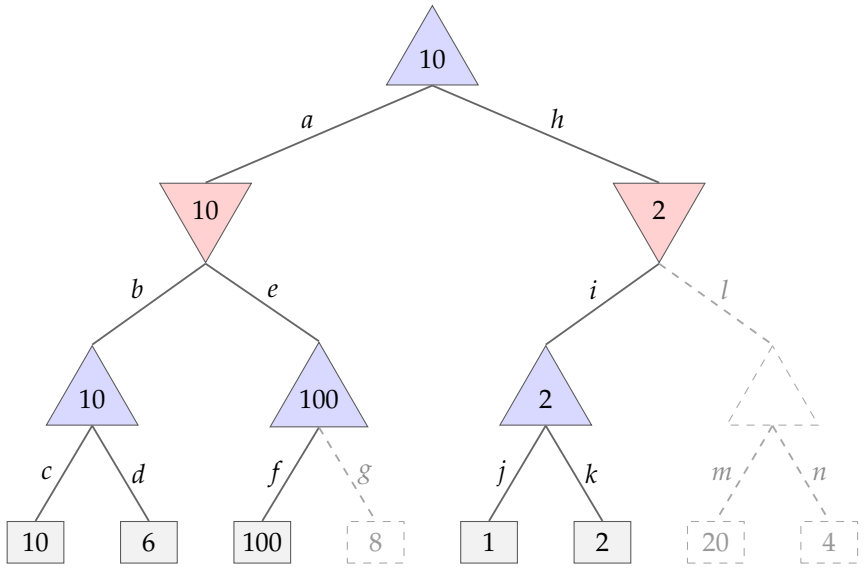


Figure 11: Alpha-beta on a deeper tree, searched left to right. Node labels are the values returned. Branch g and the whole subtree under l are pruned.

In figure 11, the left MIN node first learns 10 from b, so $\beta = 10$ inside e. Leaf $f = 100$ already reaches β, so g is pruned and the left MIN node has value 10. At the root $\alpha = 10$. Under h, branch i returns $\max(1, 2) = 2 \leq \alpha$, so the right MIN node is worth at most 2 and l is pruned. The root value is 10, reached along a. In the two-level tree with MIN children $(10, 8)$ and $(4, 50)$, similarly, the left value 8 lets the right node stop at 4, pruning 50.

5.5 Depth-limited search and evaluation functions

Realistic games cannot be searched to the leaves. DEPTH-LIMITED SEARCH stops at a fixed depth and replaces terminal utilities with an EVALUATION FUNCTION for nonterminal positions. With 100 seconds per move at 10,000 nodes per second, a program can examine 10^6 nodes. For chess, $35^d = 10^6$ gives $d \approx 3.9$ for plain minimax, while perfectly ordered alpha-beta reaches $d \approx 7.8$, about 8 plies, enough for a decent program. Optimal play is no longer guaranteed, but extra plies make a large difference. Iterative deepening yields an ANYTIME ALGORITHM: it always holds the result of the deepest completed search.

For example, if the depth limit is reached at estimates $-1, -2$ below the left MIN node and $4, 9$ below the right, the MIN nodes get -2 and 4 and the root gets 4.

The ideal evaluation function returns the true minimax value of the position. In practice it is often a

weighted linear sum of FEATURES,

$$\text{Eval}(s) = w_1 f_1(s) + w_2 f_2(s) + \cdots + w_n f_n(s),$$

for example $f_1(s)$ = (number of white queens) − (number of black queens) in chess. A checkers evaluation might be

$$\text{Eval}(s) = 2 \cdot \text{agent kings} + \text{agent pawns} - 2 \cdot \text{opponent kings} - \text{opponent pawns},$$

weighting kings more heavily and opponent pieces negatively. Nonlinear functions, such as neural networks trained by self-play reinforcement learning, are also common. Evaluation functions play the role that heuristics play in ordinary search: better states should receive higher scores as consistently as possible.

Depth-limited replanning can misbehave. Suppose Pacman's horizon is two moves and a dot lies beside him. Eating it now, then moving away, scores the same as moving away and returning to eat it, and nothing else scores within the horizon. Waiting therefore looks as good as eating, so he may step away, replan, step back, and repeat indefinitely. This is a danger of replanning agents; a better evaluation function fixes it.

Evaluation functions are always imperfect, but the deeper they are buried in the tree, the less their quality matters. This trades complexity of features against complexity of computation.

Evaluation functions also help alpha-beta. Expanding the most promising children first, as ranked by the evaluation function, makes a good alternative on the path to the root likely to exist early, much as a heuristic guides A*. Moreover, a MIN node's running value only decreases. If an evaluation function gives an upper bound on a MIN node's value, and that bound is already below MAX's best option on the path to the root, the node can be pruned without expanding it. The roles are swapped for MAX nodes.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 5 (Spring 2024); Nikhil Sharma, Note 5.

Lecture 6: Games: Expectimax and Monte Carlo Tree Search

6.1 Uncertain outcomes and expectimax

Minimax assumes a worst-case adversary and can be too pessimistic when outcomes are controlled by chance. If MAX chooses between leaves $(10, 10)$ and $(9, 100)$ resolved by MIN, minimax takes the left branch with value 10. If each leaf is equally likely instead, the right branch is worth $(9 + 100)/2 = 54.5$. Uncertainty arises from explicit randomness, such as dice; from unpredictable opponents, such as ghosts that move randomly; and from actions that can fail, such as a robot whose wheels slip. Values should then reflect average-case rather than worst-case outcomes.

A random variable represents an event whose outcome is unknown, and a probability distribution assigns nonnegative weights summing to one to its outcomes. For traffic $T \in \{\text{none}, \text{light}, \text{heavy}\}$, we might have $\mathbb{P}(T = \text{heavy}) = 0.25$, while $\mathbb{P}(T = \text{heavy} \mid \text{Hour} = 8\text{am}) = 0.60$: evidence can change probabilities. The expected value of a function of a random variable is its probability-weighted average. Trip times of 20, 30, and 60 minutes with probabilities 0.25, 0.50, and 0.25 have expectation $0.25 \cdot 20 + 0.50 \cdot 30 + 0.25 \cdot 60 = 35$ minutes.

Definition 6.1 [Expectimax value]. Expectimax search replaces MIN nodes by chance nodes whose value is the expected value of their children:

$$V(s) = \begin{cases} \text{its utility}, & s \text{ terminal}, \\ \max_{s' \in \text{successors}(s)} V(s'), & \text{MAX to move}, \\ \sum_{s' \in \text{successors}(s)} p(s' \mid s)\, V(s'), & s \text{ a chance node}. \end{cases}$$

Here $p(s' \mid s)$ is the probability that a random outcome or an opponent's choice leads from s to s'.

Expectimax computes the average score under optimal play by MAX. Minimax is the special case in which every chance node puts probability 1 on its lowest-valued child. Later, problems with uncertain results are formalized as Markov decision processes.

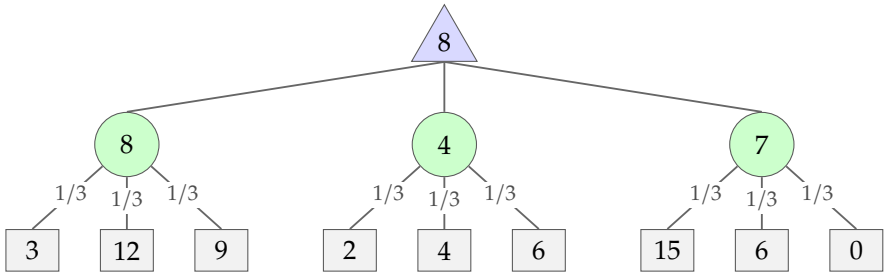


Figure 12: Uniform chance nodes average their leaves to 8, 4, and 7, so the root value is 8.

A chance node with children 8, 24, and −12 and probabilities $\frac{1}{2}$, $\frac{1}{3}$, and $\frac{1}{6}$ has value

$$\tfrac{1}{2} \cdot 8 + \tfrac{1}{3} \cdot 24 + \tfrac{1}{6} \cdot (-12) = 4 + 8 - 2 = 10.$$

Algorithm 6.1: Expectimax value

Input: State s, with a probability for each successor of a chance node.
Output: The expectimax value $V(s)$.

```
if s is terminal then
    return the utility of s
if MAX is to move in s then
    v ← −∞
    for each successor s′ of s do
        v ← max(v, Value(s′))
else
    v ← 0
    for each successor s′ of s do
        v ← v + p(s′ | s) Value(s′)
return v
```

6.2 Pruning and depth limits

Expectimax cannot prune in general. In figure 12, after the first chance node gives 8, seeing the leaf 2 under the second does not settle anything: an unseen child could be arbitrarily large and raise the average above 8. A single value can move an expectation arbitrarily far, unlike a minimum or maximum. Pruning becomes possible only with known finite bounds on values. If every value is at most U, a chance node whose seen children have total probability q and weighted sum w is worth at most $w + (1 - q)U$; once this is no better than MAX's best alternative, its remaining children can be skipped.

Depth-limited expectimax works as for minimax. An evaluation function estimates the true expectimax value at the depth limit, since computing that value exactly would require a great deal of further search.

6.3 Choosing the probabilities

Expectimax needs a probabilistic model of how the opponent or environment behaves in each state. The model may be as simple as a uniform distribution, as for a fair die, or may require extensive computation, and it may even say that adversarial actions are likely. Every outcome outside the agent's control becomes a chance node. A probabilistic belief about another agent's action does not mean that the agent is flipping coins.

Suppose the opponent runs depth-2 minimax and plays its result 80% of the time, moving uniformly

at random otherwise. With two moves available, the minimax move has probability $0.8 + 0.2 \cdot \frac{1}{2} = 0.9$ and the other $0.2 \cdot \frac{1}{2} = 0.1$. Expectimax with these probabilities is the right search, but each chance node requires simulating the opponent's own search. This becomes slow very quickly, and worse still if the opponent simulates us. Minimax is the exception: modeling an optimal opponent who models us collapses into a single game tree.

Both modeling errors are dangerous. Assuming chance when the world is adversarial is *dangerous optimism*; assuming the worst case when it is unlikely is *dangerous pessimism*. In five games each on a small maze, with Pacman searching to depth 4 using an evaluation function that avoids trouble and the adversarial ghost searching to depth 2 with an evaluation function that seeks Pacman, the results were:

	Adversarial ghost	Random ghost
Minimax Pacman	won 5/5, average 483	won 5/5, average 493
Expectimax Pacman	won 1/5, average −303	won 5/5, average 503

Against a random ghost, expectimax scores slightly higher. Against an adversarial ghost, its optimism is costly, while minimax remains safe.

6.4 Mixed layer types

Game trees need not alternate strictly between two kinds of node. Expectiminimax treats the environment as an extra random player that moves after each MAX or MIN agent, and each node combines its children by the rule for its type. With four ghosts, a MAX layer for Pacman is followed by four MIN layers before the next MAX layer; the minimizers then cooperate implicitly. If one ghost is random and the other optimal, the layers repeat as MAX, chance, MIN.

In backgammon, dice rolls increase the branching factor: two dice give 21 distinct rolls, and a position has about 20 legal moves. Looking ahead over four moves with three rolls between them gives about $20 \cdot (21 \cdot 20)^3 \approx 1.5 \times 10^9$ leaves. As depth increases, the probability of reaching any given node shrinks, so deep search is less useful and limiting depth is less damaging, though pruning is harder. TD-Gammon combined depth-2 search, a very good evaluation function, and reinforcement learning to reach world-champion level, the first AI world champion in any game.

6.5 Monte Carlo tree search

Alpha-beta methods assume a fixed horizon, which is hopeless for Go with $b > 300$. Monte Carlo tree search (MCTS) combines two ideas:

- Evaluation by rollouts: from a state, play many games to termination with a simple, fast rollout policy, and count wins and losses. The fraction of wins correlates with the true value

of the position, and a better rollout policy helps.

- Selective search: explore the parts of the tree that will improve the decision at the root, regardless of depth.

The simplest version does N rollouts from each child of the root and picks the move with the best fraction of wins. The following allocations of rollouts among three root moves show how this can be improved.

Allocation	Left	Middle	Right
Equal	57/100	39/100	65/100
Equal, hopeless middle	57/100	0/100	59/100
Favor promising moves	77/140	0/10	90/150
Favor promising moves	61/100	6/10	48/100

In the second row, 100 rollouts are spent confirming that the middle move loses. Shifting most of them to the other moves, as in the third row, sharpens the comparison that matters. The last row shows the opposite risk: the middle move looks best, with a 60% win rate, but that estimate rests on only 10 rollouts and has high variance. We should therefore also allocate rollouts to *uncertain* moves.

Definition 6.2 [UCB1]**.** For a node n, let $N(n)$ be the number of rollouts through n, and let $U(n)$ be their total utility (for example, the number of wins) for the player who moves at n's parent. Then

$$\mathrm{UCB1}(n) = \frac{U(n)}{N(n)} + C\sqrt{\frac{\log N(\mathrm{parent}(n))}{N(n)}}.$$

The first term measures how promising n is and the second how uncertain its estimate is. The parameter C balances exploitation against exploration.

UCB1 has provable guarantees for bandit problems, which are related to but not the same as tree search. In the last row of the table, with $C = 1$ and the natural logarithm, the parent has $N = 210$ and the scores are 0.84, 1.33, and 0.71, so the next rollout goes to the uncertain middle move. Larger C favors exploration; with many trials accumulated, less exploration may be appropriate.

Algorithm 6.2: UCT (MCTS with UCB1 selection)

Input: Current state s_0 and a rollout policy.
Output: A move at s_0.

Start with a search tree containing only s_0
while time remains **do**
  $n \leftarrow s_0$
  while n is fully expanded and nonterminal **do**
    $n \leftarrow$ the child of n maximizing UCB1
  Add a new child c of n, and run a rollout from c
  for each node on the path from c up to s_0 **do**
    Increment its N and add the rollout's utility to its U
return the move leading to the child of s_0 with the largest N

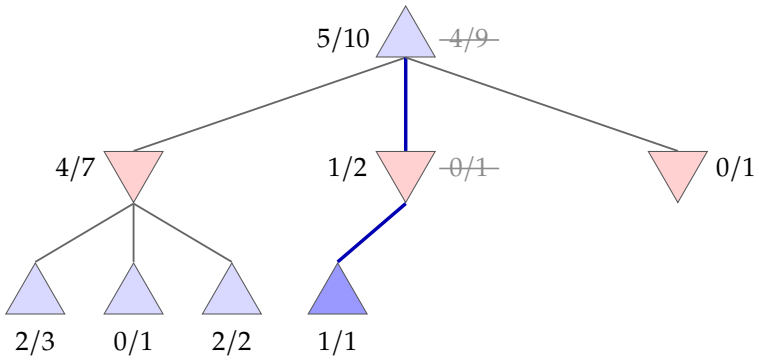


Figure 13: One UCT iteration. Labels are U/N for each node. Selection followed the blue path to the middle node, which gained the new dark child; its winning rollout changed the counts from 0/1 to 1/2 and from 4/9 to 5/10.

Each node below the root counts the rollout made when it was added plus all rollouts below it. For example, the left node has 4/7: its own losing rollout plus 2/3, 0/1, and 2/2 from its children.

UCT has no explicit min or max. The value estimate $U(n)/N(n)$ is a weighted average of its children's estimates, weighted by their rollout counts. As $N \to \infty$, UCB1 concentrates almost all rollouts in the best children, so these averages approach the max or min, and UCT selects the minimax move in the limit. In practice N never approaches infinity. MCTS is also easily parallelized, so modern hardware can run very many rollouts.

Game playing has forced efficient use of computation and produced lasting research ideas: reinforcement learning (checkers), iterative deepening (chess), rational metareasoning (Othello), Monte Carlo tree search (chess and Go), and solution methods for partial-information games used in economics (poker). Video games are much harder still, with $b = 10^{500}$, $|S| = 10^{4000}$, horizon $m = 10{,}000$, partial observability, and often more than two players.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 6 (Spring 2024); Nikhil Sharma, Note 6.

Lecture 7: Logic: Propositional Logic and Planning

7.1 Knowledge-based agents

A knowledge-based agent maintains a knowledge base (KB), a set of sentences in a formal language, and uses logical inference to draw new conclusions from it. Rather than learning only from reward, the agent is told facts about its world, such as that the air is hot and hazy near lava, and reasons about what to do.

The approach is declarative. We *tell* the agent what it needs to know, or let it learn the knowledge, and it then *asks* itself what to do; the answers should follow from the KB. The KB holds domain-specific facts, while the inference engine is generic code, so a single inference algorithm can answer any answerable question. Useful knowledge includes the effects of actions (a transition model), how the world affects sensors (a sensor model), and facts about the current state. With these an agent can track a partially observable world and formulate plans.

7.2 Syntax, semantics, and entailment

A logic is defined by three things: its syntax, which says which sentences are allowed; its set of possible worlds; and its semantics, a definition of which sentences are true in which worlds. The sentence $x > y$, for instance, is true in the world $x = 5,\ y = 2$ and false in the world $x = 2,\ y = 3$.

- In propositional logic, a world assigns true or false to each proposition symbol, as in $\{P = \text{true}, Q = \text{true}, R = \text{false}, S = \text{true}\}$, written 1101.
- In first-order logic, a world consists of objects, the relations that hold among them, and the values of functions on them.
- A relational database stores ground relational sentences, such as $\mathit{Sibling}(\mathit{Ali}, \mathit{Bo})$. Sentences in the database are true and everything else is false, so it cannot express disjunction, implication, or universal statements. Query languages such as SQL are typically variants of first-order logic. A knowledge graph is roughly a relational database plus an ontology of types and relations.

Definition 7.1 [Entailment]. Let $\mathcal{M}(\alpha)$ be the set of worlds in which α is true. Then α entails β, written $\alpha \models \beta$, if β is true in every world where α is true: $\mathcal{M}(\alpha) \subseteq \mathcal{M}(\beta)$.

For example, $\neg Q \wedge R \wedge S \wedge W \models \neg Q$: the first sentence holds in fewer worlds, all of which satisfy $\neg Q$.

A proof demonstrates an entailment. An inference algorithm is sound if everything it proves is entailed, and complete if it can prove everything that is entailed. There are two broad methods.

- Model checking enumerates the possible worlds and checks that β holds in each world where α holds. This is feasible for propositional logic, which has finitely many worlds, but not in general for first-order logic.
- Theorem proving searches for a sequence of applications of inference rules leading from α to β. For instance, Modus Ponens infers Q from P and $P \Rightarrow Q$.

7.3 Propositional logic

Given proposition symbols $X_1, \ldots, X_n$, often supplemented by *True* and *False*, the sentences are defined recursively: each X_i is a sentence, and if α and β are sentences, then so are

$$\neg\alpha, \qquad \alpha \wedge \beta, \qquad \alpha \vee \beta, \qquad \alpha \Rightarrow \beta, \qquad \alpha \Leftrightarrow \beta.$$

There are no other sentences. A literal is a symbol or its negation. A sentence joined by $\wedge$ is a conjunction of conjuncts; one joined by $\vee$ is a disjunction of disjuncts.

A model m assigns true or false to every symbol. The truth value of a symbol is read from m; the connectives are evaluated by the truth table below. In words, $\alpha \Rightarrow \beta$ is true unless α is true and β false, and $\alpha \Leftrightarrow \beta$ is true when $\alpha \Rightarrow \beta$ and $\beta \Rightarrow \alpha$ both are.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	false	true	false	false	true	true
false	true	true	false	true	true	false
true	false	false	false	true	false	false
true	true	false	true	true	true	true

The truth of a compound sentence is computed by recursion over syntax: look up a symbol in the model, and otherwise apply the top-level connective to the recursively computed values of its arguments. For $P \wedge (\neg Q \vee R)$ in the model 110, that is P, Q true and R false, the disjunction $\neg Q \vee R$ is false, so the sentence is false.

7.4 A knowledge base for partially observable Pacman

Suppose Pacman knows the map but perceives only whether there is a wall or a gap in each of the four directions. We use symbols for the following, writing the time step as a superscript:

- walls: $Wall_{x,y}$, true if there is a wall at $[x, y]$ (N symbols for N locations);
- percepts: $Blocked^t_W$, true if there is a wall to the west at time t, and similarly for N, E, S ($4T$ symbols over T steps);

- actions: W^t, true if Pacman moves west at time t, and similarly N^t, E^t, S^t ($4T$ symbols);
- location: $At^t_{x,y}$, true if Pacman is at $[x, y]$ at time t (NT symbols).

That is $N + 8T + NT = O(NT)$ variables and $2^{O(NT)}$ worlds, each a complete history, most of them strange. With $N = 200$ and $T = 400$, the 80,000 location variables alone give $2^{80000} \approx 10^{24082}$ worlds.

The KB has four parts.

- *Map.* Pacman knows where the walls are and are not: $Wall_{0,0} \wedge Wall_{0,1} \wedge \cdots$ and $\neg Wall_{1,1} \wedge \neg Wall_{1,2} \wedge \cdots$.
- *Initial state.* Pacman is somewhere, $At^0_{1,1} \vee At^0_{1,2} \vee \cdots$, and in at most one place: $\neg(At^0_{1,1} \wedge At^0_{1,2}) \wedge \neg(At^0_{1,1} \wedge At^0_{1,3}) \wedge \cdots$.
- *Sensor model.* Each percept variable is equivalent to a condition on the world at the same time. Pacman perceives a wall to the west at time t if and only if he is at some $[x, y]$ and there is a wall at $[x - 1, y]$:

  $$Blocked^0_W \Leftrightarrow (At^0_{1,1} \wedge Wall_{0,1}) \vee (At^0_{1,2} \wedge Wall_{0,2}) \vee (At^0_{1,3} \wedge Wall_{0,3}) \vee \cdots .$$

  There are $4T$ such sentences, each of size $O(N)$, and they hold for any map.
- *Transition model.* Each state variable gets its value from a SUCCESSOR-STATE AXIOM:

  $$\begin{aligned} X^t \Leftrightarrow \big[&X^{t-1} \wedge \neg(\text{some action at } t-1 \text{ made it false})\big] \\ &\vee \big[\neg X^{t-1} \wedge (\text{some action at } t-1 \text{ made it true})\big]. \end{aligned}$$

  For Pacman's location,

  $$\begin{aligned} At^{17}_{3,3} \Leftrightarrow \big[&At^{16}_{3,3} \wedge \neg\big((\neg Wall_{3,4} \wedge N^{16}) \vee (\neg Wall_{4,3} \wedge E^{16}) \vee \cdots\big)\big] \\ &\vee \big[\neg At^{16}_{3,3} \wedge \big((At^{16}_{3,2} \wedge \neg Wall_{3,3} \wedge N^{16}) \vee (At^{16}_{2,3} \wedge \neg Wall_{3,3} \wedge E^{16}) \vee \cdots\big)\big]. \end{aligned}$$

  Pacman stays unless he successfully moves away, and arrives if he moves in from a neighbor without a wall in the way.

The $O(NT)$ transition sentences dominate the KB. At about ten lines each, $N = 200$ and $T = 400$ give some 800,000 lines, or 20,000 pages. This reflects the limited expressive power of propositional logic; code generates these sentences rather than a person. First-order logic needs only $O(1)$ transition sentences.

7.5 Entailment, implication, and reasoning tasks

Entailment $\alpha \models \beta$ is a fact about sets of models, while implication $\alpha \Rightarrow \beta$ is a sentence of the logic. They are connected through the KB, which we treat as one large conjunction: if $\alpha \Rightarrow \beta \in$ KB, then

$\alpha \wedge \text{KB} \models \beta$ by Modus Ponens. To make α together with the KB entail β, add $\alpha \Rightarrow \beta$ to the KB.

Many tasks pose questions to the same KB:

- Localization: given the map, an initial KB, and a sequence of percepts and actions, where am I?
- Mapping: given a location sensor, what is the map?
- Simultaneous localization and mapping: where am I, and what is the map?
- Planning: which action sequence is guaranteed to reach the goal?

All of these use the same knowledge base and the same inference algorithm.

Source. Stuart Russell, *CS 188: Introduction to Artificial Intelligence*, Lecture 7 (Spring 2024); Henry Zhu, edited by Peyrin Kao, Note 7.

Lecture 8: Logic: Logical Inference, Theorem Proving, Boolean Satisfiability and DPLL

8.1 Validity, satisfiability, and entailment

With N proposition symbols there are 2^N models. A sentence is VALID if it is true in every model, as *True* is; SATISFIABLE if it is true in at least one; and UNSATISFIABLE if it is true in none. For instance, $A \wedge B$ is satisfiable but not valid, and $A \wedge \neg A$ is unsatisfiable.

The inference problem asks whether KB $\models q$ for a knowledge base KB and a query q. Two equivalences reduce it to these notions:

$$\alpha \models \beta \iff \alpha \Rightarrow \beta \text{ is valid} \iff \neg(\alpha \Rightarrow \beta) \text{ is unsatisfiable} \iff \alpha \wedge \neg\beta \text{ is unsatisfiable.}$$

Showing validity of $\alpha \Rightarrow \beta$ is a DIRECT PROOF. Showing that $\alpha \wedge \neg\beta$ is unsatisfiable, by adding the negated conclusion to what we know, is a PROOF BY CONTRADICTION (*reductio ad absurdum*). A fast satisfiability solver therefore also tests entailment.

Model checking by enumeration takes $O(2^N)$ time; propositional entailment is co-NP-complete, and satisfiability (SAT) is NP-complete. Exponential worst cases are unavoidable, but practical algorithms often finish far sooner.

8.2 Logical equivalences and conjunctive normal form

Two sentences are LOGICALLY EQUIVALENT, written $\alpha \equiv \beta$, if they are true in the same models. Besides commutativity and associativity of $\wedge$ and $\vee$, the standard equivalences are:

$\neg(\neg\alpha) \equiv \alpha$	double negation
$\alpha \Rightarrow \beta \equiv \neg\beta \Rightarrow \neg\alpha$	contraposition
$\alpha \Rightarrow \beta \equiv \neg\alpha \vee \beta$	implication elimination
$\alpha \Leftrightarrow \beta \equiv (\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$	biconditional elimination
$\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$	De Morgan
$\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$	De Morgan
$\alpha \wedge (\beta \vee \gamma) \equiv (\alpha \wedge \beta) \vee (\alpha \wedge \gamma)$	distributivity of $\wedge$ over $\vee$
$\alpha \vee (\beta \wedge \gamma) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$	distributivity of $\vee$ over $\wedge$

A sentence is in CONJUNCTIVE NORMAL FORM (CNF) if it is a conjunction of CLAUSES, each a disjunction

of literals, such as

$$(A \vee \neg B \vee \neg C) \wedge (\neg A) \wedge (\neg D \vee B \vee C \vee E \vee F) \wedge (\neg E \vee \neg F) \wedge (B \vee E).$$

Every sentence has an equivalent CNF, so an entire KB can be written as one CNF sentence. Efficient SAT solvers operate on this form. The conversion uses four steps:

1. Eliminate $\Leftrightarrow$: replace $\alpha \Leftrightarrow \beta$ by $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$, where α, β may be long expressions.
2. Eliminate $\Rightarrow$: replace $\alpha \Rightarrow \beta$ by $\neg\alpha \vee \beta$.
3. Move $\neg$ inward to the literals with De Morgan's laws and double negation; for example, $\neg(\alpha \wedge \neg\beta)$ becomes $\neg\alpha \vee \beta$.
4. Distribute $\vee$ over $\wedge$ wherever a disjunction sits above a conjunction. For example, $(\alpha \wedge \beta \wedge \gamma) \vee (\varepsilon \wedge \delta)$ becomes the six clauses $(\alpha \vee \varepsilon) \wedge (\beta \vee \varepsilon) \wedge (\gamma \vee \varepsilon) \wedge (\alpha \vee \delta) \wedge (\beta \vee \delta) \wedge (\gamma \vee \delta)$.

Example: For $A \Leftrightarrow (B \vee C)$, the steps give $(A \Rightarrow B \vee C) \wedge (B \vee C \Rightarrow A)$, then $(\neg A \vee B \vee C) \wedge (\neg(B \vee C) \vee A)$, then $(\neg A \vee B \vee C) \wedge ((\neg B \wedge \neg C) \vee A)$, and finally

$$(\neg A \vee B \vee C) \wedge (\neg B \vee A) \wedge (\neg C \vee A).$$

Similarly, the Pacman sensor fact $At^0_{1,1} \Rightarrow (Wall_{0,1} \Leftrightarrow Blocked^0_W)$ becomes $(\neg At^0_{1,1} \vee \neg Wall_{0,1} \vee Blocked^0_W) \wedge (\neg At^0_{1,1} \vee \neg Blocked^0_W \vee Wall_{0,1})$.

8.3 Forward chaining

A DEFINITE CLAUSE is either a single symbol X, equivalent to $True \Rightarrow X$, or an implication from a conjunction of symbols to one symbol, $X_1 \wedge \cdots \wedge X_n \Rightarrow Y$. FORWARD CHAINING applies Modus Ponens: from $X_1 \wedge \cdots \wedge X_n \Rightarrow Y$ and $X_1, \ldots, X_n$, infer Y. It keeps adding new facts until the query appears or nothing more can be added. Two tricks make it run in time linear in the size of the KB: each symbol records the clauses whose premises contain it, and each clause keeps a count of its premises not yet known to be true.

Theorem 8.1 [FORWARD CHAINING]. *Forward chaining is sound and complete for knowledge bases of definite clauses.*

Proof. Soundness follows from the soundness of Modus Ponens. For completeness, the algorithm reaches a fixed point at which no new symbol can be derived. Let m be the model that makes the inferred symbols true and all others false. Every clause of KB is true in m: if some $a_1 \wedge \cdots \wedge a_k \Rightarrow b$ were false in m, then $a_1, \ldots, a_k$ would be inferred and b not, so the count for this clause would be zero and b would still be added, contradicting the fixed point. Hence m is a model of KB. If KB $\models q$, then q is true in every model of KB, including m, so q was inferred. □

Algorithm 8.1: Forward chaining for definite clauses

```
Input: A knowledge base of definite clauses, and a query symbol q.
Output: Whether KB |= q.
count[c] ← number of symbols in the premise of each clause c
inferred[s] ← false for every symbol s
agenda ← queue of the symbols asserted true in KB
while agenda is not empty do
    p ← pop the front of agenda
    if p = q then
        return true
    if not inferred[p] then
        inferred[p] ← true
        for each clause c whose premise contains p do
            decrement count[c]
            if count[c] = 0 then
                add the conclusion of c to agenda
return false
```

The *inferred* table prevents a symbol from being processed twice, which also stops loops caused by clauses such as $P \Rightarrow Q$ and $Q \Rightarrow P$.

Example: Take the clauses (1) $A \Rightarrow B$, (2) $A \Rightarrow C$, (3) $B \wedge C \Rightarrow D$, (4) $D \wedge E \Rightarrow Q$, (5) $A \wedge D \Rightarrow Q$, and (6) A, with query Q. Initially $count = [1, 1, 2, 2, 2, 0]$ and the agenda is $[A]$. The table shows the state after each symbol is processed.

Pop	Counts for clauses 1–5	Agenda
A	$[0, 0, 2, 2, 1]$	$[B, C]$
B	$[0, 0, 1, 2, 1]$	$[C]$
C	$[0, 0, 0, 2, 1]$	$[D]$
D	$[0, 0, 0, 1, 0]$	$[Q]$
Q	query reached: return true	

By contrast, for the KB A, B, D, $A \wedge B \Rightarrow C$, $C \wedge D \Rightarrow E$ and query F, the algorithm infers C and E, empties the agenda, and returns false. By completeness, F is not entailed.

Backward chaining works from the other end: it keeps track of what it is trying to prove and looks for clauses whose conclusions would get it there.

8.4 DPLL satisfiability

The DPLL algorithm, named for Davis, Putnam, Logemann, and Loveland, is the core of modern SAT solvers. It takes a CNF sentence and runs a depth-first, backtracking search over partial models, which assign values to only some symbols. It adds three refinements to plain search:

- Early termination: a clause is true once any literal is true, and the sentence is false once

any clause is false. For example, $(A \vee B) \wedge (A \vee \neg C)$ is satisfied by $A = \text{true}$ and falsified by $A = B = \text{false}$.

- Pure symbols: if a symbol appears with only one sign in the clauses not yet satisfied, assign it the value that makes those literals true. In $(A \vee B) \wedge (A \vee \neg C) \wedge (C \vee \neg B)$, set A true. This cannot hurt: the assignment satisfies every clause containing the symbol and falsifies none.
- Unit clauses: if a clause has only one literal still unassigned and its others are false, that literal must be made true. If $A = \text{false}$, then $(A \vee B) \wedge (\neg B \vee \neg C)$ forces B true. Satisfying unit clauses often creates more of them, a cascade called unit propagation.

Algorithm 8.2: DPLL

Input: Clauses of a CNF sentence, unassigned symbols, partial model m (initially empty).
Output: Whether the clauses are satisfiable.

```
if every clause is true in m then
    return true
if some clause is false in m then
    return false
keep only the clauses not yet true in m
if some symbol P is pure with satisfying value v then
    return DPLL(clauses, symbols −P, m ∪ {P = v})
if some clause is a unit clause, forcing P = v then
    return DPLL(clauses, symbols −P, m ∪ {P = v})
P ← the first remaining symbol
return DPLL(clauses, rest, m ∪ {P = true}) or DPLL(clauses, rest, m ∪ {P = false})
```

Example: Start with $(A \vee B) \wedge (\neg A \vee \neg C) \wedge (C \vee \neg B) \wedge (A \vee B \vee C)$. There are no pure symbols or unit clauses, so branch on $A = \text{true}$. The first and last clauses are satisfied, leaving $(\neg C) \wedge (C \vee \neg B)$. Now B is pure, appearing only as $\neg B$, so set B false; this leaves $(\neg C)$, which makes C pure and also a unit clause. Setting C false satisfies everything with the partial model $\{A = \text{true}, B = \text{false}, C = \text{false}\}$.

An unsatisfiable example shows the search tree. Consider

$$(\neg N \vee \neg S) \wedge (M \vee Q \vee N) \wedge (L \vee \neg M) \wedge (L \vee \neg Q) \wedge (\neg L \vee \neg P) \wedge (R \vee P \vee N) \wedge (\neg R \vee \neg L) \wedge (S).$$

No symbol is pure. The unit clause (S) sets S true, which reduces the first clause to the unit $(\neg N)$, so N is false. The remaining clauses $(M \vee Q) \wedge (L \vee \neg M) \wedge (L \vee \neg Q) \wedge (\neg L \vee \neg P) \wedge (R \vee P) \wedge (\neg R \vee \neg L)$ have no pure symbol or unit clause, so the search branches on M.

With M true, Q occurs only as $\neg Q$, so it is set false; then (L), $(\neg P)$, and finally $(R) \wedge (\neg R)$ follow. With M false, the unit (Q) forces Q true, then L true and P false, reaching the same contradiction.

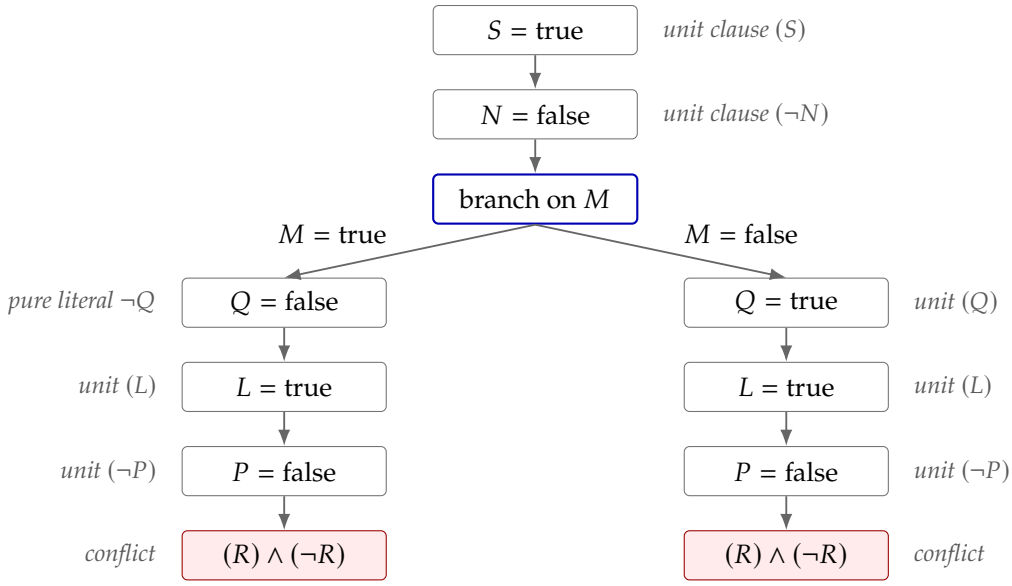


Figure 14: DPLL on the eight-clause sentence. Both values of M propagate to the contradictory unit clauses (R) and $(\neg R)$, so the sentence is unsatisfiable.

A naive DPLL handles about 100 variables. Industrial solvers handle around 10^8 by adding smart variable and value ordering, divide and conquer, caching unsolvable subcases as extra clauses, and indexing so that each step takes typically constant time: index the clauses where each variable occurs positively and negatively, and track the number of satisfied clauses and of remaining literals in each clause. SAT solvers are used for circuit, software, and protocol verification; software and protocol synthesis; many combinatorial problems; and planning.

8.5 Resolution

Every clause can be written as an implication from a conjunction of symbols to a disjunction of symbols: $A \vee B \vee \neg C \vee \neg D$ is $C \wedge D \Rightarrow A \vee B$. The RESOLUTION rule combines two clauses that contain a symbol with opposite signs and removes that symbol:

$$\frac{A \wedge B \wedge C \Rightarrow U \vee V \qquad D \wedge E \wedge U \Rightarrow X \vee Y}{A \wedge B \wedge C \wedge D \wedge E \Rightarrow V \vee X \vee Y}.$$

A CNF sentence is unsatisfiable if and only if repeated resolution produces the empty clause $() \Rightarrow ()$. Used this way, resolution is sound and complete for propositional logic, but it takes exponential time in the worst case. Modus Ponens and AND-ELIMINATION, which infers A from $A \wedge B$, are other common inference rules.

8.6 Logical agents and planning as satisfiability

A knowledge-based agent tells its KB each percept, asks it for an action, and records the chosen action.

Algorithm 8.3: Knowledge-based agent

Input: The current percept; persistent KB and time t, initially 0.
Output: An action.

```
Tell(KB, MakePerceptSentence(percept, t))
action ← Ask(KB, MakeActionQuery(t))
Tell(KB, MakeActionSentence(action, t))
t ← t + 1
return action
```

In the fully observable, deterministic case, a SAT solver can make plans. For $T = 1, 2, \ldots$, build a KB of PacPhysics for T steps and assert that the goal holds at time T. The problem has a plan of T steps exactly when this sentence is satisfiable, and the plan is read off from the truth values of the action variables. PacPhysics for planning contains:

- the map: where walls and food are and are not;
- the initial state: Pacman's start, in exactly one place, and the ghosts;
- the actions: exactly one action at each step;
- the transition model: $At^t_{x,y}$ holds iff Pacman was at $[x, y]$ and stayed put, or was next to it and moved there; food at $[x, y]$ remains iff it was there and was not eaten; similar axioms for the ghosts;
- the goal: Pacman achieves it by time T.

8.7 State estimation, localization, and mapping

For partially observable Pacman, the KB contains the wall, percept, action, and location variables; the sensor model; the map; the initial-state constraints; domain constraints such as one action per step; and the transition model. State estimation keeps track of what is true now. A logical agent can simply ask whether KB $\wedge$ actions $\wedge$ percepts $\models At^6_{2,2}$, but this lazy approach reanalyzes the whole history at every step. An eager form instead updates after each action and percept: for each state variable X^t, add X^t to the KB if KB $\wedge$ action^{t-1} $\wedge$ percept^t $\models X^t$, and add $\neg X^t$ if that is entailed.

- *Localization in a known map.* Initialize PacPhysics for T steps and run the agent, applying the eager update to each $At^t_{x,y}$ before choosing an action. Pacman's possible locations are those not provably false. Each percept and move eliminates locations whose surroundings are inconsistent with the evidence.
- *Mapping from a known relative location.* Call the start $[0, 0]$. When percepts reveal which actions

succeed, Pacman always knows where he is (DEAD RECKONING). After each step, add $Wall_{x,y}$ or $\neg Wall_{x,y}$ whenever it is entailed; the wall variables constitute the map.

- *Simultaneous localization and mapping (SLAM).* If sensors only count adjacent walls, Pacman cannot tell which actions worked and so does not know where he is. He still starts at $[0, 0]$ and, after each step, adds entailed wall literals and entailed location literals.

The eager method is not always accurate. There can be cases where neither X^t nor $\neg X^t$ is entailed, and neither Y^t nor $\neg Y^t$, but the constraint $X^t \vee Y^t$ is: either the study at time t was flawed, or meat causes cancer. Recording only single literals loses this information. Exact state estimation must track properties of combinations of state variables, which is intractable in general.

Source. Stuart Russell, *CS 188: Introduction to Artificial Intelligence*, Lecture 8 (Spring 2024); Henry Zhu, edited by Peyrin Kao, Note 8.

Lecture 9: Logic: First-Order Logic

9.1 Fluents and successor-state axioms

State attributes that change over time are fluents, written with a time index such as Hot^t. A fluent is true at the next step if some action made it true, or if it was already true and no action made it false:

$$F^{t+1} \Leftrightarrow ActionCausesF^t \vee \left(F^t \wedge \neg ActionCausesNotF^t\right).$$

For example, $Hot^{t+1} \Leftrightarrow StepCloseToLava^t \vee (Hot^t \wedge \neg StepAwayFromLava^t)$. Food in Pacman needs only the persistence half, since no action creates food. If Pacman eats a dot on the step he occupies its square, then

$$Food^{17}_{3,3} \Leftrightarrow Food^{16}_{3,3} \wedge \neg At^{17}_{3,3}.$$

A planning query conjoins the initial state, the successor-state axioms, and a goal such as $InOasis^T \wedge Alive^T$; a satisfying assignment yields an action sequence that reaches the goal.

9.2 Representations and expressive power

Representations form a spectrum.

- *Atomic*: a state is an indivisible label, as in search and game playing.
- *Factored*: a state is a vector of variable values, as in propositional logic, planning, and Bayes nets.
- *Structured*: a state consists of objects and relations among them, as in first-order logic, databases, logic programs, and probabilistic programs.

First-order logic (FOL) represents objects with constant symbols, relations with predicate symbols, and functions with function symbols. It is far more concise than propositional logic: the rules of chess take about 100,000 pages in propositional logic and one page in FOL. One sentence states when Pacman stays alive:

$$\forall t \;\; Alive(t) \Leftrightarrow \Big[Alive(t-1) \wedge \neg\exists g, x, y \;\left(Ghost(g) \wedge At(Pacman, x, y, t-1) \wedge At(g, x, y, t-1)\right)\Big].$$

Likewise, Pacman cannot be in two places at once:

$$\forall x_1, y_1, x_2, y_2, t \;\; \left(At(x_1, y_1, t) \wedge At(x_2, y_2, t)\right) \Rightarrow (x_1 = x_2 \wedge y_1 = y_2).$$

Propositional logic needs a clause such as $\neg(At^0_{1,1} \wedge At^0_{1,2})$ for every pair of locations and time.

9.3 Possible worlds

Definition 9.1 [First-order possible world]. A possible world, or model, for a first-order language consists of

- a nonempty set of objects;
- for each k-ary predicate, the set of k-tuples of objects that satisfy it;
- for each k-ary function, a mapping from k-tuples of objects to objects;
- for each constant symbol, a particular object. Constants are 0-ary functions.

A world must define every function, relation, and constant mentioned in the language. For instance, a world with five objects, King John, Richard, their two left legs, and a crown, interprets *LeftLeg* as a unary function, *OnHead* and *Brother* as binary relations, *Person*, *King*, and *Crown* as unary relations, and *John* and *Richard* as constants. A propositional model is finite, but if the number of objects is unconstrained there are infinitely many first-order models. Even on three fixed objects, the language with one binary predicate *Knows*, one unary function *BFF*, and constants *A*, *B*, *EvilKingJohn* has $2^9 \cdot 3^3 \cdot 3^3 = 373{,}248$ interpretations.

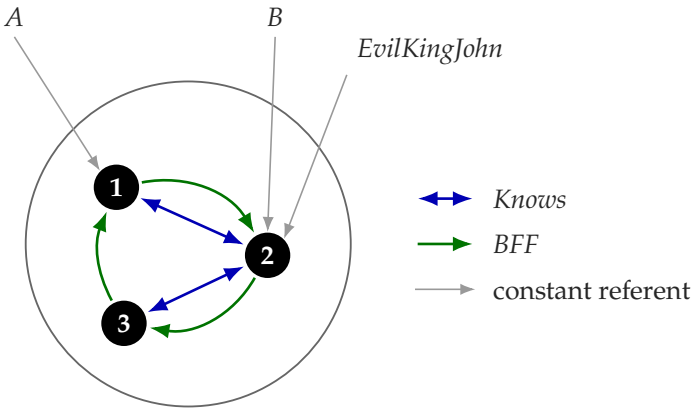


Figure 15: One world for this language. *Knows* holds for the pairs $(1,2), (2,1), (2,3), (3,2)$; *BFF* maps $1 \mapsto 2$, $2 \mapsto 3$, and $3 \mapsto 1$; *A* refers to 1, and *B* and *EvilKingJohn* both refer to 2.

9.4 Syntax and semantics

A term refers to an object. It is a constant symbol, whose referent the world fixes; a function symbol applied to terms, whose value the world specifies from the referents of its arguments; or a logical variable such as x. In the figure, $BFF(EvilKingJohn) \rightarrow BFF(2) \rightarrow 3$. Function symbols are names for objects, such as $LeftLeg(John)$, rather than procedures.

An atomic sentence is a predicate applied to terms, true if the referenced objects are in the relation,

or an equality between terms, true if both refer to the same object:

$$\mathit{Knows}(A, \mathit{BFF}(B)) \rightarrow \mathit{Knows}(1, \mathit{BFF}(2)) \rightarrow \mathit{Knows}(1,3) \rightarrow \text{false},$$
$$\mathit{BFF}(\mathit{BFF}(\mathit{BFF}(B))) = B \rightarrow \mathit{BFF}(\mathit{BFF}(3)) = 2 \rightarrow \mathit{BFF}(1) = 2 \rightarrow \text{true}.$$

The sentence $\mathit{Wife}(\mathit{Einstein}) = \mathit{FirstCousin}(\mathit{Einstein}) \wedge \mathit{Wife}(\mathit{Einstein}) = \mathit{SecondCousin}(\mathit{Einstein})$ is true.

Complex sentences combine sentences with $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$, or with quantifiers. The universal quantifier sentence $\forall x\ \alpha$ is true in a world w if α is true in every extension of w that assigns x to an object of w; the existential quantifier sentence $\exists x\ \alpha$ is true if α is true in some such extension. In the figure, $\mathit{Knows}(x, \mathit{BFF}(x))$ is true for $x = 1$ and $x = 2$ but false for $x = 3$, since $\mathit{Knows}(3,1)$ fails. Hence $\forall x\ \mathit{Knows}(x, \mathit{BFF}(x))$ is false and $\exists x\ \mathit{Knows}(x, \mathit{BFF}(x))$ is true.

The quantifiers act as a conjunction and a disjunction over all objects, so they obey De Morgan's laws:

$$\forall x\ \neg P \equiv \neg\exists x\ P, \qquad \neg\forall x\ P \equiv \exists x\ \neg P, \qquad \forall x\ P \equiv \neg\exists x\ \neg P, \qquad \exists x\ P \equiv \neg\forall x\ \neg P.$$

In the standard grammar, operator precedence runs $\neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow$, and a quantifier extends over everything to its right.

9.5 Writing sentences

- Everyone knows President Obama: $\forall n\ \mathit{Person}(n) \Rightarrow \mathit{Knows}(n, \mathit{Obama})$.
- There is someone that nobody else knows: $\exists s\ \mathit{Person}(s) \wedge \forall n\ \big(\mathit{Person}(n) \wedge \neg(n = s)\big) \Rightarrow \neg\mathit{Knows}(n, s)$.
- Everyone knows someone: $\forall x\ \mathit{Person}(x) \Rightarrow \exists y\ \mathit{Person}(y) \wedge \mathit{Knows}(x, y)$. Because a quantifier extends to the right, this is the same as $\forall x\ \big(\mathit{Person}(x) \Rightarrow \exists y\ (\mathit{Person}(y) \wedge \mathit{Knows}(x, y))\big)$.
- Any two people of the same nationality speak a common language, with $\mathit{Nationality}(x, n)$ and $\mathit{Speaks}(x, l)$:

$$\forall x, y\ \Big[\big(\exists n\ \mathit{Nationality}(x, n) \wedge \mathit{Nationality}(y, n)\big) \Rightarrow \big(\exists l\ \mathit{Speaks}(x, l) \wedge \mathit{Speaks}(y, l)\big)\Big].$$

9.6 Inference in first-order logic

Entailment is defined exactly as before: $\alpha \models \beta$ if β is true in every world where α is. For example, $\forall x\ \mathit{Knows}(x, \mathit{Obama}) \models \exists y\ \forall x\ \mathit{Knows}(x, y)$. Beyond yes or no, inference can answer an existential query with a substitution (binding) σ that makes the instantiated sentence entailed. Writing $\alpha\sigma$ for the result of applying σ to α, the KB $\forall x\ \mathit{Knows}(x, \mathit{Obama})$ answers the query $\exists y\ \forall x\ \mathit{Knows}(x, y)$ with $\sigma = \{y/\mathit{Obama}\}$, since $(\forall x\ \mathit{Knows}(x, y))\sigma = \forall x\ \mathit{Knows}(x, \mathit{Obama})$.

9.7 Propositionalization

To test entailment, convert KB $\wedge \neg\alpha$ to propositional logic and test its satisfiability with a SAT solver. Replace variables by ground terms and treat each ground atomic sentence as a symbol:

- $\exists x\ \mathit{Knows}(x, \mathit{Obama})$ becomes $\mathit{Knows}(X_1, \mathit{Obama})$ for a new constant X_1, that is, the symbol $\mathit{Knows}_{X_1,\mathit{Obama}}$;
- $\forall x\ \mathit{Knows}(x, \mathit{Obama}) \wedge \mathit{Democrat}(\mathit{Feinstein})$ becomes $\mathit{Knows}_{\mathit{Obama},\mathit{Obama}} \wedge \mathit{Knows}_{\mathit{Feinstein},\mathit{Obama}} \wedge \mathit{Democrat}_{\mathit{Feinstein}}$.

With function symbols there are infinitely many ground terms: $\forall x\ \mathit{Knows}(\mathit{Mother}(x), x)$ yields $\mathit{Knows}(\mathit{Mother}(\mathit{Obama}), \mathit{Obama})$, $\mathit{Knows}(\mathit{Mother}(\mathit{Mother}(\mathit{Obama})), \mathit{Mother}(\mathit{Obama}))$, and so on. So iterate: for $k = 1, 2, \ldots$, collect the terms built from constants by at most k nested function applications, propositionalize as if these were all the terms, and halt if a contradiction is found. By Herbrand's theorem, an unsatisfiable first-order sentence has a contradiction among the ground instances for some finite k. If the sentence is satisfiable the loop may run forever, so entailment in FOL is semidecidable.

9.8 Lifted inference

Lifted inference applies inference rules to first-order sentences directly. From $\mathit{Person}(\mathit{Socrates})$ and $\forall x\ \mathit{Person}(x) \Rightarrow \mathit{Mortal}(x)$ we conclude $\mathit{Mortal}(\mathit{Socrates})$ with $\sigma = \{x/\mathit{Socrates}\}$. The general rule is Generalized Modus Ponens:

$$\text{from } \alpha \Rightarrow \beta \text{ and } \alpha' \text{ with } \alpha\sigma = \alpha'\sigma, \text{ conclude } \beta\sigma.$$

For example, from $\forall x\ \mathit{Knows}(x, \mathit{Obama})$ and $\forall y, z\ \mathit{Knows}(y, z) \Rightarrow \mathit{Likes}(y, z)$, the substitution $\{y/x,\ z/\mathit{Obama}\}$ gives $\mathit{Likes}(x, \mathit{Obama})$ for every x. First-order forward chaining repeatedly applies Generalized Modus Ponens until the query is derived or nothing new follows. Prolog uses backward chaining; Datalog and production rule systems use forward chaining; resolution theorem provers apply the resolution rule.

FOL describes domains such as circuits, software, planning, law, taxes, security protocols, e-commerce, geographic information, and knowledge graphs. Inference is semidecidable in general but often efficient in practice, especially for logic programming.

Source. Stuart Russell, *CS 188: Introduction to Artificial Intelligence*, Lecture 9 (Spring 2024); Henry Zhu, edited by Peyrin Kao, Note 9.

Lecture 10: Introduction to Probability

10.1 Uncertainty and probability models

Real agents rarely know the state of the world. Whether a trip to the airport arrives in time depends on partially observed road conditions, noisy sensors such as traffic reports, the complexity of predicting traffic and security lines, and unknown dynamics such as a tire bursting. A probabilistic assertion summarizes both ignorance and the laziness of not modeling every detail. Combined with utilities, probabilities give decision theory: choose

$$a^* = \arg\max_a \sum_s P(s \mid a)\, U(s),$$

the action of maximum expected utility.

Definition 10.1 [Probability model]. Let Ω be a set of possible worlds. A probability model assigns each $\omega \in \Omega$ a number $P(\omega)$ with

$$0 \le P(\omega) \le 1, \qquad \sum_{\omega \in \Omega} P(\omega) = 1.$$

An event is a subset $A \subseteq \Omega$, with $P(A) = \sum_{\omega \in A} P(\omega)$.

For a fair die, $\Omega = \{1, \dots, 6\}$ with each world of probability $1/6$, and $P(\text{roll} < 4) = P(1)+P(2)+P(3) = 1/2$. These laws are not arbitrary: de Finetti showed in 1931 that anyone who bets according to numbers violating them can be forced to lose money on every set of bets.

A random variable is a deterministic function of the world, such as $Odd(\omega) \in \{\text{true}, \text{false}\}$, a temperature $T \in \{\text{hot}, \text{cold}\}$, a travel time $D \in [0, \infty)$, or a ghost location $L \in \{(0,0), (0,1), \dots\}$. Its range is the set of possible values. Its distribution gives

$$P(X = x) = \sum_{\{\omega : X(\omega) = x\}} P(\omega)$$

for each value. Capital letters denote variables and lowercase letters values; $P(x)$ abbreviates $P(X = x)$, and $P(X)$ denotes the whole table. For a Boolean variable we write *odd* for $Odd = \text{true}$ and $\neg odd$ for its negation.

10.2 Joint and marginal distributions

Possible worlds are often built from variables: a world is an assignment of values to every variable. Two dice give 36 worlds. A JOINT DISTRIBUTION over n variables with ranges of size d has d^n entries, so all but the smallest cannot be written out by hand. The running example is the joint distribution of temperature and weather:

$P(T, W)$	T = hot	T = cold	$P(W)$
W = sun	0.45	0.15	0.60
W = rain	0.02	0.08	0.10
W = fog	0.03	0.27	0.30
W = meteor	0.00	0.00	0.00
$P(T)$	0.50	0.50	

Any event probability follows by summing worlds. Here $P(\text{hot} \wedge \text{sun}) = 0.45$, $P(\text{hot}) = 0.50$, and $P(\text{hot} \vee \neg\text{fog}) = 1 - P(\text{cold}, \text{fog}) = 0.73$.

MARGINALIZATION (SUMMING OUT) collapses a dimension of the table:

$$P(X = x) = \sum_y P(X = x, Y = y).$$

The margins of the table are $P(T)$ and $P(W)$. Marginals can be deduced from the joint, but not conversely: many joints share these margins.

10.3 Conditional probability and normalization

Definition 10.2 [CONDITIONAL PROBABILITY]. For $P(b) > 0$,

$$P(a \mid b) = \frac{P(a, b)}{P(b)}.$$

For example,

$$P(W = \text{sun} \mid T = \text{cold}) = \frac{0.15}{0.15 + 0.08 + 0.27 + 0.00} = \frac{0.15}{0.50} = 0.3.$$

A CONDITIONAL DISTRIBUTION $P(W \mid T = t)$ is one such table for each value t, and the family $P(W \mid T)$ collects them:

	sun	rain	fog	meteor
$P(W \mid T = \text{hot})$	0.90	0.04	0.06	0.00
$P(W \mid T = \text{cold})$	0.30	0.16	0.54	0.00

The denominator need not be computed separately. Select the entries consistent with the condition, then NORMALIZE: multiply every entry by $\alpha = 1/(\text{sum of entries})$ so that the table sums to one. Selecting $T = \text{cold}$ gives $(0.15, 0.08, 0.27, 0.00)$, and $\alpha = 1/0.50 = 2$ yields $(0.30, 0.16, 0.54, 0.00)$. In symbols, $P(W \mid T = c) = \alpha P(W, T = c)$.

10.4 The product and chain rules

Rearranging the definition gives the PRODUCT RULE

$$P(a \mid b)\, P(b) = P(a, b),$$

which builds a joint from a conditional and a marginal. Multiplying the columns of $P(W \mid T)$ by $P(T) = (0.5, 0.5)$ recovers the joint table above. Applying it repeatedly gives the CHAIN RULE

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1}),$$

valid for any ordering of the variables; for three variables, $P(x_1, x_2, x_3) = P(x_3 \mid x_1, x_2) P(x_2 \mid x_1) P(x_1)$.

10.5 Inference by enumeration

PROBABILISTIC INFERENCE computes a desired probability from a model, typically a query variable given evidence. Such probabilities are the agent's beliefs and change with evidence: $P(\text{on time} \mid \text{no accidents}) = 0.90$ may rise to 0.95 given that it is 5 a.m., and fall to 0.80 if it is also raining.

Partition the variables into EVIDENCE VARIABLES $E = e$, observed values on the right of the bar; QUERY VARIABLES Q; and HIDDEN VARIABLES H, the rest. INFERENCE BY ENUMERATION computes $P(Q \mid e)$ from the full joint in three steps:

1. select the entries consistent with the evidence;
2. sum out the hidden variables, $P(Q, e) = \sum_h P(Q, h, e)$;
3. normalize, $P(Q \mid e) = \alpha P(Q, e)$.

Example: Consider a joint over season, temperature, and weather:

S	T	W	P	S	T	W	P
summer	hot	sun	0.30	winter	hot	sun	0.10
summer	hot	rain	0.05	winter	hot	rain	0.05
summer	cold	sun	0.10	winter	cold	sun	0.15
summer	cold	rain	0.05	winter	cold	rain	0.20

For $P(W \mid S = \text{winter})$, select the four winter rows and sum out T: sun gives $0.10 + 0.15 = 0.25$ and rain gives $0.05 + 0.20 = 0.25$, so both have probability 0.5. Similarly $P(S \mid W = \text{sun})$ uses the unnormalized values 0.40 for summer and 0.25 for winter, giving $P(\text{summer} \mid \text{sun}) = 0.40/0.65 \approx 0.62$. With two evidence variables, $P(\text{winter} \mid \text{cold}, \text{rain}) = 0.20/0.25 = 0.8$.

Enumeration answers any query, including several query variables, but with n variables of range d it needs $O(d^n)$ time in the worst case (exponential in the number of hidden variables), $O(d^n)$ space to store the joint, and $O(d^n)$ data to estimate its entries.

10.6 Bayes' rule

Writing the product rule both ways, $P(a \mid b)P(b) = P(a, b) = P(b \mid a)P(a)$, and dividing gives BAYES' RULE

$$P(a \mid b) = \frac{P(b \mid a)\, P(a)}{P(b)}.$$

It builds one conditional from its reverse, which helps when one direction is easy to specify and the other is not. It also describes an update from a prior $P(a)$ to a posterior $P(a \mid b)$, a simple formal theory of learning: for hypotheses h and data d,

$$P(h \mid d) = \frac{P(d \mid h)P(h)}{\sum_{h'} P(d \mid h')P(h')}.$$

Example: *Diagnosis from causal knowledge.* Let m be meningitis and s a stiff neck, with $P(s \mid m) = 0.8$, $P(m) = 0.0001$, and $P(s) = 0.01$. Then

$$P(m \mid s) = \frac{P(s \mid m)P(m)}{P(s)} = \frac{0.8 \times 0.0001}{0.01} = 0.008.$$

The posterior is still small, but it is $P(s \mid m)/P(s) = 80$ times the prior, because a stiff neck is 80 times likelier with meningitis than in general. A small probability of a serious disease can still justify a check-up once utilities are considered.

10.7 Independence

Definition 10.3 [INDEPENDENCE]. Variables X and Y are INDEPENDENT, written $X \perp\!\!\!\perp Y$, if

$$P(x, y) = P(x)\, P(y) \qquad \text{for all } x, y.$$

The joint then factors into two simpler distributions. By the product rule $P(x, y) = P(x \mid y)P(y)$, independence is equivalent to $P(x \mid y) = P(x)$ whenever $P(y) > 0$, and symmetrically $P(y \mid x) = P(y)$. For two dice, $P(Roll_1 = 5, Roll_2 = 3) = \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}$ and $P(Roll_2 = 3 \mid Roll_1 = 5) = P(Roll_2 = 3)$. For n independent fair coins, n two-entry tables replace a joint table with 2^n entries. Independence gives this exponential reduction in representation size, but it is rare in practice.

10.8 Conditional independence

Definition 10.4 [Conditional independence]. X is conditionally independent of Y given Z, written $X \perp\!\!\!\perp Y \mid Z$, if

$$P(x \mid y, z) = P(x \mid z) \qquad \text{for all } x, y, z,$$

or equivalently $P(x, y \mid z) = P(x \mid z)P(y \mid z)$ for all x, y, z.

The forms are equivalent because $P(x, y \mid z) = P(x \mid y, z)P(y \mid z)$; replacing $P(x \mid y, z)$ by $P(x \mid z)$ gives the product, and dividing the product by $P(y \mid z)$ recovers the first form. The relation is symmetric in X and Y. Conditional independence is the most basic and robust form of knowledge about uncertain environments, and unlike independence it is ubiquitous. Traffic and carrying an umbrella are dependent, since both indicate rain, but they are plausibly independent given whether it rains. An alarm and a fire are dependent, but the alarm responds to smoke, so the alarm is plausibly independent of fire given smoke.

Example: *Ghostbusters.* A ghost occupies one square G of a 3×3 grid, uniformly a priori. Probing square (x, y) returns a color $C_{x,y} \in \{\text{red}, \text{orange}, \text{yellow}, \text{green}\}$ whose distribution $P(C_{x,y} \mid G)$ depends only on the distance to the ghost: usually red on the ghost, mostly orange at distance 1 or 2, typically yellow at distance 3 or 4, and often green at distance 5 or more. For instance, $P(C_{1,1} = \text{yellow} \mid G = (1, 1)) = 0.1$.

The joint $P(G, C_{1,1}, \ldots, C_{3,3})$ has $9 \times 4^9 = 2{,}359{,}296$ entries. The readings are not independent: an orange reading at $C_{1,2}$ suggests a nearby ghost and changes the probability of yellow at $C_{1,1}$. Given the location, however, each reading depends only on its own distance:

$$P(C_{1,1} = \text{yellow} \mid G = (2, 3)) = P(C_{1,1} = \text{yellow} \mid G = (2, 3), C_{1,2} = \text{orange}),$$

so $C_{1,1} \perp\!\!\!\perp C_{1,2} \mid G$. The chain rule gives

$$P(G, C_{1,1}, \ldots, C_{3,3}) = P(G)\, P(C_{1,1} \mid G)\, P(C_{1,2} \mid G, C_{1,1}) \cdots P(C_{3,3} \mid G, C_{1,1}, \ldots, C_{3,2}),$$

and conditional independence simplifies every factor to $P(C_{x,y} \mid G)$:

$$P(G, C_{1,1}, \ldots, C_{3,3}) = P(G) \prod_{x,y} P(C_{x,y} \mid G).$$

With n squares the factored form stores $n + 4n^2$ numbers, not $n \cdot 4^n$.

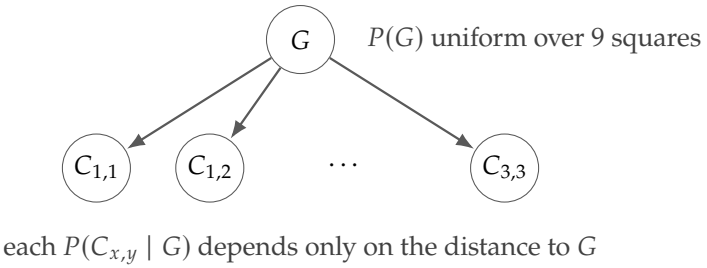


Figure 16: The Ghostbusters naive Bayes model.

This is a NAIVE BAYES MODEL: one discrete query variable, called the CLASS or category, with all other variables potentially observed as evidence and conditionally independent given the class.

Source. Stuart Russell and Peyrin Kao, *CS 188: Introduction to Artificial Intelligence*, Lecture 10 (Spring 2024); Josh Hug and Jacky Liang, edited by Regina Wang, Note 10.

Lecture 11: Bayes Nets: Bayesian Networks

11.1 From conditional independence to compact models

A full joint over n variables with d values has d^n entries. Full independence would shrink it to n small tables, but it rarely holds. Conditional independence usually does, and it is enough. Consider rain R, traffic T, and an umbrella U, with joint distribution

R	T	U	P	product	R	T	U	P	product
F	F	F	0.504	0.314	T	F	F	0.018	0.135
F	F	T	0.056	0.141	T	F	T	0.072	0.060
F	T	F	0.126	0.169	T	T	F	0.042	0.072
F	T	T	0.014	0.076	T	T	T	0.168	0.033

The marginals are $P(R = \mathrm{T}) = 0.3$, $P(T = \mathrm{T}) = 0.35$, and $P(U = \mathrm{T}) = 0.31$. Their product, in the last column of each half, disagrees badly with the joint: the variables are not independent. The chain rule $P(R, T, U) = P(R)P(T \mid R)P(U \mid R, T)$ is always exact, with

$$P(T = \mathrm{T} \mid R) = \begin{cases} 0.2, & R = \mathrm{F}, \\ 0.7, & R = \mathrm{T}, \end{cases} \qquad P(U = \mathrm{T} \mid R, T) = \begin{cases} 0.1, & R = \mathrm{F}, \\ 0.8, & R = \mathrm{T}. \end{cases}$$

The second table does not depend on T, so $U \perp\!\!\!\perp T \mid R$ and $P(U \mid R, T) = P(U \mid R)$. Three small tables, $P(R)$, $P(T \mid R)$, and $P(U \mid R)$, reproduce all eight joint entries.

Likewise, if a patient has a cavity, the probability that a dental probe catches does not depend on a toothache, and the same holds without a cavity: $P(\mathit{Catch} \mid \mathit{Toothache}, \mathit{Cavity}) = P(\mathit{Catch} \mid \mathit{Cavity})$. Equivalent statements are $P(\mathit{Toothache} \mid \mathit{Catch}, \mathit{Cavity}) = P(\mathit{Toothache} \mid \mathit{Cavity})$ and $P(\mathit{Toothache}, \mathit{Catch} \mid \mathit{Cavity}) = P(\mathit{Toothache} \mid \mathit{Cavity})P(\mathit{Catch} \mid \mathit{Cavity})$.

11.2 Bayes net syntax

A Bayes net describes a complex joint distribution using simple local conditional distributions. It is one kind of graphical model and exploits locality: the world has many variables, each interacting directly with only a few others.

Definition 11.1 [Bayes net]. A Bayes net consists of

- a directed acyclic graph with one node per variable X_i, and

- for each node a CONDITIONAL PROBABILITY TABLE (CPT) $P(X_i \mid \mathrm{Pa}(X_i))$, giving a distribution over X_i for each combination of values of its parents $\mathrm{Pa}(X_i)$.

Nodes may be observed or unobserved. An arc indicates direct influence; it is convenient to read arrows as causation, but in general they need not be causal. Formally, the absence of arcs encodes conditional independence.

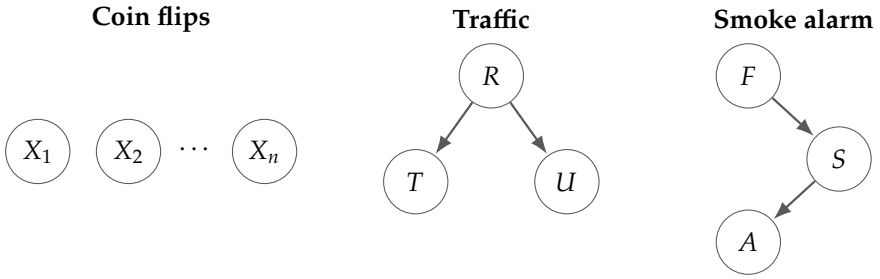


Figure 17: Independent coin flips have no arcs. Rain influences traffic and umbrellas, which are independent given rain. Fire causes smoke, which sets off the alarm.

The Ghostbusters naive Bayes model is also a Bayes net, with arcs from G to every reading. Its CPTs are $P(G)$, uniform at 0.11 per square, and $P(C_{x,y} \mid G)$, one distribution per ghost location. For example:

G	green	yellow	orange	red
$(1,1)$	0.01	0.1	0.3	0.59
$(1,2)$	0.1	0.3	0.5	0.1
$(1,3)$	0.3	0.5	0.19	0.01

Larger nets capture domains such as car-insurance risk, with 27 variables linking age, driving behavior, vehicle properties, accidents, and costs, or car diagnosis, where battery, charging, fuel, and starter faults explain observable lights, gauges, and a car that will not start.

11.3 Global semantics

Definition 11.2 [JOINT DISTRIBUTION OF A BAYES NET]. A Bayes net represents the joint distribution

$$P(x_1, \ldots, x_n) = \prod_{i=1}^{n} P(x_i \mid \mathrm{Pa}(X_i)).$$

The probability of a full assignment is the product of the relevant CPT entries. In the ALARM NETWORK, a burglary B or an earthquake E can set off an alarm A, and John (J) and Mary (M) call when they hear it. The priors are $P(+b) = 0.001$ and $P(+e) = 0.002$, and the other CPTs give the probability of the true value:

B	E	$P(+a \mid B, E)$	A	$P(+j \mid A)$	$P(+m \mid A)$
$+b$	$+e$	0.95	$+a$	0.9	0.7
$+b$	$-e$	0.94	$-a$	0.05	0.01
$-b$	$+e$	0.29			
$-b$	$-e$	0.001			

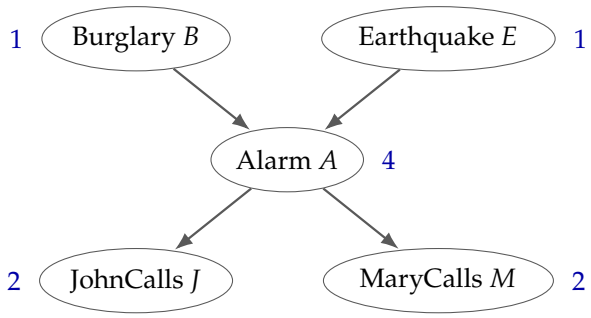


Figure 18: The alarm network. Blue numbers count free parameters in each CPT, 10 in total.

For example,

$$\begin{aligned} P(+b, -e, +a, -j, -m) &= P(+b)P(-e)P(+a \mid +b, -e)P(-j \mid +a)P(-m \mid +a) \\ &= 0.001 \times 0.998 \times 0.94 \times 0.1 \times 0.3 \approx 2.8 \times 10^{-5}. \end{aligned}$$

Each CPT row sums to one, so a child with d values and parents with ranges $d_1, \ldots, d_k$ has $(d-1)\prod_i d_i$ free parameters. The alarm network needs $1+1+4+2+2 = 10$, versus $2^5 - 1 = 31$ for its full joint. In general a joint over N variables with d values has d^N entries, while a net whose nodes have at most k parents needs $O(Nd^k)$. Both determine $P(X_1, \ldots, X_N)$, but the net saves enormous space when the graph is sparse, has local CPTs that are easier to elicit, and supports faster queries.

11.4 Conditional independence semantics

Order the variables topologically, parents before children, so that $\mathrm{Pa}(X_i) \subseteq \{X_1, \ldots, X_{i-1}\}$. The chain rule gives $P(x_1, \ldots, x_n) = \prod_i P(x_i \mid x_1, \ldots, x_{i-1})$. Comparing with the global semantics, a Bayes net asserts

$$P(X_i \mid X_1, \ldots, X_{i-1}) = P(X_i \mid \mathrm{Pa}(X_i)).$$

When building a net, choose the parents of X_i so that they shield it from its other predecessors.

Theorem 11.3 [Local and global semantics]. *For a directed acyclic graph and a topological order, the joint factors as $\prod_i P(x_i \mid \mathrm{Pa}(X_i))$ if and only if each X_i is conditionally independent of its other predecessors given its parents.*

Proof. If each X_i is independent of its predecessors given its parents, each chain-rule factor $P(x_i \mid x_1, \ldots, x_{i-1})$ equals $P(x_i \mid \mathrm{Pa}(X_i))$, giving the product.
Conversely, suppose the product form holds. Sum out x_n: only the last factor involves it, and $\sum_{x_n} P(x_n \mid \mathrm{Pa}(X_n)) = 1$. Repeating in reverse order gives $P(x_1, \ldots, x_i) = \prod_{j \le i} P(x_j \mid \mathrm{Pa}(X_j))$ for every i. Dividing the expressions for i and $i-1$ gives $P(x_i \mid x_1, \ldots, x_{i-1}) = P(x_i \mid \mathrm{Pa}(X_i))$. □

Two graphical consequences are used throughout:

- each node is conditionally independent of its non-descendants given its parents;
- each node is conditionally independent of all other variables given its Markov blanket: its parents, its children, and its children's other parents.

Any topological order may place every non-descendant of X before X, so the first statement follows from the theorem.

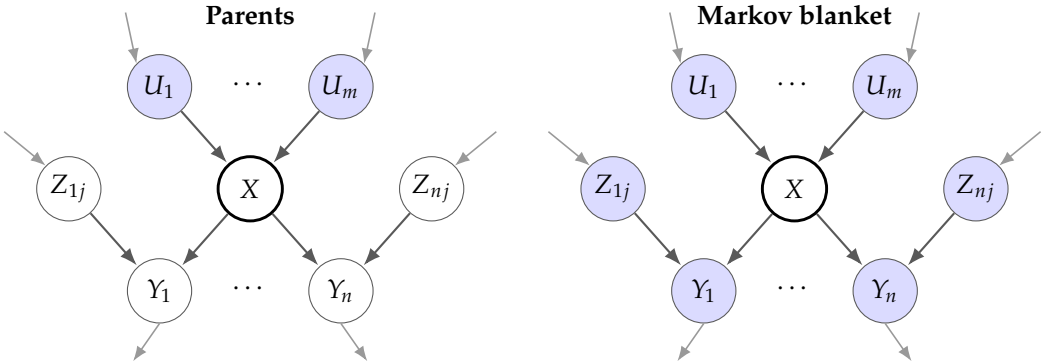


Figure 19: Shaded nodes are the conditioning sets for X: its parents (left) and its Markov blanket (right).

11.5 Node ordering and causality

A net can be built in any variable order, but the order affects its size. Ordering burglary, earthquake, alarm gives the causal net above, with $1 + 1 + 4 = 6$ parameters for these three variables. Ordering alarm, burglary, earthquake forces B to depend on A, and E on both: once the alarm is known, a burglary explains it away and changes the probability of an earthquake. This needs $1 + 2 + 4 = 7$ parameters, whose values are also harder to assess.

Sometimes both directions are equally compact. With $P(+r) = 1/4$, $P(+t \mid +r) = 3/4$, and $P(+t \mid -r) = 1/2$, the net $R \to T$ gives the joint $P(+r, +t) = 3/16$, $P(+r, -t) = 1/16$, $P(-r, +t) = 6/16$, and $P(-r, -t) = 6/16$. The reversed net $T \to R$ represents the same joint with $P(+t) = 9/16$,

$P(+r \mid +t) = 1/3$, and $P(+r \mid -t) = 1/7$.

Causal nets are often simpler, with fewer parents, and easier to reason about and elicit from experts. Bayes nets need not be causal, however. Sometimes no causal net exists over the chosen variables, especially when relevant variables are missing, and the arrows then reflect correlation. The topology may happen to encode causal structure, but what it really encodes is conditional independence, $P(x_i \mid x_1, \ldots, x_{i-1}) = P(x_i \mid \mathrm{Pa}(X_i))$.

The structures range from strict independence, with no arcs, through naive Bayes and sparse nets, to a fully connected graph that represents an arbitrary joint distribution. The choice trades modeling accuracy against memory and computation.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 11 (Spring 2024); Josh Hug and Jacky Liang, edited by Regina Wang, Note 11.

Lecture 12: Bayes Nets: Inference

12.1 Inference by enumeration in Bayes nets

Inference computes a useful quantity from a joint distribution, such as a posterior $P(Q \mid E_1 = e_1, \ldots, E_k = e_k)$ or a most likely explanation $\arg\max_q P(Q = q \mid E_1 = e_1, \ldots)$. A Bayes net defines the joint as a product of CPTs, so enumeration is always possible. In the alarm network,

$$P(B \mid +j, +m) \propto_B P(B, +j, +m) = \sum_{e,a} P(B, e, a, +j, +m)$$
$$= \sum_{e,a} P(B)\, P(e)\, P(a \mid B, e)\, P(+j \mid a)\, P(+m \mid a).$$

Expanding the sum gives four products of five factors for each value of B, and many subexpressions repeat: $P(B)P(+e)$ appears in two terms, and $P(+j \mid +a)P(+m \mid +a)$ in two others. With the CPTs of the previous lecture, normalizing gives $P(+b \mid +j, +m) \approx 0.284$. For a large net such as car insurance, enumeration is hopeless.

The remedy is the distributive law. The sum $uwy + uwz + uxy + uxz + vwy + vwz + vxy + vxz$ needs 16 multiplications and 7 additions, but equals $(u + v)(w + x)(y + z)$, which needs 2 and 3. Moving summations inward as far as possible,

$$P(B \mid +j, +m) = \alpha\, P(B) \sum_e P(e) \sum_a P(a \mid B, e)\, P(+j \mid a)\, P(+m \mid a),$$

and evaluating from the inside out, first over a and then over e. Each inner sum is a table indexed by the variables still free, such as B and e, not a single number.

Enumeration is slow because it builds the whole joint before summing out the hidden variables. Variable elimination interleaves joining and marginalizing. Exact inference remains NP-hard in general, but variable elimination is usually much faster.

12.2 Factors

Definition 12.1 [Factor]. A factor is a multidimensional table of nonnegative numbers. A factor written $P(Y_1, \ldots, Y_N \mid X_1, \ldots, X_M)$ has entries $P(y_1, \ldots, y_N \mid x_1, \ldots, x_M)$. Each capitalized variable is a dimension; a lowercase, assigned variable is a dimension already selected away.

We sometimes write $P(A, b \mid c, D)$ as $f(A, b, c, D)$, another name for the same table. Using

$P(T, W) =$

	sun	rain
hot	0.4	0.1
cold	0.2	0.3

$P(W \mid T) =$

	sun	rain
hot	0.8	0.2
cold	0.4	0.6

the common kinds are:

Factor	Name	Sums to	Example
$P(X, Y)$	joint	1	$P(T, W)$
$P(x, Y)$	selected joint	$P(x)$	$P(\text{cold}, W) = (0.2, 0.3)$
$P(Y \mid x)$	single conditional	1	$P(W \mid \text{cold}) = (0.4, 0.6)$
$P(Y \mid X)$	family of conditionals	$\lvert X \rvert$	$P(W \mid T)$, two rows
$P(y \mid X)$	specified family	anything	$P(\text{rain} \mid T) = (0.2, 0.6)$

12.3 Joining and eliminating

Take the chain $R \to T \to L$ for rain, traffic, and being late, with $P(+r) = 0.1$, $P(+t \mid +r) = 0.8$, $P(+t \mid -r) = 0.1$, $P(+l \mid +t) = 0.3$, and $P(+l \mid -t) = 0.1$. The initial factors are the CPTs, with any evidence already selected.

Joining factors works like a database join: collect all factors mentioning the joining variable and build one factor over the union of their variables, with pointwise products as entries. Joining on R gives $P(r, t) = P(r)\,P(t \mid r)$:

$$P(R) \times P(T \mid R) \;\to\; P(R, T) = \big(0.08,\ 0.02,\ 0.09,\ 0.81\big) \quad \text{for } (+r, +t), (+r, -t), (-r, +t), (-r, -t).$$

Eliminating a variable sums it out of a factor, a projection that shrinks the table: summing R out of $P(R, T)$ gives $P(+t) = 0.17$ and $P(-t) = 0.83$.

Enumeration joins everything first. Joining on R and then on T builds $P(R, T, L)$ with eight entries $0.024, 0.056, 0.002, 0.018, 0.027, 0.063, 0.081, 0.729$, ordered with R slowest and L fastest. Summing out R gives $P(T, L) = (0.051, 0.119, 0.083, 0.747)$, and summing out T gives $P(+l) = 0.134$ and $P(-l) = 0.866$.

Variable elimination marginalizes early: join on R, sum out R to obtain $P(T) = (0.17, 0.83)$, then join with $P(L \mid T)$ and sum out T. The answer is the same, but no factor ever has more than two variables.

Example: *Evidence.* For $P(L \mid +r)$, the initial factors are $P(+r) = 0.1$, $P(T \mid +r) = (0.8, 0.2)$, and $P(L \mid T)$. Eliminating T leaves the selected joint

$$P(+r, +l) = 0.1(0.8 \cdot 0.3 + 0.2 \cdot 0.1) = 0.026, \qquad P(+r, -l) = 0.074,$$

which normalizes to $P(+l \mid +r) = 0.26$ and $P(-l \mid +r) = 0.74$.

12.4 General variable elimination

Algorithm 12.1: Variable elimination

Input: Query Q, evidence $E_1 = e_1, \ldots, E_k = e_k$, Bayes net CPTs.
Output: The distribution $P(Q \mid e_1, \ldots, e_k)$.

$\mathcal{F} \leftarrow$ the CPTs, with each evidence value selected
while some hidden variable (neither Q nor evidence) remains **do**
  choose a hidden variable H
  $\mathcal{G} \leftarrow$ the factors of $\mathcal{F}$ that mention H
  $f \leftarrow \sum_h \prod_{g \in \mathcal{G}} g$ *// join on H, then sum it out*
  $\mathcal{F} \leftarrow (\mathcal{F} \setminus \mathcal{G}) \cup \{f\}$
return the normalized product of all factors in $\mathcal{F}$

In the alarm example with factors $P(B)$, $P(E)$, $P(A \mid B, E)$, $P(+j \mid A)$, and $P(+m \mid A)$:

1. eliminate A: join $P(A \mid B, E)$, $P(+j \mid A)$, and $P(+m \mid A)$ into $P(+j, +m, A \mid B, E)$, then sum to $f_1(B, E) = P(+j, +m \mid B, E)$;
2. eliminate E: join $P(E)$ and f_1 into $P(+j, +m, E \mid B)$, then sum to $f_2(B) = P(+j, +m \mid B)$;
3. join $P(B)$ and f_2 into $P(+j, +m, B)$ and normalize.

In equations, each step applies $x(y + z) = xy + xz$:

$$P(B \mid +j, +m) \propto \sum_{e,a} P(B)P(e)P(a \mid B, e)P(+j \mid a)P(+m \mid a) = P(B) \sum_e P(e) f_1(B, e) = P(B) f_2(B).$$

A joined factor can be named mechanically. Variables left of the bar in the original factors stay on the left, and all remaining variables go on the right. In the treasure network below, T is taking a treasure, C a falling cage, S released snakes, and E escaping. To compute $P(T \mid +e)$, eliminate C and then S:

1. join $P(C \mid T)$ and $P(+e \mid C, S)$ into $f_1(C, +e, T, S) = P(C, +e \mid T, S)$, then sum to $f_2 = P(+e \mid T, S)$;
2. join $P(S \mid T)$ and f_2 into $f_3 = P(+e, S \mid T)$, then sum to $f_4 = P(+e \mid T)$;
3. form $f_5(+e, T) = f_4\, P(T)$ and normalize.

The name is justified by the chain rule: $P(T, C, S, +e) = P(S, T)\, P(C \mid T)\, P(+e \mid C, S)$, so $P(C \mid T)P(+e \mid C, S) = P(T, C, S, +e)/P(S, T) = P(C, +e \mid T, S)$, even though its rows need not sum to one. The largest factor has 8 rows, rather than the 16 of the full joint. Enumeration computes $\alpha \sum_s \sum_c P(T)P(s \mid T)P(c \mid T)P(+e \mid c, s)$; elimination computes $\alpha P(T) \sum_s P(s \mid T) \sum_c P(c \mid T)P(+e \mid c, s)$, the same expression with irrelevant terms moved outside each sum.

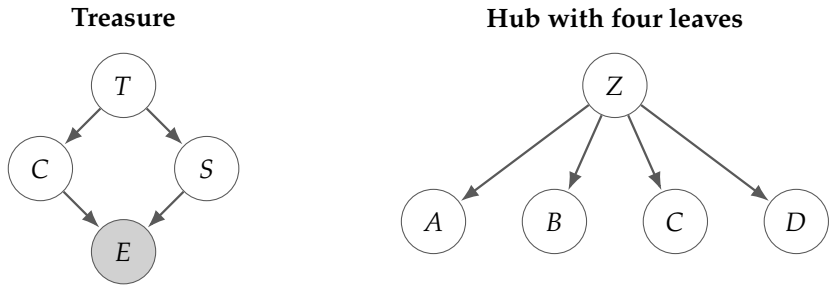


Figure 20: Left: the treasure network, with evidence E shaded. Right: a hub network in which the elimination order determines the largest factor.

12.5 Elimination order and complexity

Variable elimination improves on enumeration only if the largest factor stays small. For the hub network, eliminating the leaves first,

$$P(D) = \alpha \sum_{z} P(z) \sum_{a} P(a \mid z) \sum_{b} P(b \mid z) \sum_{c} P(c \mid z)\, P(D \mid z),$$

creates factors over at most two variables, D and Z. Eliminating Z first,

$$P(D) = \alpha \sum_{a} \sum_{b} \sum_{c} \sum_{z} P(a \mid z) P(b \mid z) P(c \mid z) P(D \mid z) P(z),$$

joins every factor and creates a factor over A, B, C, D; with n leaves it has 2^n entries. In the car-insurance net, enumeration needs 227 million operations and elimination 221 thousand.

Time and space are governed by the largest factor, and space is usually the binding constraint. The ordering can change its size dramatically, as in the 2^n versus 2 above, but no ordering keeps factors small in every net.

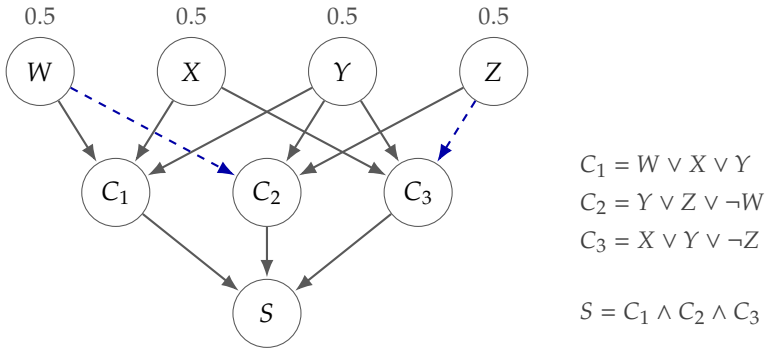


Figure 21: A Bayes net encoding a CNF sentence. Each variable has prior 0.5; each clause node is the deterministic disjunction of its literals, with dashed arcs entering negated, and S is their conjunction.

Proposition 12.2 [Hardness of exact inference]. *Exact inference in Bayes nets is NP-hard, and in fact #P-hard.*

Proof. Given a CNF sentence over n variables, give each variable an independent uniform prior, add one node per clause whose CPT deterministically computes the disjunction of its literals, and add a node S computing their conjunction. The net has size polynomial in the sentence, and $P(+s) = K \cdot 2^{-n}$, where K is the number of satisfying assignments. Deciding whether $P(+s) > 0$ decides satisfiability, and computing $P(+s)$ counts satisfying assignments. □

The displayed sentence has 11 satisfying assignments of W, X, Y, Z, so $P(+s) = 11/16$.

A polytree is a directed graph with no undirected cycles. On polytrees, variable elimination takes time linear in the network size when it eliminates from the leaves toward the roots. For a near-polytree, cut-set conditioning chooses a set of variables whose removal leaves a polytree, instantiates them, and solves each resulting polytree separately.

Exact inference evaluates sums of products of CPT entries. Enumeration is always exponential; variable elimination avoids recomputing repeated subexpressions, giving large practical speedups and linear time on polytrees, though the problem is #P-hard in general.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 12 (Spring 2024); Josh Hug and Jacky Liang, edited by Regina Wang, Note 12.

Lecture 13: Bayes Nets: Sampling

13.1 Why sample

Exact inference can be prohibitively expensive. Sampling approximates a query instead: draw N samples from a sampling distribution S, compute an approximate posterior from them, and show that the estimate converges to the true probability P. Sampling is often a fast way to a decent answer. The algorithms are simple and general, need only $O(n)$ memory, and still apply to large models where exact algorithms blow up.

Example: *Monopoly.* The probability that program A beats program B is $\sum_s P(s)V(s)$, where s ranges over sequences of dice rolls and cards and $V(s) \in \{0, 1\}$ records a win. There are infinitely many sequences, but playing N games with sampled sequences s_i estimates the probability by the fraction of wins, $\frac{1}{N} \sum_i V(s_i)$.

Samples come from simple primitives. To sample a categorical distribution, draw u uniform on $[0, 1)$ and assign each outcome a subinterval of length equal to its probability. For $P(\text{red}) = 0.6$, $P(\text{green}) = 0.1$, and $P(\text{blue}) = 0.3$, the intervals are $[0, 0.6)$, $[0.6, 0.7)$, and $[0.7, 1)$, so $u = 0.83$ yields blue. Even an unknown bias can be removed: to get a fair bit from a coin with unknown heads probability p, flip twice until the outcomes differ, and output 0 for HT and 1 for TH. Both outcomes have probability $p(1 - p)$.

The running example is the sprinkler network, with these CPTs:

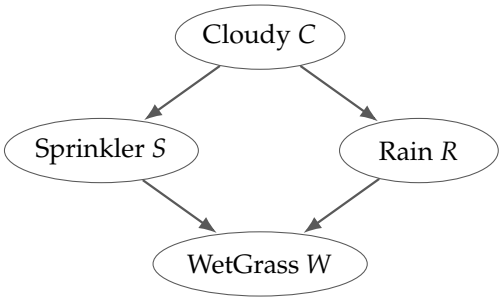


Figure 22: The sprinkler network.

$P(+c)$	C	$P(+s \mid C)$	C	$P(+r \mid C)$	S	R	$P(+w \mid S, R)$
0.5	$+c$	0.1	$+c$	0.8	$+s$	$+r$	0.99
	$-c$	0.5	$-c$	0.2	$+s$	$-r$	0.90
					$-s$	$+r$	0.90
					$-s$	$-r$	0.01

13.2 Prior sampling

Prior sampling simulates the net: visit the variables in topological order and sample each from its CPT given the sampled values of its parents.

Algorithm 13.1: Prior sampling, with optional rejection

Input: Bayes net over $X_1, \dots, X_n$ in topological order; optional evidence e.
Output: A sample $(x_1, \dots, x_n)$, or no sample.

```
for i = 1 to n do
    sample x_i from P(X_i | Pa(X_i)) using the sampled parent values
    if evidence is given and x_i contradicts it then
        return no sample                                   // rejection
return (x_1, ..., x_n)
```

Proposition 13.1 [Consistency of prior sampling]. *A prior sample equals $(x_1, \dots, x_n)$ with probability*

$$S_{PS}(x_1, \dots, x_n) = \prod_i P(x_i \mid \mathrm{Pa}(X_i)) = P(x_1, \dots, x_n).$$

If $N_{PS}(x_1, \dots, x_n)$ counts that event among N samples, the estimate $Q_N = N_{PS}/N$ satisfies $\lim_{N\to\infty} Q_N = S_{PS} = P$.

The limit is the law of large numbers, so the procedure is consistent. For instance,

$$S_{PS}(+c, -s, +r, +w) = 0.5 \times 0.9 \times 0.8 \times 0.9 = 0.324.$$

From the five samples

$$(+c, -s, +r, +w),\ (+c, +s, +r, +w),\ (-c, +s, +r, -w),\ (+c, -s, +r, +w),\ (-c, -s, -r, +w),$$

the counts for W are 4 and 1, giving the estimate $P(W) \approx (0.8, 0.2)$. The exact value is $P(+w) = 0.65$; more samples move the estimate closer.

Prior sampling wastes effort on unlikely evidence. In a two-variable net with $P(+t) = 0.99$, estimating $P(C \mid -t)$ discards 99% of the samples.

13.3 Rejection sampling

Rejection sampling estimates a conditional probability by counting only samples consistent with the evidence. For $P(C \mid +r, +w) = \alpha P(C, +r, +w)$, samples with $-r$ or $-w$ are irrelevant. Checking each variable as it is sampled rejects inconsistent samples early, so bad samples are at least cheap. The procedure is also consistent: the accepted samples are prior samples conditioned on the evidence.

Of the samples

$$(+c,-s,+r,+w),\ (+c,+s,-r),\ (-c,+s,+r,-w),\ (+c,-s,-r),\ (-c,-s,+r,+w),$$

three are rejected, and the remaining two estimate $P(+c \mid +r,+w) \approx 1/2$. The exact value is about 0.79. When the evidence is unlikely, almost every sample is rejected; in the car-insurance net, the error of rejection sampling for a cost query is still about 0.01 after a million samples.

13.4 Likelihood weighting

Rejection sampling ignores the evidence while sampling. To estimate $P(\mathit{Shape} \mid \mathit{Color} = \text{blue})$ in a net $\mathit{Shape} \to \mathit{Color}$, it throws away every sample that is not blue. Fixing the evidence and sampling only the rest wastes nothing, but then the samples have the wrong distribution: upstream variables never account for how likely the evidence is. Likelihood weighting fixes the evidence and weights each sample by the probability of the evidence given its parents.

Algorithm 13.2: Likelihood weighting: one weighted sample

Input: Bayes net over $X_1,\dots,X_n$ in topological order, evidence e.
Output: A sample $(x_1,\dots,x_n)$ and weight w.

```
w ← 1
for i = 1 to n do
    if X_i is an evidence variable then
        x_i ← its observed value
        w ← w · P(x_i | Pa(X_i))
    else
        sample x_i from P(X_i | Pa(X_i))
return (x_1, ..., x_n), w
```

The estimate of $P(Q = q \mid e)$ is the total weight of samples with $Q = q$ divided by the total weight of all samples. With evidence $+s$ and $+w$ in the sprinkler net, sampling $+c$ and $+r$ gives the sample $(+c,+s,+r,+w)$ with weight

$$w = 1.0 \times P(+s \mid +c) \times P(+w \mid +s,+r) = 0.1 \times 0.99 = 0.099.$$

Proposition 13.2 [Consistency of likelihood weighting]. *Let Z be the sampled variables and $E = e$ the evidence. The weighted sampling distribution equals the joint:*

$$S_{WS}(z,e)\,w(z,e) = \prod_j P(z_j \mid \mathrm{Pa}(Z_j)) \prod_k P(e_k \mid \mathrm{Pa}(E_k)) = P(z,e).$$

Proof. Each sampled variable is drawn from its CPT given values already fixed, so $S_{WS}(z,e) = \prod_j P(z_j \mid \mathrm{Pa}(Z_j))$. The weight multiplies in exactly the evidence CPT entries, $w(z,e) = \prod_k P(e_k \mid$

$\mathrm{Pa}(E_k)$). Every CPT appears once in the product, giving $P(z, e)$. The expected weight of samples with $Q = q$ is therefore proportional to $P(q, e)$, and normalizing the weighted counts converges to $P(q \mid e)$. □

Likelihood weighting is an instance of IMPORTANCE SAMPLING. To estimate a quantity under a distribution P that is hard to sample, sample from a tractable Q and weight each sample x by $P(x)/Q(x)$.

All samples are used, and downstream variables are influenced by upstream evidence. On the car-insurance query, its error is far below that of rejection sampling for the same number of samples. Weaknesses remain. Upstream variables are unaffected by downstream evidence; given a video of a traffic accident, the causes are still sampled from their priors. With evidence at k leaves, weights are typically $O(2^{-k})$, and one lucky sample can carry much more weight than all the others and dominate the result. Ideally every variable would see all the evidence.

13.5 Markov chain Monte Carlo

MARKOV CHAIN MONTE CARLO (MCMC) is a family of randomized algorithms for approximating quantities over very large state spaces. A MARKOV CHAIN is a random walk in which each state is chosen conditioned only on the previous state; a MONTE CARLO algorithm is a sampling-based algorithm that may return an incorrect answer. MCMC wanders around for a while and averages what it sees.

Example: *Random walks.* Walk on an undirected graph by following a uniformly random edge out of the current node. In the long run the fraction of time at node v is $\pi(v) = \deg(v)/(2|E|)$, since

$$\sum_u \pi(u) \frac{m_{uv}}{\deg(u)} = \sum_u \frac{m_{uv}}{2|E|} = \frac{\deg(v)}{2|E|} = \pi(v),$$

where m_{uv} counts edges between u and v. On a triangle every node gets $1/3$. If the edge between b and c is doubled, the degrees become $2, 3, 3$ and the fractions $1/4$, $3/8$, $3/8$.

13.6 Gibbs sampling

GIBBS SAMPLING is MCMC whose states are complete assignments, as in local search and simulated annealing. Evidence variables stay fixed. Initialize the others arbitrarily, then repeatedly pick a non-evidence variable X_i and resample it from

$$\begin{aligned} P(X_i \mid x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_n) &= P(X_i \mid \text{Markov blanket of } X_i) \\ &= \alpha\, P(X_i \mid \mathrm{Pa}(X_i)) \prod_j P(y_j \mid \mathrm{Pa}(Y_j)), \end{aligned}$$

where the Y_j are the children of X_i. In the joint $\prod_k P(x_k \mid \mathrm{Pa}(X_k))$, only the CPT of X_i and those of its children mention x_i. All other factors cancel on normalizing, so each update needs only the Markov blanket. Moves tend toward states of higher probability but can go down too.

Algorithm 13.3: Gibbs sampling

Input: Bayes net, evidence e, query Q, number of steps N.
Output: An estimate of $P(Q \mid e)$.

$x \leftarrow$ evidence values with random values for the non-evidence variables
for $t = 1$ **to** N **do**
  choose a non-evidence variable X_i
  sample x_i from $P(X_i \mid \text{Markov blanket of } X_i)$
  Increment the count for the current value of Q
return the normalized counts

For $P(S \mid +r)$ in the sprinkler net, fix $R = +r$, initialize C, S, W randomly, and cycle through updates such as $S \sim P(S \mid +c, +r, -w)$, $C \sim P(C \mid +s, +r)$, and $W \sim P(W \mid +s, +r)$. The exact answer is $P(+s \mid +r) = 0.18$.

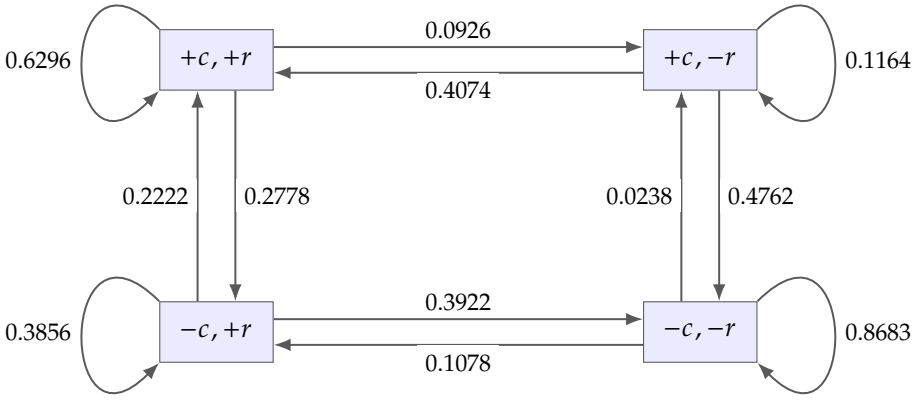


Figure 23: The Gibbs chain over (C, R) given $+s$ and $+w$, choosing C or R with probability $1/2$ at each step.

Theorem 13.3 [Consistency of Gibbs sampling]. *If variables are chosen fairly and every Gibbs conditional is bounded away from 0 and 1, the distribution of the state converges to $P(x_1, \ldots, x_n \mid e)$.*

Proof. Let π_t be the distribution of the state after t steps and $k(x' \mid x)$ the probability that one step moves x to x'. Then $\pi_{t+1}(x') = \sum_x k(x' \mid x)\pi_t(x)$, or $\pi_{t+1} = K\pi_t$. The posterior is a fixed point of each update of X_i: writing x_{-i} for the other variables,

$$\sum_{x_i} P(x_i, x_{-i} \mid e)\, P(x_i' \mid x_{-i}, e) = P(x_{-i} \mid e)\, P(x_i' \mid x_{-i}, e) = P(x_i', x_{-i} \mid e),$$

so $K\pi = \pi$ for $\pi = P(\cdot \mid e)$. Under the stated conditions the chain is ERGODIC, meaning connected and aperiodic, and such a chain has a unique stationary distribution to which π_t converges. □

The chain in the figure illustrates the theorem. Its stationary distribution gives probabilities 0.1424, 0.0324, 0.1780, and 0.6472 to $(+c, +r)$, $(+c, -r)$, $(-c, +r)$, and $(-c, -r)$, exactly the posterior $P(C, R \mid +s, +w)$. How long the chain takes to get there depends on the CPTs, and convergence is slower when they are nearly deterministic. Samples soon reflect all the evidence in the network. On the car-insurance cost query, Gibbs sampling is noisier than likelihood weighting at first. For a query about an upstream variable such as age, it is far more accurate.

In practice Gibbs sampling and other MCMC methods are the most common choice for large Bayes nets, with tools such as BUGS, JAGS, Stan, infer.net, and BLOG. Compiled samplers remove data-structure overhead and just multiply and sample, reaching about 100 million samples per second on a laptop. They can run asynchronously in parallel, with one processor per variable, and some cognitive scientists suggest that the brain runs on MCMC.

Method	Estimates	Idea
Prior sampling	$P(x_1, \ldots, x_n)$	generate complete samples from the joint
Rejection sampling	$P(Q \mid e)$	discard samples that do not match e
Likelihood weighting	$P(Q \mid e)$	weight samples by how well they predict e
Gibbs sampling	$P(Q \mid e)$	wander through assignments consistent with e

Source. Stuart Russell and Dawn Song, *CS 188: Introduction to Artificial Intelligence*, Lecture 13 (Spring 2024); Josh Hug and Jacky Liang, edited by Regina Wang, Note 13.

Lecture 14: HMMs: Markov Chains and Hidden Markov Models

Many tasks reason about a sequence of observations while the underlying system changes: speech recognition, robot localization, user attention, medical monitoring, and climate. Such models need an explicit notion of time, with one random variable per time step.

14.1 Markov models

A Markov model (Markov chain) has a state X_t at each time $t = 0, 1, 2, \ldots$, usually discrete and finite. It is specified by an initial distribution $P(X_0)$ and a transition model $P(X_t \mid X_{t-1})$ describing how the state evolves. Two assumptions make this compact:

- Stationarity: the transition probabilities are the same at every time step, so one table $P(X_t \mid X_{t-1})$ serves for all t.
- The Markov property: the future is independent of the past given the present,

$$X_{t+1} \perp\!\!\!\perp \{X_0, \ldots, X_{t-1}\} \mid X_t.$$

  This is a first-order model; a kth-order model lets X_{t+1} depend on the k preceding states.

By the chain rule and the Markov property,

$$P(X_0, \ldots, X_T) = P(X_0) \prod_{t=1}^{T} P(X_t \mid X_{t-1}).$$

For example, $P(X_0, X_1, X_2) = P(X_0)P(X_1 \mid X_0)P(X_2 \mid X_1, X_0)$ reduces to $P(X_0)P(X_1 \mid X_0)P(X_2 \mid X_1)$ because $X_2 \perp\!\!\!\perp X_0 \mid X_1$.

A Markov model is a Bayes net in one sense: it is a directed acyclic graph whose joint distribution is the product of its conditionals. It is not a standard Bayes net in another: it has infinitely many variables unless truncated, and the repeated use of one transition table is not part of ordinary Bayes net syntax.

Example: *Three chains.* A *random walk* on the integers starts at 0 and moves to $k \pm 1$ with probability 0.5 each. Its expected distance from the origin grows as $O(\sqrt{t})$; it returns to 0 with probability 1 in one and two dimensions, but only with probability 0.34053733 in three. An *n-gram model* of text takes the state to be the word at position t, with transition probabilities from empirical frequencies: a unigram model uses $P(\text{Word}_t)$, a bigram model $P(\text{Word}_t \mid \text{Word}_{t-1})$, and a trigram model conditions on two previous words. These support text classification, spam detection, author and language identification, and speech recognition. A *web browser* at URL X_t follows a random

outgoing link with probability p and jumps to an arbitrary page with probability $1 - p$; the long-run fraction of time spent on each page underlies PageRank.

14.2 Prediction with the mini-forward algorithm

The running example has states $\{\text{sun}, \text{rain}\}$, initial distribution $P(X_0) = \langle 0.5, 0.5 \rangle$, and the transition model in Figure 24.

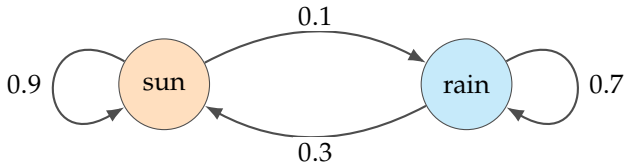


Figure 24: The weather chain. Arrows are labeled with $P(X_t \mid X_{t-1})$: sun stays sunny with probability 0.9, and rain turns sunny with probability 0.3.

Computing $P(X_t)$ by summing the full joint costs $O(d^t)$ for d states. Instead, marginalize over the previous state:

$$P(X_t) = \sum_{x_{t-1}} P(X_t, x_{t-1}) = \sum_{x_{t-1}} P(x_{t-1})\, P(X_t \mid x_{t-1}).$$

This MINI-FORWARD ALGORITHM is a recursive update $P_t = g(P_{t-1})$, iterated from P_0; each step costs $O(d^2)$. For the weather chain,

$$\begin{aligned} P(X_1) &= 0.5\langle 0.9, 0.1 \rangle + 0.5\langle 0.3, 0.7 \rangle = \langle 0.6, 0.4 \rangle, \\ P(X_2) &= 0.6\langle 0.9, 0.1 \rangle + 0.4\langle 0.3, 0.7 \rangle = \langle 0.66, 0.34 \rangle, \\ P(X_3) &= 0.66\langle 0.9, 0.1 \rangle + 0.34\langle 0.3, 0.7 \rangle = \langle 0.696, 0.304 \rangle. \end{aligned}$$

In matrix form, let $T_{ij} = P(X_t = j \mid X_{t-1} = i)$. Then $P_t = T^{\mathsf{T}} P_{t-1}$; for instance

$$P(X_2) = \begin{pmatrix} 0.9 & 0.3 \\ 0.1 & 0.7 \end{pmatrix} \begin{pmatrix} 0.6 \\ 0.4 \end{pmatrix} = \begin{pmatrix} 0.66 \\ 0.34 \end{pmatrix}.$$

14.3 Stationary distributions

The limiting distribution P_∞ of the chain, when it exists, is a STATIONARY DISTRIBUTION: one more step of the transition model leaves it unchanged,

$$P_\infty = P_{\infty+1} = T^{\mathsf{T}} P_\infty, \qquad \text{that is,} \qquad P_\infty(x') = \sum_x P(x' \mid x) P_\infty(x).$$

For d states this gives d linear equations, which together with $\sum_x P_\infty(x) = 1$ determine P_∞ here. Writing $P_\infty = \langle p, 1-p \rangle$ for the weather chain,

$$0.9p + 0.3(1-p) = p \quad \Longrightarrow \quad p = 0.75.$$

So $P_\infty = \langle 0.75, 0.25 \rangle$, and the predictions $0.6, 0.66, 0.696, \ldots$ approach 0.75. For this chain the same limit is reached from every starting distribution.

The browsing chain above asks exactly this question: its stationary distribution gives the fraction of time a long walk spends on each page.

Note. Gibbs sampling is a Markov chain whose states are complete assignments to the Bayes net variables. Its transitions use only local information, and its stationary distribution is the desired posterior $P(X_1, \ldots, X_n \mid \mathbf{e})$. The key fact is that when X_i is resampled, the ratio of the probabilities of moving to $X_i = x_i$ versus $X_i = x_i'$ equals

$$\frac{P(x_1, \ldots, x_{i-1}, x_i, x_{i+1}, \ldots, x_n \mid \mathbf{e})}{P(x_1, \ldots, x_{i-1}, x_i', x_{i+1}, \ldots, x_n \mid \mathbf{e})}.$$

14.4 Hidden Markov models

Usually the state is not observed directly. A HIDDEN MARKOV MODEL (HMM) has an underlying Markov chain over hidden states X_t and an observed EVIDENCE VARIABLE E_t at each step $t \geq 1$. Here X_t is a single discrete variable, while E_t may be continuous or consist of several variables. An HMM is specified by three tables: the initial distribution $P(X_0)$, the transition model $P(X_t \mid X_{t-1})$, and the SENSOR MODEL $P(E_t \mid X_t)$, which is also stationary.

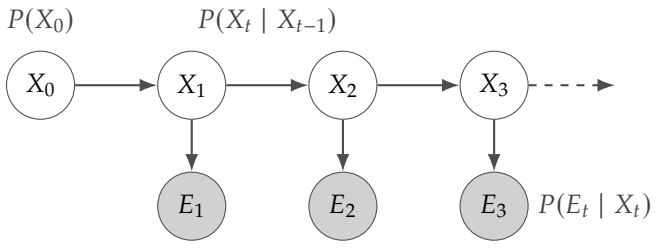


Figure 25: A hidden Markov model; shaded evidence nodes are observed. Deleting the evidence row leaves a Markov chain.

The joint distribution is

$$P(X_0, X_1, E_1, \ldots, X_T, E_T) = P(X_0) \prod_{t=1}^{T} P(X_t \mid X_{t-1})\, P(E_t \mid X_t).$$

Future states are independent of the past given the present state, and the current evidence is

independent of everything else given the current state:

$$X_{t+1} \perp\!\!\!\perp \{X_0, \ldots, X_{t-1}, E_1, \ldots, E_t\} \mid X_t, \qquad E_t \perp\!\!\!\perp \{X_0, \ldots, X_{t-1}, E_1, \ldots, E_{t-1}\} \mid X_t.$$

The evidence variables are *not* independent of one another: the path $E_1 \leftarrow X_1 \rightarrow X_2 \rightarrow E_2$ is active while the states are hidden, so earlier evidence is informative about later evidence. We write $x_{a:b} = x_a, x_{a+1}, \ldots, x_b$, and similarly $e_{1:t}$.

Example: *Weather and umbrellas.* The hidden weather W_t follows the chain in Figure 24, and the evidence U_t records whether an umbrella is seen:

$$P(U_t = \text{true} \mid W_t = \text{sun}) = 0.2, \qquad P(U_t = \text{true} \mid W_t = \text{rain}) = 0.9.$$

Real HMMs include speech recognition, with continuous acoustic observations and states for positions within words (tens of thousands of them); machine translation, with words as observations and translation options as states; robot tracking, with continuous range readings and map positions; and molecular biology, with nucleotides ACGT observed and coding, non-coding, start, stop, or splice-site states hidden.

14.5 Inference tasks

Given evidence $e_{1:t}$, the standard queries are:

- Filtering: $P(X_t \mid e_{1:t})$, the belief state that a rational agent uses to decide.
- Prediction: $P(X_{t+k} \mid e_{1:t})$ for $k > 0$, used to evaluate action sequences; it is filtering without the newest evidence.
- Smoothing: $P(X_k \mid e_{1:t})$ for $0 \le k < t$, a better estimate of past states that is essential for learning.
- Most likely explanation: $\arg\max_{x_{1:t}} P(x_{1:t} \mid e_{1:t})$, as in speech recognition or decoding a noisy channel.

Filtering, also called monitoring or state estimation, maintains $f_{1:t} = P(X_t \mid e_{1:t})$ over time, starting from an initial f_0 that is usually uniform. A common notation writes $B(X_t) = P(X_t \mid e_{1:t})$ and $B'(X_t) = P(X_t \mid e_{1:t-1})$ for the belief before the newest evidence. The Kalman filter, for continuous variables with linear dynamics and Gaussian noise, dates to 1960 and was used for trajectory estimation in the Apollo program; its core ideas go back to Gauss's planetary observations.

Example: *Robot localization.* A robot moves in a known grid map. Each reading gives four bits, wall or no wall in each direction, with never more than one bit wrong; each move may fail with small probability. The belief starts uniform. After the first reading it concentrates on cells consistent with the reading, with less weight on cells that require one sensor mistake. As the robot keeps moving

and sensing, the cells that remain consistent with the whole history shrink, and after five steps nearly all probability lies on the true cell.

Source. Stuart Russell and Dawn Song, *CS 188: Introduction to Artificial Intelligence*, Lecture 14 (Spring 2024); Nikhil Sharma, Note 14.

Lecture 15: HMMs: Forward and Viterbi Algorithms, Dynamic Bayes Nets and Particle Filtering

15.1 Recursive filtering

We seek a recursive update $P(X_{t+1} \mid e_{1:t+1}) = g\big(e_{t+1}, P(X_t \mid e_{1:t})\big)$, so that the belief is carried forward in constant time and space per step.

Proposition 15.1 [Forward update]. *In an HMM,*

$$P(X_{t+1} \mid e_{1:t+1}) = \alpha\, P(e_{t+1} \mid X_{t+1}) \sum_{x_t} P(X_{t+1} \mid x_t)\, P(x_t \mid e_{1:t}),$$

where $\alpha = 1/P(e_{t+1} \mid e_{1:t})$ does not depend on X_{t+1}.

Proof. Split off the newest evidence and apply Bayes' rule with $e_{1:t}$ held fixed:

$$P(X_{t+1} \mid e_{1:t}, e_{t+1}) = \frac{P(e_{t+1} \mid X_{t+1}, e_{1:t})\, P(X_{t+1} \mid e_{1:t})}{P(e_{t+1} \mid e_{1:t})}.$$

A variable is independent of its non-descendants given its parents, so $P(e_{t+1} \mid X_{t+1}, e_{1:t}) = P(e_{t+1} \mid X_{t+1})$: once X_{t+1} is known, nothing else helps predict e_{t+1}. For the second factor, condition on X_t and marginalize:

$$P(X_{t+1} \mid e_{1:t}) = \sum_{x_t} P(X_{t+1} \mid x_t, e_{1:t})\, P(x_t \mid e_{1:t}) = \sum_{x_t} P(X_{t+1} \mid x_t)\, P(x_t \mid e_{1:t}),$$

using that X_{t+1} is independent of the earlier evidence given its parent X_t. □

The update has three parts. The sum is the time elapse or predict step, $B'(X_{t+1}) = \sum_{x_t} P(X_{t+1} \mid x_t) B(x_t)$, which is the mini-forward algorithm. Multiplying by the sensor model is the observation or update step, $B(X_{t+1}) \propto P(e_{t+1} \mid X_{t+1}) B'(X_{t+1})$. Finally, α normalizes. Normalization can be postponed: the unnormalized vectors equal $P(X_t, e_{1:t})$, so one may normalize once at the end.

Algorithm 15.1: Forward algorithm (filtering)

Input: HMM tables $P(X_0)$, $P(X_t \mid X_{t-1})$, $P(E_t \mid X_t)$; evidence $e_1, \dots, e_T$.
Output: The belief $f_{1:T} = P(X_T \mid e_{1:T})$.

$f \leftarrow P(X_0)$
for $t = 1$ **to** T **do**
  for each state x **do**
    $b'(x) \leftarrow \sum_{x'} P(X_t = x \mid x')\, f(x')$ *// predict*
    $b(x) \leftarrow P(e_t \mid x)\, b'(x)$ *// update*
  $f \leftarrow b / \sum_x b(x)$ *// normalize*
return f

Writing $f_{1:t+1} = \textsc{Forward}(f_{1:t}, e_{t+1})$, each step costs $O(|X|^2)$ for $|X|$ states, independent of t. In matrix form, with the transition matrix T and a diagonal observation matrix O_t holding $P(e_t \mid x)$ on its diagonal,

$$f_{1:t+1} = \alpha\, O_{t+1} T^{\mathsf{T}} f_{1:t}, \qquad \text{e.g.} \quad O_1 = \begin{pmatrix} 0.2 & 0 \\ 0 & 0.9 \end{pmatrix} \text{ for } U_1 = \text{true}.$$

Quadratic cost in $|X|$ is infeasible when the state consists of many variables, which motivates approximate filters.

Example: *Umbrella filtering.* Use the weather chain with $P(\text{sun} \mid \text{sun}) = 0.9$, $P(\text{sun} \mid \text{rain}) = 0.3$, sensor $P(U = \text{true} \mid \text{sun}) = 0.2$, $P(U = \text{true} \mid \text{rain}) = 0.9$, and $f_0 = \langle 0.5, 0.5 \rangle$ over $\langle \text{sun}, \text{rain} \rangle$. Umbrellas appear on days 1 and 2.

$$\begin{aligned} &\text{Day 1:} \quad \text{predict } \langle 0.6, 0.4 \rangle, \quad \text{update } \propto \langle 0.12, 0.36 \rangle \to \langle 0.25, 0.75 \rangle; \\ &\text{Day 2:} \quad \text{predict } \langle 0.45, 0.55 \rangle, \quad \text{update } \propto \langle 0.09, 0.495 \rangle \to \langle 0.154, 0.846 \rangle. \end{aligned}$$

Each umbrella shifts belief toward rain, and the second sighting reinforces the first.

15.2 Most likely explanation and the Viterbi algorithm

The most likely state sequence given the evidence is

$$\arg\max_{x_{1:t}} P(x_{1:t} \mid e_{1:t}) = \arg\max_{x_{1:t}} P(x_{1:t}, e_{1:t}) = \arg\max_{x_{1:t}} P(x_0) \prod_{s=1}^{t} P(x_s \mid x_{s-1})\, P(e_s \mid x_s),$$

since $P(e_{1:t})$ does not depend on $x_{1:t}$. The initial state x_0 is maximized over as well. Draw a state trellis: one column of states per time step, with an arc $x_{s-1} \to x_s$ of weight $P(x_s \mid x_{s-1}) P(e_s \mid x_s)$ and arcs into the first column of weight $P(x_0)$. A path's weight product is its joint probability with the evidence. The forward algorithm sums over all paths into each state; Viterbi takes the maximum.

Define $m_t(x_t) = \max_{x_{0:t-1}} P(x_{0:t}, e_{1:t})$, the weight of the best path ending in x_t. Since the factors

involving earlier states do not depend on x_t once x_{t-1} is fixed, the maximum splits:

$$m_t(x_t) = P(e_t \mid x_t) \max_{x_{t-1}} P(x_t \mid x_{t-1}) \max_{x_{0:t-2}} P(x_{0:t-1}, e_{1:t-1}) = P(e_t \mid x_t) \max_{x_{t-1}} P(x_t \mid x_{t-1})\, m_{t-1}(x_{t-1}),$$

with $m_0(x_0) = P(x_0)$. A backpointer $a_t(x_t) = \arg\max_{x_{t-1}} P(x_t \mid x_{t-1}) m_{t-1}(x_{t-1})$ records the last transition of that best path.

Algorithm 15.2: Viterbi algorithm

Input: HMM tables and evidence $e_1, \dots, e_T$.
Output: A most likely state sequence $x^*_{0:T}$.

for each state x **do** $m_0(x) \leftarrow P(X_0 = x)$
for $t = 1$ **to** T **do**
  for each state x **do**
    $a_t(x) \leftarrow \arg\max_{x'} P(x \mid x')\, m_{t-1}(x')$
    $m_t(x) \leftarrow P(e_t \mid x)\, P(x \mid a_t(x))\, m_{t-1}(a_t(x))$
$x^*_T \leftarrow \arg\max_x m_T(x)$
for $t = T$ **to** 1 **do** $x^*_{t-1} \leftarrow a_t(x^*_t)$
return $x^*_{0:T}$

The forward pass costs $O(|X|^2 T)$ time, and the backpointers take $O(|X|T)$ space, although there are $|X|^{T+1}$ paths.

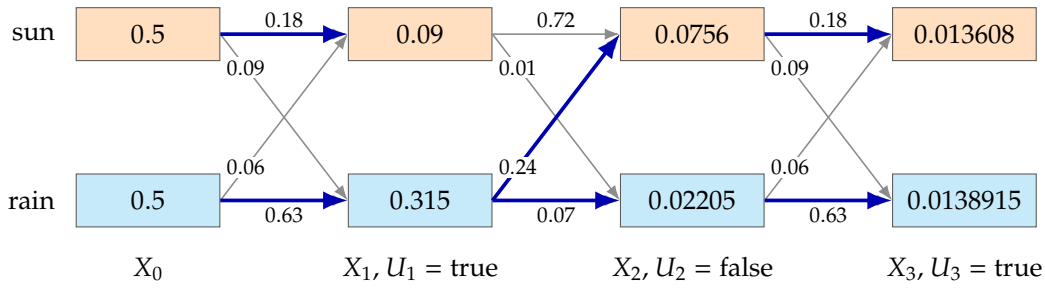


Figure 26: Viterbi on the umbrella model with U_1 = true, U_2 = false, U_3 = true and $P(X_0) = \langle 0.5, 0.5 \rangle$. Arc labels are $P(x_t \mid x_{t-1})P(u_t \mid x_t)$; the label nearer an arc's start belongs to that crossing arc. Boxes show m_t, and blue arcs are the backpointers.

For instance, $m_2(\text{sun}) = 0.8 \max(0.9 \cdot 0.09,\ 0.3 \cdot 0.315) = 0.8 \cdot 0.0945 = 0.0756$, so the best path into sun at time 2 comes from rain. At time 3, $m_3(\text{rain}) = 0.0138915$ beats $m_3(\text{sun}) = 0.013608$, and following the backpointers gives rain, rain, rain, rain. The day-2 evidence alone would favor sun, but switching costs more than it gains.

Taking $-\log$ of each arc weight turns the maximum product into the minimum sum: the most likely explanation is a minimum-cost path through the trellis from a start node to the final column. For example, the step-1 arc rain→rain costs $-\log_2 0.63 \approx 0.67$ bits. Viterbi is then a layer-by-layer shortest-path search, and shortest-path search methods such as A* also apply.

15.3 Dynamic Bayes nets

To track several variables from several evidence sources, a DYNAMIC BAYES NET (DBN) repeats a fixed Bayes net structure at each time step; variables at time t may have parents at time t or $t-1$.

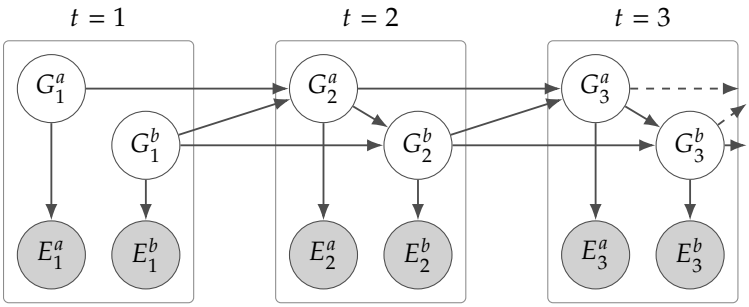


Figure 27: A DBN tracking two ghost positions G_t^a and G_t^b with separate shaded (observed) sensor readings. Dashed arrows continue the pattern to later slices.

Every HMM is a DBN with one state variable, and every discrete DBN is an HMM whose state is the Cartesian product of the DBN state variables. The DBN is far more compact when dependencies are sparse. With 20 Boolean state variables each having 3 parents, the DBN needs about $20 \cdot 2^3 = 160$ parameters, while the equivalent HMM transition matrix has $2^{20} \times 2^{20} \approx 10^{12}$ entries.

Variable elimination applies to DBNs. Offline, UNROLL the network for T steps and eliminate everything except X_T to obtain $P(X_T \mid e_{1:T})$. Online, eliminate the previous slice's variables after each step and store factors for the current slice only. The difficulty is that the largest factor then contains all variables of the current slice, plus a few more, so exact inference is exponential in the number of state variables.

Example: *ICU monitoring.* The state describes the patient's physiology, the evidence comes from monitoring devices, the transition model encodes physiological and sensor dynamics, and queries ask about dangerous conditions. A heart-rate model distinguishes a slowly varying *parameter* (resting heart rate), *state* variables (true heart rate, patient movement), *sensor state* variables (whether the ECG or pulse oximeter has detached), and *sensor* readings. Modeling sensor faults lets the DBN explain artifacts in arterial blood-pressure readings, such as pressure-bag and zeroing events, instead of raising false alarms. In the reported comparison it detected bag events more accurately than an SVM, a model-based classifier, or a physician.

15.4 Particle filtering

When $|X|$ exceeds about 10^6, exact filtering is infeasible; three ghosts in a 10×20 grid already give $200^3 = 8 \times 10^6$ states. Likelihood weighting over whole trajectories also fails: samples are drawn from the transition model while ignoring the evidence, so they drift into low-probability regions, and the number needed grows exponentially with T. On the umbrella model, its average error

approaches 1 within 50 steps even with 10,000 samples, while a particle filter with 25 samples stays accurate. The fix is RESAMPLING: kill the samples that fit the evidence badly and copy the ones that fit well, keeping the population in the high-probability region.

A PARTICLE FILTER represents the belief by a list of $N \ll |X|$ PARTICLES, each a complete assignment to the state variables at one time step. $P(x)$ is approximated by the fraction of particles with value x, so many states have estimated probability 0; more particles give more accuracy. Low-dimensional marginals, such as one ghost's position, are estimated more accurately in relative terms than joint events. Each step takes time linear in N:

1. *Predict*: move each particle by sampling $x_{t+1}^{(j)} \sim P(X_{t+1} \mid x_t^{(j)})$.
2. *Weight*: after observing e_{t+1}, set $w^{(j)} = P(e_{t+1} \mid x_{t+1}^{(j)})$, as in likelihood weighting.
3. *Resample*: draw N new particles with replacement, with probability proportional to weight, and reset all weights to $1/N$. If every weight is zero, reinitialize the particles.

Particle	1	2	3	4	5	6	7	8	9	10
At time t	(3,3)	(2,3)	(3,3)	(3,2)	(3,3)	(3,2)	(1,2)	(3,3)	(3,3)	(2,3)
Predicted	(3,2)	(2,3)	(3,2)	(3,1)	(3,3)	(3,2)	(1,3)	(2,3)	(3,2)	(2,2)
Weight	0.9	0.2	0.9	0.4	0.4	0.9	0.1	0.2	0.9	0.4
Normalized	0.17	0.04	0.17	0.08	0.08	0.17	0.02	0.04	0.17	0.08
Resampled	(3,2)	(2,2)	(3,2)	(2,3)	(3,3)	(3,2)	(1,3)	(2,3)	(3,2)	(3,2)

Here most particles move clockwise, and the observation favors $(3, 2)$: the weights sum to 5.3, so each 0.9 normalizes to 0.17.

Example: *Temperature particles.* Let $T_i \in \{10, \ldots, 20\}$. Each step stays put or moves one degree; the option closest to 15 has probability 0.8 and the others split 0.2 evenly. The particles [15, 12, 12, 10, 18, 14, 12, 11, 11, 10] give $B(10) = B(11) = 0.2$, $B(12) = 0.3$, and $B(14) = B(15) = B(18) = 0.1$. To sample $P(T_{i+1} \mid T_i = 15) = \langle 0.1, 0.8, 0.1 \rangle$ over $\langle 14, 15, 16 \rangle$, map a uniform $r \in [0, 1)$ to 14 if $r < 0.1$, to 15 if $r < 0.9$, and to 16 otherwise. With r = [0.467, 0.452, 0.583, 0.604, 0.748, 0.932, 0.609, 0.372, 0.402, 0.026], prediction gives [15, 13, 13, 11, 17, 15, 13, 12, 12, 10]. The forecast is correct with probability 0.8 and equals each other value with probability 0.02. After forecast 13, the three particles at 13 weigh 0.8 and the rest 0.02, for a total of 2.54; state 13 gets $2.4/2.54 \approx 0.945$. Resampling with [0.315, 0.829, 0.304, 0.368, 0.459, 0.891, 0.282, 0.980, 0.898, 0.341] gives nine particles at 13 and one at 15.

The particle filter is consistent: as $N \to \infty$ its estimates converge to the exact filtered distribution, provided the DBN probabilities are bounded away from 0. It is how robot localization works in practice. In SLAM (simultaneous localization and mapping) the robot knows neither map nor location, so a particle holds a pose and a map; usually each map is inferred exactly from that particle's sampled poses (a Rao–Blackwellized particle filter).

Source. Stuart Russell and Dawn Song, *CS 188: Introduction to Artificial Intelligence*, Lecture 15 (Spring 2024); Nikhil Sharma, Note 15 and Note 16.

Lecture 16: Utility Theory, Rationality, Decision Networks and Value of Perfect Information

16.1 Utilities and maximum expected utility

A utility function maps outcomes, or states of the world, to real numbers describing an agent's preferences. In a game it may be as simple as ± 1; in general it summarizes the agent's goals. We hard-wire utilities and let behavior emerge, rather than hard-wiring behavior. The principle of maximum expected utility (MEU) says that a rational agent chooses the action maximizing its expected utility given its knowledge. This raises the questions of where utilities come from, why they should exist, and why averaging them is meaningful.

The scale of utilities matters for averaging. For worst-case minimax reasoning, only the order of terminal values matters: the optimal decision is invariant under any monotonic transformation. For average-case expectimax reasoning, magnitudes matter. With leaves $(0, 40)$ under one action and $(20, 30)$ under the other, both minimax and expectimax prefer the second (min: $0 < 20$; mean: $20 < 25$). After squaring the leaves to $(0, 1600)$ and $(400, 900)$, minimax still prefers the second, but the means become $800 > 650$ and expectimax switches to the first.

16.2 Rational preferences

An agent has preferences among prizes $A, B, \ldots$ and lotteries, situations with uncertain prizes. The lottery $L = [p, A; 1 - p, B]$ yields A with probability p and B otherwise. We write $A \succ B$ for strict preference, $A \sim B$ for indifference, and $A \succeq B$ for either.

Some constraints are needed before preferences can be called rational. An agent with intransitive preferences $A \succ B$, $B \succ C$, $C \succ A$ can be drained of all its money: holding C, it pays a cent for B; then a cent for A; then a cent for C, and the cycle repeats.

Definition 16.1 [Axioms of rationality]. For all prizes or lotteries A, B, C:

- Orderability: $(A \succ B) \vee (B \succ A) \vee (A \sim B)$.
- Transitivity: $(A \succ B) \wedge (B \succ C) \Rightarrow A \succ C$.
- Continuity: $A \succ B \succ C \Rightarrow \exists p \;\; [p, A; 1 - p, C] \sim B$.
- Substitutability: $A \sim B \Rightarrow [p, A; 1 - p, C] \sim [p, B; 1 - p, C]$.
- Monotonicity: $A \succ B \Rightarrow \big(p \geq q \iff [p, A; 1 - p, B] \succeq [q, A; 1 - q, B]\big)$.

Theorem 16.2 [Ramsey 1931; von Neumann and Morgenstern 1944]. *Given preferences satisfying*

these axioms, there is a real-valued function U such that

$$U(A) \geq U(B) \iff A \succeq B, \qquad U([p_1, S_1; \ldots; p_n, S_n]) = \sum_{i=1}^{n} p_i U(S_i).$$

Thus U preserves preferences over both prizes and lotteries, and rational behavior is describable as maximizing expected utility. The optimal policy is invariant under any positive affine transformation $U' = aU + b$ with $a > 0$, since every expected utility transforms the same way. Rationality does not require the agent to represent or manipulate utilities and probabilities: a lookup table for perfect tic-tac-toe is rational.

16.3 Human utilities and money

A standard way to elicit a human utility for a prize A compares it with a STANDARD LOTTERY L_p that gives the best possible prize $S_\top$ with probability p and the worst possible catastrophe $S_\bot$ otherwise. Adjust p until $A \sim L_p$; normalizing $U(S_\top) = 1$ and $U(S_\bot) = 0$, the resulting p is $U(A) \in [0, 1]$. For example, someone indifferent between paying \$50 and the lottery [0.999999, no change; 0.000001, instant death] reveals how they trade money against a one-in-a-million risk of death.

Money is not itself a utility, but we can speak of the utility of having money. For $L = [p, \$X; 1-p, \$Y]$, the EXPECTED MONETARY VALUE is $\mathrm{EMV}(L) = pX + (1-p)Y$, while $U(L) = pU(\$X) + (1-p)U(\$Y)$. Typically $U(L) < U(\mathrm{EMV}(L))$: people are RISK-AVERSE. The CERTAINTY EQUIVALENT $\mathrm{CE}(L)$ is the cash amount with $\mathrm{CE}(L) \sim L$, and the INSURANCE PREMIUM is $\mathrm{EMV}(L) - \mathrm{CE}(L)$. A risk-neutral agent would pay no premium; risk-averse agents pay one to escape lotteries such as a house fire, a data breach, or the loss of a family's income.

Example: *Three attitudes.* Compare $L = [0.5, \$0; 0.5, \$1000]$ with a sure \$500 under $U_1(\$x) = x$, $U_2(\$x) = \sqrt{x}$, and $U_3(\$x) = x^2$:

Utility	Lottery	Sure \$500	Attitude
x	500	500	risk-neutral (indifferent)
$\sqrt{x}$	15.81	22.36	risk-averse (takes the payment)
x^2	500,000	250,000	risk-seeking (takes the lottery)

For instance $U_2(L) = 0.5\sqrt{1000} + 0.5\sqrt{0} \approx 15.81 < \sqrt{500}$. A concave utility makes the spread of the lottery costly.

16.4 Utilities of sequences

For sequences of rewards we must decide between more and less, such as $[1, 2, 2]$ versus $[2, 3, 4]$, and between now and later, such as $[0, 0, 1]$ versus $[1, 0, 0]$. Preferences are STATIONARY if prefixing the same reward does not change them:

$$[a_1, a_2, \ldots] \succ [b_1, b_2, \ldots] \iff [c, a_1, a_2, \ldots] \succ [c, b_1, b_2, \ldots].$$

Theorem 16.3 [STATIONARY PREFERENCES]. *If preferences over reward sequences are stationary, the only consistent utility is the* ADDITIVE DISCOUNTED UTILITY

$$U([r_0, r_1, r_2, \ldots]) = r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots,$$

where $\gamma \in (0, 1]$ *is the* DISCOUNT FACTOR.

16.5 Decision networks

A DECISION NETWORK operationalizes MEU by adding action and utility nodes to a Bayes net:

- CHANCE NODES (ovals) behave exactly as in Bayes nets.
- ACTION NODES (rectangles) are chosen by the agent; they have no parents and act as observed evidence.
- A UTILITY NODE (diamond) depends on action and chance nodes and outputs their utility.

To select an action: instantiate all evidence; set the action node each possible way; compute the posterior of all chance-node parents of the utility node given the evidence; compute each action's expected utility; and choose a maximizing action. With evidence e and chance parents $X_1, \ldots, X_n$ of U,

$$\mathrm{EU}(a \mid e) = \sum_{x_1, \ldots, x_n} P(x_1, \ldots, x_n \mid e)\, U(a, x_1, \ldots, x_n), \qquad \mathrm{MEU}(e) = \max_a \mathrm{EU}(a \mid e).$$

Inside $\mathrm{EU}(\cdot)$ we write the action, after the bar the evidence; inside $\mathrm{MEU}(\cdot)$ we write only the evidence, with $\mathrm{MEU}(\varnothing)$ for none.

Example: *Umbrella.* Let $U(\text{leave}, \text{sun}) = 100$, $U(\text{leave}, \text{rain}) = 0$, $U(\text{take}, \text{sun}) = 20$, $U(\text{take}, \text{rain}) = 70$, and $P(\text{sun}) = 0.7$. With no evidence,

$$\mathrm{EU}(\text{leave}) = 0.7 \cdot 100 + 0.3 \cdot 0 = 70, \qquad \mathrm{EU}(\text{take}) = 0.7 \cdot 20 + 0.3 \cdot 70 = 35,$$

so $\mathrm{MEU}(\varnothing) = 70$ by leaving. If the forecast is bad, inference gives $P(\text{sun} \mid F = \text{bad}) = 0.34$, and

$$\mathrm{EU}(\text{leave} \mid \text{bad}) = 0.34 \cdot 100 = 34, \qquad \mathrm{EU}(\text{take} \mid \text{bad}) = 0.34 \cdot 20 + 0.66 \cdot 70 = 53,$$

so MEU(F = bad) = 53 by taking the umbrella.

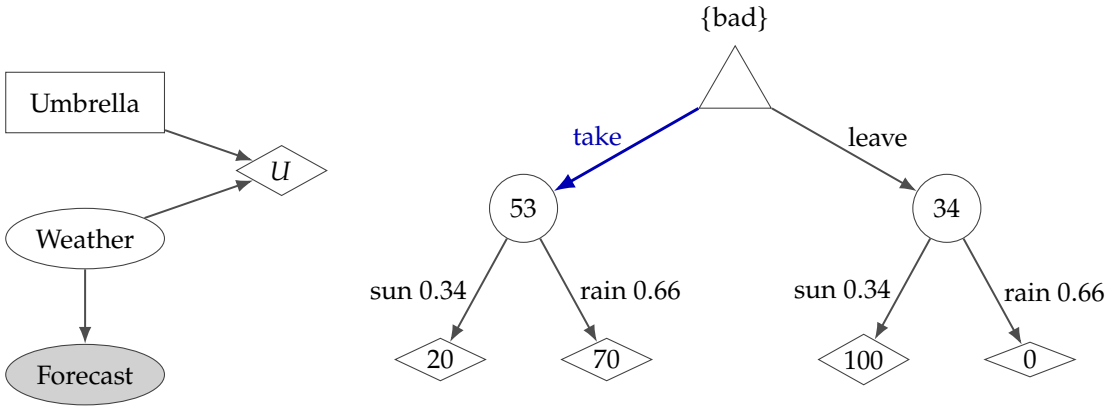


Figure 28: The umbrella decision network, with the shaded forecast observed, and its outcome tree given a bad forecast. The root maximizes, chance nodes average with the posterior $P(W \mid F = \text{bad})$, and diamonds hold $U(A, W)$.

Unrolled, a decision is an OUTCOME TREE, almost exactly an expectimax tree. The difference is that each node is annotated with what is known at that moment, such as {bad}, and the chance probabilities are posteriors from Bayes net inference. In the Ghostbusters game, for instance, the action is where to bust, the utility depends on the action and the ghost's location, and each sensor is a child of the ghost location whose reading is a noisy distance.

16.6 Value of perfect information

Evidence usually costs something, so an agent must decide whether observing more is worthwhile. Suppose we know $E = e$; acting now is worth $\text{MEU}(e) = \max_a \sum_s P(s \mid e)U(s, a)$. If we first saw $E' = e'$, acting would be worth $\text{MEU}(e, e') = \max_a \sum_s P(s \mid e, e')U(s, a)$. Since e' is unknown, we average over it:

$$\text{MEU}(e, E') = \sum_{e'} P(e' \mid e)\,\text{MEU}(e, e').$$

Definition 16.4 [VALUE OF PERFECT INFORMATION**].** The VALUE OF PERFECT INFORMATION of E' given e is the expected gain in MEU from observing E' before acting:

$$\text{VPI}(E' \mid e) = \text{MEU}(e, E') - \text{MEU}(e) = \left[\sum_{e'} P(e' \mid e) \max_a \text{EU}(a \mid e, e')\right] - \max_a \text{EU}(a \mid e).$$

It is the fair price of the observation: observe when the VPI exceeds the cost, as when an oil company considers seismic tests, a doctor a blood test, or a pedestrian a look before crossing. The decision network contains everything needed to compute it.

Example: *Oil drilling.* Exactly one of blocks a and b holds oil worth k, each with prior $1/2$, and we may drill in one. Either choice has expected utility $k/2$, so MEU = $k/2$. If a survey reveals the oil

location, we drill there and earn k whichever way it reports. Hence VPI(OilLoc) $= k - k/2 = k/2$.

Example: *Umbrella forecast.* With $P(F = \text{good}) = 0.59$, $P(F = \text{bad}) = 0.41$, MEU($F = \text{bad}$) $= 53$ (take), and MEU($F = \text{good}$) $= 95$ (leave),

$$\text{MEU}(F) = 0.59 \cdot 95 + 0.41 \cdot 53 = 77.78, \qquad \text{VPI}(F) = 77.78 - 70 = 7.78.$$

Proposition 16.5 [Properties of VPI]. *For any E' and e,* $\text{VPI}(E' \mid e) \geq 0$, *and observation order does not matter:*

$$\text{VPI}(E_j, E_k \mid e) = \text{VPI}(E_j \mid e) + \text{VPI}(E_k \mid e, E_j) = \text{VPI}(E_k \mid e) + \text{VPI}(E_j \mid e, E_k).$$

Proof. By total expectation, $\text{EU}(a \mid e) = \sum_{e'} P(e' \mid e)\text{EU}(a \mid e, e')$. A maximum of averages is at most the average of maxima, so

$$\max_a \text{EU}(a \mid e) \leq \sum_{e'} P(e' \mid e) \max_a \text{EU}(a \mid e, e') = \text{MEU}(e, E').$$

The inequality is strict when different values e' of positive probability make different actions uniquely optimal. For order independence, the two-step sums telescope: $\text{VPI}(E_j \mid e) + \text{VPI}(E_k \mid e, E_j) = \text{MEU}(e, E_j) - \text{MEU}(e) + \text{MEU}(e, E_j, E_k) - \text{MEU}(e, E_j)$, and similarly in the other order. Both equal $\text{MEU}(e, E_j, E_k) - \text{MEU}(e)$, since no action is taken until all observations are in. □

VPI is not additive in general. Observing the same E_j twice adds nothing the second time, so $\text{VPI}(E_j, E_j \mid e) = \text{VPI}(E_j \mid e)$, less than the sum of the two terms when it is positive. Observations can also be worth more together than separately: if X, Y are independent fair bits and the utility is 1 for guessing $X \oplus Y$ correctly, then $\text{VPI}(X) = \text{VPI}(Y) = 0$ but $\text{VPI}(X, Y) = 1/2$.

Example: *Quick values.* If you would order neither soup of the day, knowing which it is has VPI 0: no observation changes the decision. Knowing which of two kinds of plastic fork is slightly sturdier is worth little, since both choices are almost equally good. If a lottery pays \$100 on one of 100 numbers and you may pick any number, playing blind is worth \$1 in expectation, while knowing the winning number is worth \$100, a VPI of \$99 when utility is linear in money.

There is no separate value of imperfect information in this formulation. Information always means observing a node of the decision network; noisy data means observing a different variable that is a noisy version of the original, such as a scouting report about the oil location.

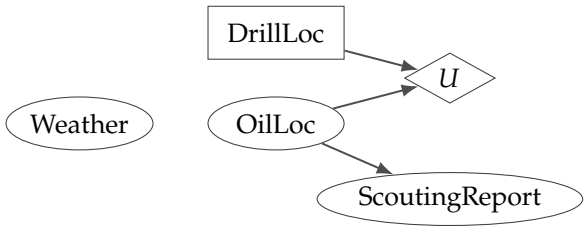


Figure 29: Oil drilling with a scouting report and an irrelevant weather variable.

Proposition 16.6 [Irrelevant evidence]. *If* $\mathrm{Pa}(U) \perp\!\!\!\perp Z \mid e$, *where* $\mathrm{Pa}(U)$ *are the chance-node parents of the utility node, then* $\mathrm{VPI}(Z \mid e) = 0$.

Proof. Each $\mathrm{EU}(a \mid e, z)$ depends on z only through $P(\mathrm{Pa}(U) \mid e, z) = P(\mathrm{Pa}(U) \mid e)$. So $\mathrm{MEU}(e, z) = \mathrm{MEU}(e)$ for every z, and averaging gives $\mathrm{MEU}(e, Z) = \mathrm{MEU}(e)$. □

In Figure 29, $\mathrm{VPI}(\text{Weather}) = 0$ because Weather is independent of OilLoc. Given OilLoc, the report is irrelevant, so $\mathrm{VPI}(\text{ScoutingReport} \mid \text{OilLoc}) = 0$. In contrast, $\mathrm{VPI}(\text{OilLoc} \mid \text{ScoutingReport})$ can be positive when the report is noisy, and $\mathrm{VPI}(\text{ScoutingReport}) \leq \mathrm{VPI}(\text{OilLoc})$, since learning OilLoc first makes the report worthless: by order independence, $\mathrm{VPI}(\text{OilLoc}, \text{ScoutingReport})$ equals both $\mathrm{VPI}(\text{OilLoc}) + 0$ and $\mathrm{VPI}(\text{ScoutingReport}) + \mathrm{VPI}(\text{OilLoc} \mid \text{ScoutingReport})$.

16.7 The optimizer's curse

We usually have only estimates of utilities. Suppose an unbiased expert estimates the net profits $V_1, \ldots, V_k$ of k investments, and we choose the best estimate V^*. With high probability the chosen investment's actual value is considerably less than V^*. If every true profit is 0 and each estimate is $\mathcal{N}(0, 1)$, then V^* is the maximum of k independent standard normals, with density $k\,\varphi(x)\Phi(x)^{k-1}$; its mean is about 0.85, 1.54, and 2.04 for $k = 3$, 10, and 30. The same disappointment affects top mutual funds, drug trials, statistical significance in papers, and auction winners.

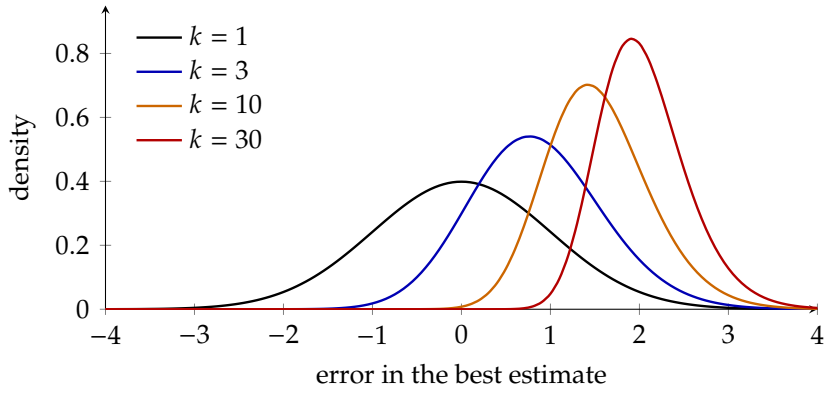


Figure 30: Error of the best of k unbiased estimates when every true value is 0. Choosing the maximum selects for favorable errors.

Source. Stuart Russell, Peyrin Kao, Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 16 (Spring 2024); Nikhil Sharma, Note 16.

Lecture 17: MDPs: States, Values, Policies and Q-values

A Markov decision process models sequential decisions whose actions have uncertain outcomes. It combines search over actions, probabilities over outcomes, and rewards accumulated over time.

17.1 Markov decision processes

Definition 17.1 [Markov decision process]. A Markov decision process (MDP) consists of

- a set of states S and a set of actions A;
- a transition function $T(s, a, s') = P(s' \mid s, a)$, the probability that action a in state s leads to s', also called the model or dynamics;
- a reward function $R(s, a, s')$, sometimes simplified to $R(s)$ or $R(s')$;
- a start state s_0, possibly terminal states, and possibly a discount factor γ.

In gridworld, an agent moves through a maze. Walls block movement, and actions are noisy: North moves north with probability 0.8 and west or east with probability 0.1 each. A move into a wall leaves the agent in place. Each step earns a small living reward, which may be negative, and the two exit squares give large final rewards. The goal is to maximize the sum of rewards.

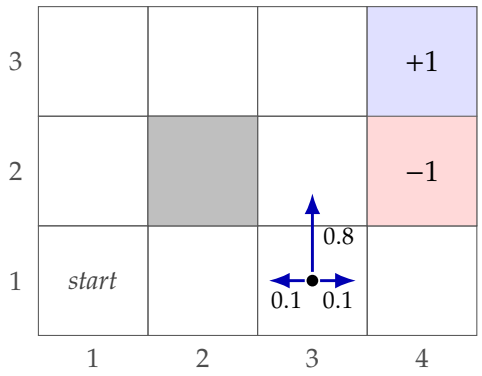


Figure 31: Gridworld. The square $(2, 2)$ is a wall, and exiting from $(4, 3)$ or $(4, 2)$ earns $+1$ or -1. Arrows show the outcomes of North from $(3, 1)$.

An MDP is a nondeterministic search problem; expectimax search is one way to solve it. It is Markov because action outcomes depend only on the current state:

$$P(S_{t+1} = s' \mid S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, \ldots, S_0 = s_0) = P(S_{t+1} = s' \mid S_t = s_t, A_t = a_t),$$

just as a search successor function depends only on the current state, not the history. A Markov chain has states alone, a hidden Markov model adds evidence emitted by hidden states, and an MDP adds chosen actions and rewards. Hidden states with actions give a partially observable MDP.

Example: *Racing.* A robot car wants to travel far, quickly. Its states are cool, warm, and overheated, which is terminal. Going fast earns double reward but risks overheating.

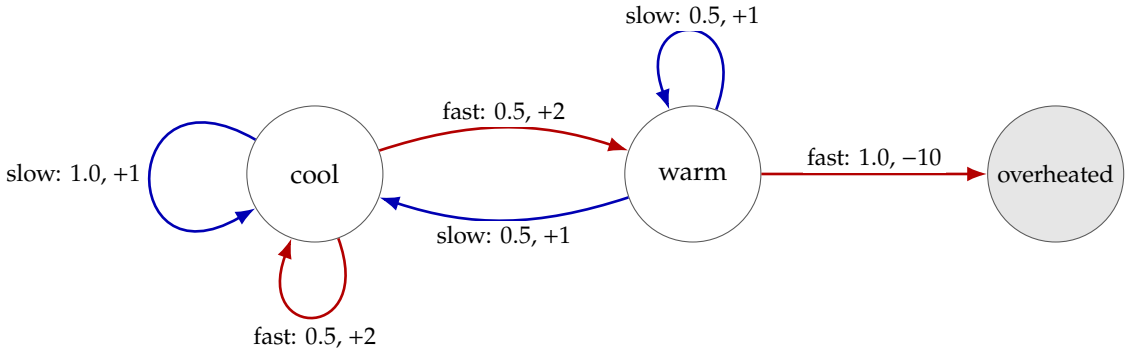


Figure 32: The racing MDP. Each edge gives the action, $T(s, a, s')$, and $R(s, a, s')$. Overheated has no actions.

17.2 Policies

In deterministic search we sought a plan, a sequence of actions from the start to a goal. An MDP instead needs a POLICY $\pi : S \to A$, an action for every state. An OPTIMAL POLICY π^* maximizes expected utility if followed. An explicit policy defines a reflex agent. Expectimax computes an action for a single state, not an entire policy.

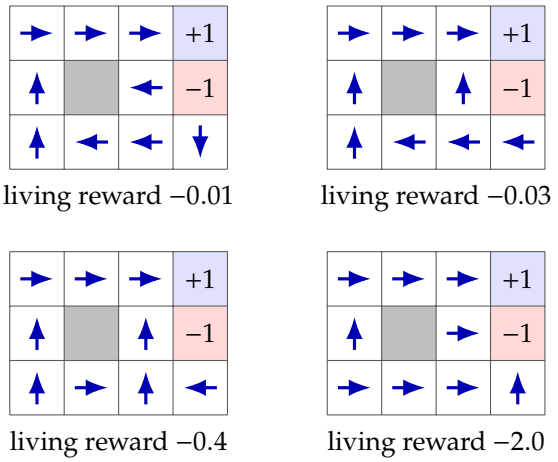


Figure 33: Optimal gridworld policies with noise 0.2 and no discounting, for four living rewards on nonterminal states.

The living reward sets how much risk is worth avoiding. With −0.01, steps are cheap: from $(3, 2)$ the agent heads west, the long way around, and from $(4, 1)$ it walks into the wall rather than risk

slipping into -1. With -0.03 and -0.4, it accepts more risk for shorter routes. With -2.0, living is so costly that from $(3, 2)$ it exits through -1 at once.

17.3 Utilities of reward sequences

An agent should prefer more reward to less, such as $[2, 3, 4]$ to $[1, 2, 2]$, and arguably reward now to reward later, such as $[1, 0, 0]$ to $[0, 0, 1]$. Discounting makes rewards decay exponentially: a reward is worth 1 now, γ one step later, and γ^2 two steps later. In a search tree, each level multiplies in one factor of γ. Discounting reflects the higher utility of sooner rewards and also helps algorithms converge. With $\gamma = 0.5$,

$$U([1, 2, 3]) = 1 + 0.5 \cdot 2 + 0.25 \cdot 3 = 2.75 < 4.25 = U([3, 2, 1]).$$

Theorem 17.2 [Stationary preferences]. *Suppose preferences over reward sequences satisfy*

$$[a_1, a_2, \ldots] \succ [b_1, b_2, \ldots] \iff [r, a_1, a_2, \ldots] \succ [r, b_1, b_2, \ldots].$$

Then utilities are either additive, $U([r_0, r_1, \ldots]) = r_0 + r_1 + r_2 + \cdots$, *or discounted,* $U([r_0, r_1, \ldots]) = r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots$.

Example: *Discounting.* States a, b, c, d, e lie in a row. East and West move deterministically; Exit is available only at a, paying 10, and at e, paying 1. Rewards are discounted per step, so walking i squares and then exiting at a is worth $10\gamma^i$. With $\gamma = 1$, every state heads west to collect 10. With $\gamma = 0.1$, the optimal policy and values are:

State	a	b	c	d	e
Action	Exit	West	West	East	Exit
Value	10	1	0.1	0.1	1

At d, West is worth $10\gamma^3$ and East is worth γ. They are equal when $\gamma^2 = 1/10$, that is, $\gamma = 1/\sqrt{10} \approx 0.316$.

If an episode can last forever, additive utilities may be infinite. Three remedies are:

- Finite horizon: end episodes after a fixed number of steps, as in depth-limited search. The resulting policies are nonstationary: the best action depends on the time left.
- *Discounting* with $0 < \gamma < 1$. If every reward is at most $R_{\max}$, then

$$U([r_0, r_1, \ldots]) = \sum_{t=0}^{\infty} \gamma^t r_t \le \sum_{t=0}^{\infty} \gamma^t R_{\max} = \frac{R_{\max}}{1 - \gamma}.$$

  A smaller γ gives a shorter effective horizon. A negative γ would make reward signs alternate

and is not meaningful in practice.

- Absorbing state: guarantee that every policy eventually reaches a terminal state, like overheated in racing.

17.4 Optimal quantities and the Bellman equations

Each state roots an expectimax-like search tree. A q-state (s, a) records that action a has been chosen in s but its outcome is not yet known. A transition (s, a, s') resolves the outcome with probability $T(s, a, s')$ and reward $R(s, a, s')$. The agent spends no time in q-states; they are a device for organizing the computation.

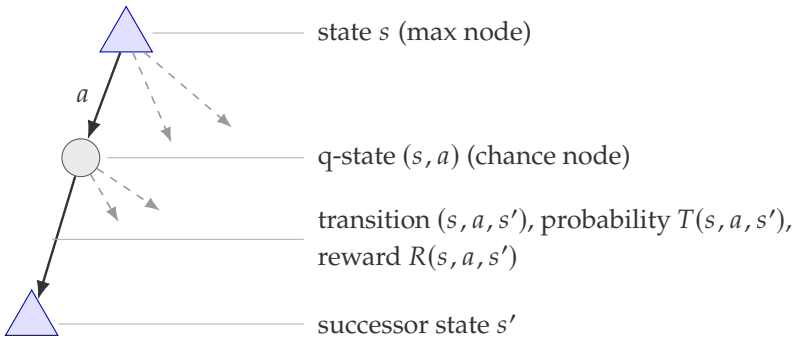


Figure 34: One layer of an MDP search tree. States are max nodes and q-states are chance nodes.

Definition 17.3 [Optimal quantities]. The value $V^*(s)$ is the expected utility of starting in s and acting optimally. The q-value $Q^*(s, a)$ is the expected utility of taking a in s and acting optimally thereafter. The optimal policy $\pi^*(s)$ is an optimal action from s.

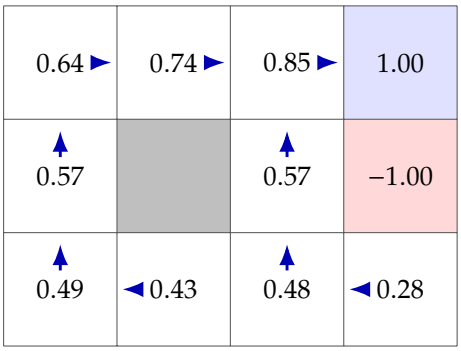


Figure 35: Optimal values and policy in gridworld with noise 0.2, $\gamma = 0.9$, and living reward 0.

Without noise and with $\gamma = 1$, every nonterminal state has value 1: the agent reaches +1 eventually and loses nothing by waiting. Noise 0.2 alone still gives value 1, since a patient agent can always step away from −1. Discounting by 0.9 makes distant states worth less, as in figure 35, and a living

reward of -0.1 lowers values further, to 0.01 at the start. The q-values at $(3, 2)$ in figure 35 are 0.57 for North, 0.53 for West, 0.30 for South, and -0.60 for East, toward -1.

Optimal values satisfy a one-step lookahead relation. The first action must be correct, and the agent must keep acting optimally afterwards:

$$V^*(s) = \max_a Q^*(s, a),$$
$$Q^*(s, a) = \sum_{s'} T(s, a, s')\big[R(s, a, s') + \gamma V^*(s')\big],$$
$$V^*(s) = \max_a \sum_{s'} T(s, a, s')\big[R(s, a, s') + \gamma V^*(s')\big].$$

These are the Bellman equations. The bracket is the utility of reaching s' through (s, a, s') and then acting optimally; Q^* averages it like an expectimax chance node, and V^* maximizes like a max node. The equations are a condition for optimality: values satisfying them at every state are the optimal values.

17.5 Time-limited values and value iteration

Expectimax on the racing tree does far too much work. States repeat, so each needed quantity should be computed once. The tree also goes on forever, so we compute depth-limited values with increasing depth until they change little. When $\gamma < 1$, deep parts of the tree eventually do not matter.

Definition 17.4 [Time-limited value]**.** $V_k(s)$ is the optimal value of s if the process ends in k more time steps. Equivalently, it is the value of depth-k expectimax from s.

Value iteration computes these values bottom up. With no time steps left, no reward can be earned, so $V_0(s) = 0$. Each iteration applies one ply of expectimax from every state:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s')\big[R(s, a, s') + \gamma V_k(s')\big].$$

Algorithm 17.1: Value iteration

Input: MDP (S, A, T, R, γ).
Output: Approximately optimal values V.

$V_0(s) \leftarrow 0$ for every $s \in S$
for $k = 0, 1, 2, \ldots$ until the values stop changing **do**
  for each $s \in S$ **do**
    $V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s, a, s')\big[R(s, a, s') + \gamma V_k(s')\big]$

Each iteration costs $O(|S|^2|A|)$: every state considers every action and every successor. The values converge to the unique optimal values, and the policy implied by them often converges long before

the values do.

Example: *Racing without discount.* From $V_0 = (0, 0, 0)$ for (cool, warm, overheated), one step gives $V_1(\text{cool}) = \max\{1, 2\} = 2$ and $V_1(\text{warm}) = \max\{1, -10\} = 1$. The next step gives

$$V_2(\text{cool}) = \max\{1 + 2,\ 0.5(2 + 2) + 0.5(2 + 1)\} = 3.5,$$
$$V_2(\text{warm}) = \max\{0.5(1 + 2) + 0.5(1 + 1),\ -10\} = 2.5.$$

Without discounting the values keep growing, since driving slowly forever earns unbounded reward; $V_3 = (5, 4, 0)$.

In gridworld with noise 0.2, $\gamma = 0.9$, and living reward 0, value spreads backward from the exits one square per iteration:

State	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$	$k = 6$	$k = 100$
$(3, 3)$	0	0.72	0.78	0.83	0.84	0.85	0.85
$(2, 3)$	0	0	0.52	0.66	0.72	0.73	0.74
$(3, 2)$	0	0	0.43	0.51	0.55	0.57	0.57
$(1, 3)$	0	0	0	0.37	0.51	0.59	0.64
$(1, 1)$	0	0	0	0	0	0.21	0.49

The extracted policy stops changing after iteration 10, while the values still move by up to 0.02.

Proposition 17.5 [Convergence of time-limited values]. *If the search tree has maximum depth M, then V_M holds the exact untruncated values. If $\gamma < 1$ and $|R| \le R_{\max}$, then for every state and every k,*

$$|V_{k+1}(s) - V_k(s)| \le \gamma^k R_{\max},$$

so the values $V_k(s)$ converge as $k \to \infty$.

Proof. With maximum depth M, depth-M expectimax already explores the whole tree. Otherwise view $V_k(s)$ and $V_{k+1}(s)$ as depth-$(k + 1)$ expectimax on the same tree. The only difference is the bottom layer: V_{k+1} receives the actual rewards there, while V_k receives zeros. Those rewards lie between $-R_{\max}$ and $R_{\max}$ and are discounted by γ^k. For any fixed way of choosing actions the two totals differ by at most $\gamma^k R_{\max}$, and maxima of functions within ε of each other are within ε. Since $\sum_k \gamma^k R_{\max} < \infty$, the sequence $V_k(s)$ is Cauchy and converges. □

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 17 (Spring 2024); Nikhil Sharma, Note 17.

Lecture 18: MDPs: Dynamic Programming

The Bellman equations characterize optimal values. This lecture computes them by dynamic programming, evaluates and extracts policies, and improves policies directly.

18.1 Value iteration as a fixed-point method

Write the value iteration update as $V_{k+1} = BV_k$, where the Bellman operator B maps a value function V to

$$(BV)(s) = \max_a \sum_{s'} T(s,a,s')\big[R(s,a,s') + \gamma V(s')\big].$$

The Bellman equations say exactly that $V^* = BV^*$, so value iteration is a method for finding a fixed point. The iterates V_k also keep their interpretation as time-limited values. The update rule and the Bellman equation look alike, but the first is a computation and the second is an optimality condition. Terminal states have no actions, so their values are 0.

Proposition 18.1 [Bellman operator is a contraction]. *For $0 < \gamma < 1$ and value functions V, V',*

$$\|BV - BV'\|_\infty \le \gamma \|V - V'\|_\infty, \qquad \textit{where } \|x\|_\infty = \max_s |x(s)|.$$

Proof. For any functions f, h, $|\max_z f(z) - \max_z h(z)| \le \max_z |f(z) - h(z)|$. Applying this at a state s, the rewards cancel:

$$\begin{aligned} |(BV)(s) - (BV')(s)| &\le \max_a \Big|\gamma \sum_{s'} T(s,a,s')\big(V(s') - V'(s')\big)\Big| \\ &\le \gamma \max_a \sum_{s'} T(s,a,s')\,\|V - V'\|_\infty = \gamma \|V - V'\|_\infty, \end{aligned}$$

since transition probabilities sum to 1 for every action. Take the maximum over s. □

Two consequences follow. If V and V' were both fixed points, then $\|V - V'\|_\infty \le \gamma \|V - V'\|_\infty$, so $V = V'$: the Bellman equations have a unique solution. Value iteration also converges geometrically:

$$\|V_k - V^*\|_\infty = \|BV_{k-1} - BV^*\|_\infty \le \gamma \|V_{k-1} - V^*\|_\infty \le \cdots \le \gamma^k \|V_0 - V^*\|_\infty.$$

Example: *Racing with $\gamma = 0.5$.* Recall that slow keeps a cool car cool for +1; fast from cool earns +2 and stays cool or becomes warm with probability 0.5 each; slow from warm earns +1 and cools or stays warm with probability 0.5 each; fast from warm overheats for −10. Starting from zero, with

the slow option listed first in each maximum,

$$V_1(\text{cool}) = \max\{1,\ 2\} = 2, \qquad V_1(\text{warm}) = \max\{1,\ -10\} = 1,$$
$$V_2(\text{cool}) = \max\{1 + 0.5 \cdot 2,\ 0.5(2 + 0.5 \cdot 2) + 0.5(2 + 0.5 \cdot 1)\} = 2.75,$$
$$V_2(\text{warm}) = \max\{0.5(1 + 0.5 \cdot 2) + 0.5(1 + 0.5 \cdot 1),\ -10\} = 1.75.$$

The values converge to $V^* = (3.5, 2.5)$. Indeed $0.5(2 + 1.75) + 0.5(2 + 1.25) = 3.5$ for fast from cool and $0.5(1 + 1.75) + 0.5(1 + 1.25) = 2.5$ for slow from warm, while the alternatives give 2.75 and -10, so $V^* = BV^*$.

18.2 Policy evaluation

Expectimax trees maximize over all actions. If we fix a policy π, each state has only one action, so the tree is simpler, though its value depends on the policy.

Definition 18.2 [Value of a fixed policy]. $V^\pi(s)$ is the expected total discounted reward from starting in s and following π. It satisfies the one-step lookahead relation

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s')\big[R(s, \pi(s), s') + \gamma V^\pi(s')\big].$$

Policy evaluation computes V^π in one of two ways:

- Turn the relation into updates, as in value iteration:

  $$V_0^\pi(s) = 0, \qquad V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s')\big[R(s, \pi(s), s') + \gamma V_k^\pi(s')\big].$$

  Each pass costs $O(|S|^2)$, since only one action is considered.

- Without the maximum, the relations form a linear system of $|S|$ equations in $|S|$ unknowns, which any linear solver handles.

In a corridor between -10 exits, going forward from the square below a $+100$ exit reaches it with probability 0.8 and slips sideways otherwise, so its value is $0.9\,[0.8 \cdot 100 + 0.1(-10) + 0.1(-10)] = 70.2$. Going right instead falls into -10 with probability 0.8, and its values are negative except next to the goal.

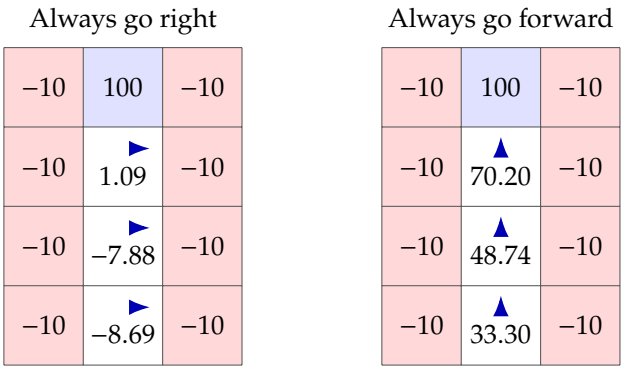


Figure 36: Values of two fixed policies with noise 0.2 and $\gamma = 0.9$. The side squares exit with -10 and the top middle square exits with $+100$.

18.3 Policy extraction

Given optimal values V^*, the best action is still not obvious. A one-step expectimax recovers it:

$$\pi^*(s) = \arg\max_a \sum_{s'} T(s,a,s')\big[R(s,a,s') + \gamma V^*(s')\big].$$

This is POLICY EXTRACTION: it gives the policy implied by a value function. With optimal q-values, acting is trivial: $\pi^*(s) = \arg\max_a Q^*(s,a)$. Actions are easier to select from q-values than from values: a single argmax replaces a one-step expectimax.

Q-VALUE ITERATION computes time-limited q-values directly:

$$Q_{k+1}(s,a) \leftarrow \sum_{s'} T(s,a,s')\big[R(s,a,s') + \gamma \max_{a'} Q_k(s',a')\big].$$

Only the position of the maximum changes: in a state the agent chooses an action before the transition, and in a q-state the transition happens before the next choice.

18.4 Policy iteration

Value iteration has three problems. Each iteration costs $O(|S|^2|A|)$; the maximizing action at each state rarely changes; and the policy often converges long before the values. In gridworld with noise 0.2 and $\gamma = 0.9$, for instance, the extracted policy is final from iteration 10 while the values are still changing.

POLICY ITERATION alternates two steps until the policy stops changing:

1. POLICY EVALUATION: compute V^{π_i} for the current, generally nonoptimal, policy π_i.

2. Policy improvement: apply one-step lookahead with these values,

$$\pi_{i+1}(s) = \arg\max_a \sum_{s'} T(s,a,s')\big[R(s,a,s') + \gamma V^{\pi_i}(s')\big].$$

Algorithm 18.1: Policy iteration

Input: MDP (S, A, T, R, γ) and an initial policy π_0.
Output: An optimal policy.

```
for i = 0, 1, 2, ... do
    Compute V^{π_i} by iterative updates or by solving the linear system
    for each s ∈ S do
        π_{i+1}(s) ← arg max_a Σ_{s'} T(s,a,s')[R(s,a,s') + γ V^{π_i}(s')]
    if π_{i+1} = π_i then
        return π_i
```

Policy iteration is still optimal. If $\pi_{i+1} = \pi_i$, then each $\pi_i(s)$ attains the maximum in the improvement step, and $V^{\pi_i}(s)$ equals the q-value of that action. Hence $V^{\pi_i} = BV^{\pi_i}$, and uniqueness of the fixed point gives $V^{\pi_i} = V^*$. Under some conditions it converges much faster than value iteration. The initial policy may be arbitrary, but one close to optimal speeds convergence.

Example: *Racing with* $\gamma = 0.5$. Start from π_0: always slow. Evaluation solves $V(\text{cool}) = 1+0.5V(\text{cool})$ and $V(\text{warm}) = 0.5[1 + 0.5V(\text{cool})] + 0.5[1 + 0.5V(\text{warm})]$, giving $V^{\pi_0} = (2, 2)$. Improvement compares

$$\begin{aligned} \text{cool}: &\ \text{slow } 1 \cdot [1 + 0.5 \cdot 2] = 2, \quad \text{fast } 0.5[2 + 0.5 \cdot 2] + 0.5[2 + 0.5 \cdot 2] = 3, \\ \text{warm}: &\ \text{slow } 0.5[1 + 0.5 \cdot 2] + 0.5[1 + 0.5 \cdot 2] = 2, \quad \text{fast } -10, \end{aligned}$$

so π_1 goes fast when cool and slow when warm. Evaluating π_1 gives $(3.5, 2.5) = V^*$, and improvement returns the same policy. Two improvement steps suffice, while value iteration is still far from converged after two updates.

Both methods compute the optimal values and are dynamic programs. Value iteration updates values and, implicitly, the policy in every iteration: the maximum recomputes the policy without tracking it. Policy iteration makes several fast passes with a fixed policy, considering one action per state, followed by one slower improvement pass. The new policy is better, or the algorithm is done. All of these methods are Bellman updates built from one-step expectimax fragments. They differ only in whether a fixed policy is plugged in or the maximum over actions is taken:

Goal	Method
Optimal values	Value iteration or policy iteration
Values of a particular policy	Policy evaluation
A policy from values	Policy extraction (one-step lookahead)

18.5 Planning versus learning

Every method above requires a model, meaning known T and R, as did search in gridworld, racing, Pacman, and route finding. Solving an MDP is OFFLINE PLANNING: all quantities are determined by computation, without acting.

Example: *Double bandit.* The actions are two slot machines, Blue and Red, and the states Win and Lose record the last outcome. Blue always pays \$1. Red pays \$2 with probability 0.75 and \$0 with probability 0.25. With no discount and 100 steps, both states have the same value; planning gives 150 for always playing Red and 100 for always playing Blue.

If Red's win probability is unknown, computation alone cannot choose. The agent must play to find out. One run of ten Red pulls paid \$2 only twice, which suggests a win chance near 0.2 and makes Blue look better. That is learning rather than planning, and specifically reinforcement learning: there is an MDP, but its parameters must be discovered by acting. It uses SAMPLING, since chance forces repeated trials, and PARAMETER ESTIMATION, the most likely explanation of the data. More data gives better estimates.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 18 (Spring 2024); Nikhil Sharma, Note 18.

Lecture 19: Machine Learning I

Earlier models were given, and we used them to reason and decide. Machine learning acquires a model from data or experience: its parameters (such as probabilities), its structure (such as a Bayes net graph), or hidden concepts (such as clusters or neural-network features). Supervised learning infers a relationship between inputs and labeled outputs in order to predict outputs for new inputs; unsupervised learning finds structure in unlabeled data. This lecture treats supervised classification with a model-based method, Naive Bayes.

19.1 Classification

In classification, each data point x has a label (class) y from a small discrete set, and the goal is an algorithm that predicts y from x. The pipeline extracts features, attributes of x that hopefully help classification, and runs a learning algorithm on them to output a predicted label. The algorithm learns patterns between features and labels from training data: many example points with their true labels. Someone must label this data by hand.

- *Spam filtering.* The input is an email and the label is spam or ham (wanted email). Features include words (`FREE!`), text patterns (`$dd`, all capitals), and non-text attributes such as whether the sender is in the contacts.
- *Digit recognition.* The input is a pixel grid and the label is a digit 0–9. Features include pixel values, such as whether pixel $(6, 8)$ is on, and shape patterns such as the number of connected components, aspect ratio, or number of loops. Increasingly, features are induced from data rather than crafted by hand.

Other examples are medical diagnosis, fraud detection, automatic essay grading, email routing, review sentiment, and language identification.

The data is split into three disjoint sets. Training data fits the model. Held-out (validation or development) data measures performance while we adjust hyperparameters, values that control the learning algorithm, or change algorithms. Test data is used once at the end, as a final exam that estimates performance on new data; we never "peek" at it during development. A common metric is accuracy, the fraction of instances predicted correctly.

19.2 The Naive Bayes model

A model-based classifier builds a Bayes net in which the label and the features are random variables, instantiates the observed features as evidence, and queries the distribution of the label. This raises

two questions: what structure the net should have, and how to learn its parameters.

With n binary features and a binary label, the full joint table has 2^{n+1} entries. The Naive Bayes model assumes that the features are conditionally independent given the label, so that

$$P(Y, F_1, \ldots, F_n) = P(Y) \prod_{i=1}^{n} P(F_i \mid Y).$$

The prior $P(Y)$ gives the probability of each label before any features are seen, and each table $P(F_i \mid Y)$ describes how the label affects one feature. In the binary case these tables have $2 + 4n$ entries, linear in n. In general the model stores $|Y| + n|F||Y|$ numbers instead of $|Y||F|^n$.

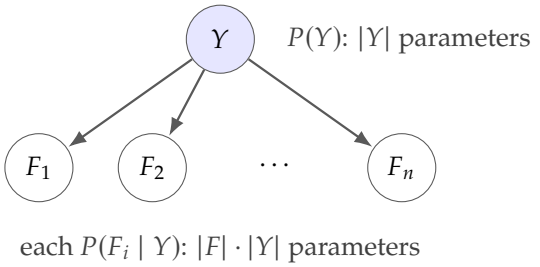


Figure 37: The Naive Bayes model. Every feature is a child of the label only, so d-separation makes the features independent given Y.

The assumption is strong, which is why the model is called naive, but it simplifies learning and inference and often works well. With few features one can add edges between them to obtain a richer model.

Training estimates $P(Y)$, how often each label occurs, and each $P(F_i \mid Y)$. Classification computes the posterior in three steps: the joint of each label with the evidence, their sum $P(f_1, \ldots, f_n)$, and the normalized ratio:

$$P(Y = y \mid f_1, \ldots, f_n) = \frac{P(y) \prod_i P(f_i \mid y)}{\sum_{y'} P(y') \prod_i P(f_i \mid y')}.$$

If only the label is needed, normalization can be skipped:

$$\text{prediction}(f_1, \ldots, f_n) = \arg\max_y \; P(y) \prod_{i=1}^{n} P(f_i \mid y).$$

Example: *A two-feature spam filter.* Let F_1 record whether the sender is known and F_2 the number of occurrences of *free*. In the six training emails below, the four spam emails contain *free* $0, 1, 1, 2$ times and the two ham emails 0 and 1 times, so counting gives $P(F_2 = 0 \mid \text{spam}) = 1/4$, $P(F_2 = 1 \mid \text{spam}) = 2/4$, $P(F_2 = 2 \mid \text{spam}) = 1/4$, and $P(F_2 \mid \text{ham}) = (1/2, 1/2, 0)$.

Email	Label	Count of *free*
Attached is my portfolio.	ham	0
Are you free for a meeting tomorrow?	ham	1
Free unlimited credit cards!!!!	spam	1
Mail $10,000 check to this address	spam	0
Sign up now for 1 free Bitcoin	spam	1
Free money free money	spam	2

Suppose a model trained on a larger dataset has $P(\text{ham}) = 0.6$, $P(F_1 = \text{yes} \mid \text{ham}) = 0.7$, $P(F_1 = \text{yes} \mid \text{spam}) = 0.1$, $P(F_2 = 1 \mid \text{ham}) = 0.07$, and $P(F_2 = 1 \mid \text{spam}) = 0.12$. For "Free food in Soda 430 today" from a known sender, the evidence is $F_1 = \text{yes}$, $F_2 = 1$:

$$P(\text{spam}, F_1{=}\text{yes}, F_2{=}1) = 0.4{\cdot}0.1{\cdot}0.12 = 0.0048, \qquad P(\text{ham}, F_1{=}\text{yes}, F_2{=}1) = 0.6{\cdot}0.7{\cdot}0.07 = 0.0294.$$

Normalizing gives $P(\text{spam} \mid \text{evidence}) = 0.0048/0.0342 \approx 0.14$ and $P(\text{ham} \mid \text{evidence}) \approx 0.86$. Comparing the joints alone already predicts ham.

19.3 Naive Bayes for digits and text

For digits, use one binary feature $F_{i,j}$ per grid position, on when the pixel intensity exceeds 0.5. Then

$$P(Y \mid F_{0,0}, \ldots, F_{15,15}) \propto P(Y) \prod_{i,j} P(F_{i,j} \mid Y),$$

and learning means estimating $P(Y)$ and one table per pixel.

For text, a BAG-OF-WORDS model lets W_i be the word at position i of the document, not the ith dictionary word, and assumes that every position has the same conditional distribution:

$$P(Y, W_1, \ldots, W_n) = P(Y) \prod_{i} P(W_i \mid Y).$$

These TIED distributions share one table $P(W \mid Y)$ across positions. The model ignores word order, and the tying lets every occurrence of a word inform the same parameters.

Products of many small probabilities underflow, so implementations sum logarithms. In one trained spam model, $P(\text{spam}) = 1/3$, and the words of "Gary would you like to lose weight while you sleep" have the probabilities below. The running totals of natural logarithms end at -76.0 for spam and -80.5 for ham (computed from unrounded probabilities), so

$$P(\text{spam} \mid w) = \frac{1}{1 + e^{-80.5+76.0}} \approx 0.989.$$

Word	$P(w \mid \text{spam})$	$P(w \mid \text{ham})$	Total spam	Total ham
(prior)	0.33333	0.66666	−1.1	−0.4
Gary	0.00002	0.00021	−11.8	−8.9
would	0.00069	0.00084	−19.1	−16.0
you	0.00881	0.00304	−23.8	−21.8
like	0.00086	0.00083	−30.9	−28.9
to	0.01517	0.01339	−35.1	−33.2
lose	0.00008	0.00002	−44.5	−44.0
weight	0.00016	0.00002	−53.3	−55.0
while	0.00027	0.00027	−61.5	−63.2
you	0.00881	0.00304	−66.2	−69.0
sleep	0.00006	0.00001	−76.0	−80.5

Words such as *lose*, *weight*, and *sleep* are much more likely under spam and overturn the prior, which favors ham.

19.4 Maximum likelihood estimation

The probability tables are the PARAMETERS θ of the model. They could be elicited from a human, but are usually estimated from data. Suppose observations $x_1, \ldots, x_N$ come from a distribution P_θ. MAXIMUM LIKELIHOOD ESTIMATION (MLE) assumes that the samples are INDEPENDENT AND IDENTICALLY DISTRIBUTED (i.i.d.) given θ, and that all values of θ are equally likely beforehand (a uniform prior). It maximizes the LIKELIHOOD

$$\mathcal{L}(\theta) = P_\theta(x_1, \ldots, x_N) = \prod_{i=1}^{N} P_\theta(x_i).$$

Since log is strictly increasing, the same θ maximizes the LOG-LIKELIHOOD $\log \mathcal{L}(\theta)$, which turns products into sums and is easier to differentiate.

Example: *Red and blue beans.* Let θ be the fraction of red beans in a jar, so each draw with replacement is red with probability θ. Drawing red, red, blue gives $\mathcal{L}(\theta) = \theta^2(1-\theta)$. Setting the derivative of the log-likelihood to zero,

$$\frac{d}{d\theta}\big(2\ln\theta + \ln(1-\theta)\big) = \frac{2}{\theta} - \frac{1}{1-\theta} = 0 \quad \Longrightarrow \quad \theta = \frac{2}{3}.$$

The other root of $\mathcal{L}'(\theta) = \theta(2-3\theta)$, namely $\theta = 0$, is a minimum: $\mathcal{L}(0) = 0 < \mathcal{L}(2/3) = 4/27$. With r red and b blue draws the same steps give $\theta = r/(r+b)$, the count estimate.

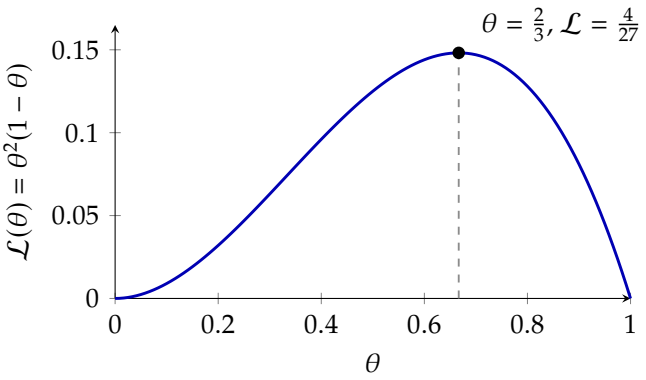


Figure 38: The likelihood of red, red, blue, maximized at the relative frequency of red.

In general, relative frequencies are maximum likelihood estimates:

$$\theta_{\text{ML}} = \arg\max_{\theta} \prod_i P_\theta(x_i), \qquad P_{\text{ML}}(x) = \frac{\text{count}(x)}{\text{total samples}}.$$

Another option is the most likely parameter given the data, the MAXIMUM A POSTERIORI (MAP) estimate

$$\theta_{\text{MAP}} = \arg\max_{\theta} P(\theta \mid X) = \arg\max_{\theta} P(X \mid \theta) P(\theta),$$

since $P(X)$ does not depend on θ. With a uniform prior it equals the MLE.

For Naive Bayes, consider $\theta = P(F_i = 1 \mid Y = \text{ham})$. Let the training set have N_h ham emails and let $f_i^{(j)} \in \{0, 1\}$ record whether word i appears in the jth one. Each factor is $\theta^{f_i^{(j)}}(1-\theta)^{1-f_i^{(j)}}$, so

$$\log \mathcal{L}(\theta) = \log\theta \sum_{j=1}^{N_h} f_i^{(j)} + \log(1-\theta) \sum_{j=1}^{N_h} \left(1 - f_i^{(j)}\right).$$

Setting the derivative to zero gives $(1-\theta)\sum_j f_i^{(j)} = \theta \sum_j (1 - f_i^{(j)})$, hence

$$\theta = \frac{1}{N_h} \sum_{j=1}^{N_h} f_i^{(j)},$$

the fraction of ham emails containing word i. Within each class, the maximum likelihood estimate of every table entry is a count divided by the number of samples in that class.

19.5 Overfitting and generalization

EMPIRICAL RISK MINIMIZATION is the basic principle of machine learning: we want the model that does best on the true test distribution, but since that distribution is unknown, we pick the model that does best on the training set, phrased as an optimization problem. The main worry is OVERFITTING:

fitting the training data very closely without generalizing. It is reduced by more training data, which lowers sampling variance and makes training more like test, and by limiting the complexity of the hypotheses, through regularization or small hypothesis spaces. A degree-15 polynomial fit to a few noisy points is the classic picture.

Relative-frequency estimates overfit. If a pixel was never on for any training 3, then $P(\text{on} \mid C = 3) = 0$, and a single such pixel vetoes class 3 however well the rest fits.

Example: For a test image, suppose $P(C = 2) = P(C = 3) = 0.1$ and four pixels have

	on	on	off	on
$C = 2$	0.8	0.1	0.1	0.01
$C = 3$	0.8	0.9	0.7	0.0

The joint for class 2 is $0.1 \cdot 0.8 \cdot 0.1 \cdot 0.1 \cdot 0.01 = 8 \times 10^{-6}$, while class 3 gets exactly 0. Class 2 wins, although the middle pixels favor 3.

Similarly, posteriors depend on odds ratios $P(w \mid \text{ham})/P(w \mid \text{spam})$. Unsmoothed estimates give infinite ratios to rare words seen in only one class: *south-west*, *nation*, *morally*, *seriously* for ham; *screens*, *minute*, *guaranteed*, *$205.00*, *delivery*, *signature* for spam. It is unlikely that every occurrence of *minute* is spam, and words absent from the training set would get probability zero in every class. In the extreme, using the whole email (a document ID) as the only feature fits deterministic training labels perfectly and does not generalize at all. The bag-of-words assumption gives some generalization, but not enough; we must SMOOTH or regularize the estimates.

19.6 Laplace smoothing

LAPLACE SMOOTHING pretends that every outcome was seen k more times than it was. For a variable X with $|X|$ values and N samples,

$$P_{\text{LAP},k}(x) = \frac{\text{count}(x) + k}{N + k|X|}, \qquad P_{\text{LAP},k}(x \mid y) = \frac{\text{count}(x, y) + k}{\text{count}(y) + k|X|},$$

where each condition y is smoothed independently. The case $k = 0$ is the MLE, and as $k \to \infty$ the estimate tends to the uniform $1/|X|$; k is the strength of the prior. The $k = 1$ estimate can be derived from a Dirichlet prior. For the draws red, red, blue,

$$P_{\text{ML}} = P_{\text{LAP},0} = \left(\frac{2}{3}, \frac{1}{3}\right), \qquad P_{\text{LAP},1} = \left(\frac{3}{5}, \frac{2}{5}\right), \qquad P_{\text{LAP},100} = \left(\frac{102}{203}, \frac{101}{203}\right).$$

For real classification smoothing is critical. After smoothing, the largest odds ratios become finite and sensible: *helvetica* (11.4), *seems*, *group*, *ago*, *areas* favor ham, and *verdana* (28.8), *Credit*, *ORDER*, `<FONT>`, *money* favor spam.

The model now has two kinds of unknowns: parameters, the probabilities $P(X \mid Y)$ and $P(Y)$, and hyperparameters such as the amount or type of smoothing. Parameters are learned from training data. Hyperparameters are tuned on separate held-out data, since training accuracy alone would always favor fitting the training set more closely. For each hyperparameter value, train and evaluate on the held-out set; choose the best value and do one final test on the test set. As k grows, training accuracy falls, while held-out and test accuracy first rise and then fall.

19.7 Features and baselines

Errors usually call for better features rather than only more words. Useful additions include whether the user has emailed the sender before, whether many other people received the same email, whether the sending information is consistent, whether the email is in capitals, whether inline URLs point where they claim, and whether the email uses the recipient's name. Each can be a new variable in the Naive Bayes model, although classifiers that accept arbitrary features more easily come next.

A BASELINE is a simple procedure that shows how hard a task is and what accuracy counts as good. A weak baseline is the MOST-FREQUENT-LABEL CLASSIFIER, which predicts the most common training label for every input. On skewed data it can be very accurate: if 66% of emails are ham, labeling everything ham scores 66%, so a 70% classifier is not impressive. Research usually uses previous work as a strong baseline. Classifier confidences, such as Naive Bayes posteriors, are useful when available.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 19 (Spring 2024); Nikhil Sharma, Note 19.

Lecture 20: Machine Learning II

Naive Bayes classifies by estimating $P(y \mid f_1, \ldots, f_n)$ through a model of how features depend on labels. This lecture classifies directly with a weighted combination of features, first without probabilities and then with them.

20.1 Linear classifiers

A feature function maps an input x to a vector $f(x)$, for example

$$\texttt{\#free} = 2, \quad \texttt{YOUR_NAME} = 0, \quad \texttt{MISSPELLED} = 2, \quad \texttt{FROM_FRIEND} = 0, \ \ldots$$

for an email, or pixel values and `NUM_LOOPS` for a digit. Loosely inspired by neurons, a linear classifier gives each feature a weight w_i and computes the activation

$$\text{activation}_w(x) = \sum_i w_i f_i(x) = w \cdot f(x).$$

A binary classifier outputs +1 if the activation is positive and −1 if it is negative; points with activation exactly 0 are assigned +1 below.

Example: With weights `#free` = 4, `YOUR_NAME` = −1, `MISSPELLED` = 1, `FROM_FRIEND` = −3, an email with features $(2, 0, 2, 0)$ scores $8 + 2 = 10$ and is spam (+1), while one with $(0, 1, 1, 1)$ scores $-1 + 1 - 3 = -3$ and is ham. Mentions of *free* and misspellings push toward spam; a known sender and the recipient's name push toward ham.

Geometrically, $w \cdot f = \|w\| \, \|f\| \cos\theta$, where θ is the angle between the vectors. Similar vectors have a small angle and a large dot product. Since norms are nonnegative, the sign is that of $\cos\theta$: positive for $\theta < 90°$, zero at $90°$, and negative beyond. The classifier therefore labels x positive when $f(x)$ points roughly in the direction of w.

In feature space, examples are points and the weight vector defines a decision boundary $w \cdot f = 0$, the set of vectors orthogonal to w. It is a line in two dimensions and a hyperplane in general, and it splits the space into a +1 side and a −1 side. With weights `free` = 4 and `money` = 2, the classifier predicts spam when $4\,\texttt{free} + 2\,\texttt{money} > 0$ and the boundary is $4\,\texttt{free} + 2\,\texttt{money} = 0$. Nonlinear classifiers have curved boundaries, but the boundary is always the surface separating the classes.

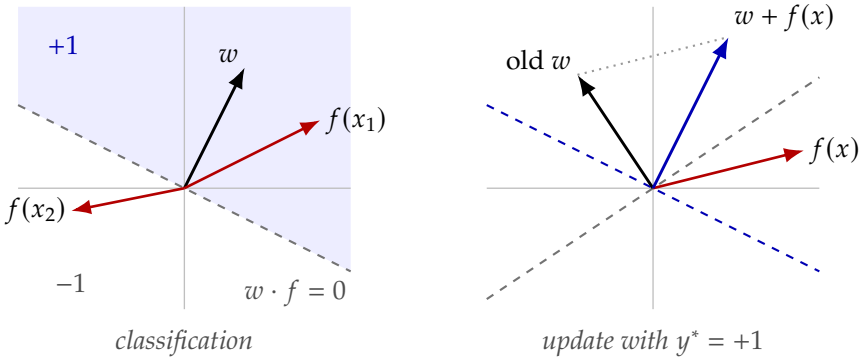


Figure 39: Left: $f(x_1)$ makes an acute angle with w and is classified $+1$; $f(x_2)$ is classified -1. Right: a positive example on the wrong side of the old boundary (gray) is added to w; the new boundary (blue) classifies it correctly.

20.2 The binary perceptron

The PERCEPTRON learns w from labeled examples $(f(x), y^*)$ with $y^* \in \{-1, +1\}$. Its goal is a boundary that separates the training data perfectly.

Algorithm 20.1: Binary perceptron

Input: Training examples $(f(x), y^*)$ with $y^* \in \{-1, +1\}$.
Output: Weights w that classify every training example correctly.

$w \leftarrow 0$
repeat
  for each training example $(f(x), y^*)$ **do**
    $y \leftarrow +1$ if $w \cdot f(x) \geq 0$, else $y \leftarrow -1$
    if $y \neq y^*$ **then**
      $w \leftarrow w + y^* f(x)$
until a full pass makes no update
return w

The update moves the activation of the misclassified example in the right direction. If $y^* = -1$ but $w \cdot f > 0$, the new weights $w' = w - f$ give

$$w' \cdot f = w \cdot f - \|f\|^2 < w \cdot f,$$

and if $y^* = +1$ but $w \cdot f < 0$, then $w' = w + f$ gives $w' \cdot f = w \cdot f + \|f\|^2$. Adding the feature vector also changes each weight in proportion to how much its feature contributed. For $w = (2, 2, 2)$, $f(x) = (4, 0, 1)$, and $y^* = -1$, the activation is $8 + 0 + 2 = 10$. The update gives $w = (-2, 2, 1)$: the first weight changes most, the second not at all. The new activation is $10 - \|f\|^2 = -7$, so this example is now correct. In general one update need not fix the mistake; that depends on $\|w\|$ and on how far $f(x)$ lies across the boundary. In figure 39, the update rotates w toward $f(x)$.

20.3 Bias

Every boundary $w \cdot f = 0$ passes through the origin, but a separating boundary need not. Adding a BIAS feature that is always 1, with its own weight w_0, gives boundaries $w \cdot f(x) + b = 0$ with $b = w_0$. Geometrically, the activation becomes an affine rather than a linear function of the features.

Example: Run one pass with bias feature first, starting from $w = (-1, 0, 0)$, on the points $(f_1, f_2; y^*)$: $(1, 1; -)$, $(3, 2; +)$, $(2, 4; +)$, $(3, 4; +)$, $(2, 3; -)$.

Step	Weights	Score	Correct?	Update
1	$(-1, 0, 0)$	$-1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = -1$	yes	none
2	$(-1, 0, 0)$	$-1 \cdot 1 + 0 \cdot 3 + 0 \cdot 2 = -1$	no	$+(1, 3, 2)$
3	$(0, 3, 2)$	$0 \cdot 1 + 3 \cdot 2 + 2 \cdot 4 = 14$	yes	none
4	$(0, 3, 2)$	$0 \cdot 1 + 3 \cdot 3 + 2 \cdot 4 = 17$	yes	none
5	$(0, 3, 2)$	$0 \cdot 1 + 3 \cdot 2 + 2 \cdot 3 = 12$	no	$-(1, 2, 3)$

The pass ends with $w = (-1, 1, -1)$. Further passes continue until one makes no mistakes.

20.4 The multiclass perceptron

With several classes, keep a weight vector w_y for each class y, score each class by $w_y \cdot f(x)$, and predict

$$y = \arg\max_y \, w_y \cdot f(x).$$

The feature space is divided into regions where each $w_y \cdot f$ is largest. Stacking the w_y as rows of a matrix W computes all scores as one product $Wf(x)$. The binary perceptron is the special case where the negative class has weight vector 0.

Learning starts with all weights 0 and processes examples one at a time. A correct prediction changes nothing. A wrong prediction $y \neq y^*$ lowers the score of the wrong answer and raises that of the right one:

$$w_y \leftarrow w_y - f(x), \qquad w_{y^*} \leftarrow w_{y^*} + f(x).$$

Other weight vectors are unchanged. For a bias, append a constant feature 1 to every $f(x)$ and an entry to every w_y.

Example: Let $f(x) = (-2, 3, 1)$ and $w_0 = (-2, 2, 1)$, $w_1 = (0, 3, 4)$, $w_2 = (1, 4, -2)$. The scores are $s_0 = 11$, $s_1 = 13$, $s_2 = 8$, so the prediction is class 1. If the true class is 2, the update gives $w_1 = (0, 3, 4) - (-2, 3, 1) = (2, 0, 3)$ and $w_2 = (1, 4, -2) + (-2, 3, 1) = (-1, 7, -1)$. The new scores are 11, -1, and 22.

20.5 Properties and problems of the perceptron

Training data is SEPARABLE if some weights classify it perfectly. In the binary case, the perceptron CONVERGES on separable data, and the number of mistakes is bounded in terms of the MARGIN δ, the degree of separability: mistakes $< k/\delta^2$ for a constant k determined by the data. Its weaknesses are:

- *Noise.* On non-separable data the weights may thrash. Averaging the weight vectors over time, the AVERAGED PERCEPTRON, helps.
- *Mediocre generalization.* It stops at any separating boundary, often one that barely separates the data.
- *Overtraining.* With more iterations, test and held-out accuracy usually rise and then fall. Overtraining is a kind of overfitting.

20.6 Logistic regression

On non-separable data even the best linear boundary makes mistakes, and a deterministic rule says nothing about how confident it is. Equivalently, the perceptron reports probability $H(z)$ that the class is +1, where $z = w \cdot f(x)$ and H is the step function: 1 if $z > 0$ and 0 otherwise. Instead, a very positive z should give probability near 1, a very negative z near 0, and z near 0 should give about 1/2. The SIGMOID

$$\phi(z) = \frac{1}{1 + e^{-z}}$$

does this, with $\phi(0) = 1/2$ and $1 - \phi(z) = \phi(-z)$. Parallel lines $w \cdot f(x) = c$ then become contours of equal probability, such as 0.9 | 0.1, 0.7 | 0.3, and 0.5 | 0.5 for the two classes.

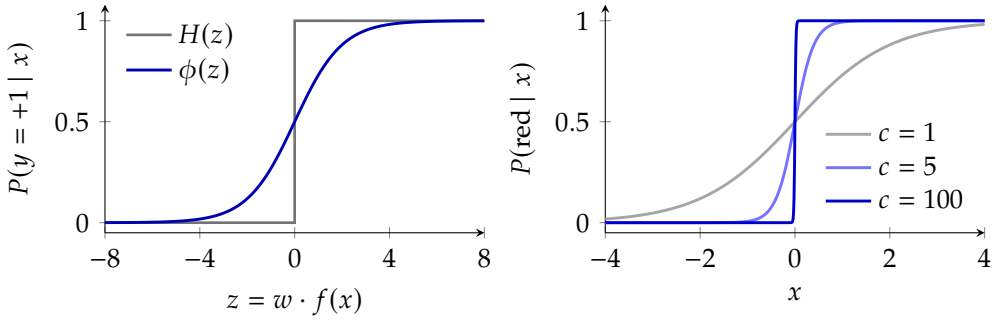


Figure 40: Left: the perceptron's step function and the sigmoid. Right: two-class softmax $P(\text{red} \mid x)$ with $w_{\text{red}} - w_{\text{blue}} = 1$ and weights scaled by c; as c grows it approaches the hard maximum.

In one dimension, $P(\text{red} \mid x) = 1/(1 + e^{-wx})$ with $w > 0$ is almost 0 for very negative x (definitely blue), almost 1 for very positive x (definitely red), and uncertain near $x = 0$. To choose w, apply

maximum likelihood. Assuming independent training points,

$$\max_w ll(w) = \max_w \sum_i \log P\big(y^{(i)} \mid x^{(i)}; w\big),$$

with

$$P\big(y^{(i)} = +1 \mid x^{(i)}; w\big) = \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}, \qquad P\big(y^{(i)} = -1 \mid x^{(i)}; w\big) = 1 - \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}.$$

This is LOGISTIC REGRESSION. On separable data many boundaries are deterministic solutions, but the likelihood has a clear preference: it favors a boundary whose probability contours keep the training points far on their correct sides.

20.7 Multiclass logistic regression

The multiclass perceptron scores $z_y = w_y \cdot f(x)$. The SOFTMAX turns scores into probabilities:

$$z_1, z_2, z_3 \mapsto \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}.$$

MULTICLASS LOGISTIC REGRESSION maximizes the same log-likelihood $ll(w)$ with

$$P\big(y^{(i)} \mid x^{(i)}; w\big) = \frac{e^{w_{y^{(i)}} \cdot f(x^{(i)})}}{\sum_y e^{w_y \cdot f(x^{(i)})}}.$$

Scaling all weights by c, equivalently using base e^c, sharpens the softmax: for large c it looks like $\max_y w_y \cdot f(x)$, the perceptron's rule (figure 40, right).

Binary logistic regression is the two-class case. Fix the weight vector of the negative class at 0 and let w be the positive class's weight. The negative class always has activation 0, so the perceptron predicts +1 exactly when $w \cdot f > 0$, and the softmax becomes

$$P(+1 \mid x) = \frac{e^{w \cdot f(x)}}{e^{w \cdot f(x)} + e^0} = \frac{1}{1 + e^{-w \cdot f(x)}},$$

the sigmoid. Finding the maximizing w is an optimization problem, the subject of the next lecture.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 20 (Spring 2024); Nikhil Sharma, Note 20.

Lecture 21: Machine Learning III

A feature function f maps a raw input x to a vector $f(x)$. A classification problem predicts a discrete label from it, such as spam or the digit in an image. A regression problem predicts a continuous output, such as the market rent of an office from its floor area, distance to the nearest train station, number of offices, and number of views. Features may be continuous or categorical.

21.1 Linear regression

Linear regression predicts with a weighted sum of the features,

$$h_w(x) = \sum_i w_i f_i(x) = w \cdot f(x).$$

An intercept w_0 is either added to this sum or obtained by including a constant feature equal to 1; the two are equivalent. For the rent example, weights (5000, 0.8, 100, 300, 1000) on the features (constant, square feet, distance to the station, offices, views) give the prediction as one dot product per office. The positive weight on distance is the least plausible, since rent should not rise with distance from the station.

Recall that $a \cdot b = \sum_i a_i b_i = |a|\,|b| \cos\theta$, where θ is the angle between the vectors. It is positive for $\theta < 90°$, zero at $90°$, and negative beyond. Thus $w \cdot f(x)$ measures how far $f(x)$ extends in the direction of w, scaled by $|w|$. With two features, the prediction is a plane over the feature space, and w points in the direction in which this plane rises most steeply.

Stack N training points as the rows of a design matrix. With x_j^i the jth feature of point i and a leading constant feature,

$$\mathbf{y} = \begin{bmatrix} y^1 \\ \vdots \\ y^N \end{bmatrix}, \qquad X = \begin{bmatrix} 1 & x_1^1 & \cdots & x_n^1 \\ \vdots & \vdots & & \vdots \\ 1 & x_1^N & \cdots & x_n^N \end{bmatrix}, \qquad w = \begin{bmatrix} w_0 \\ \vdots \\ w_n \end{bmatrix}.$$

Row i of Xw is the prediction $w_0 + w_1 x_1^i + \cdots + w_n x_n^i$, and we want Xw to look like $\mathbf{y}$. The least-squares loss is

$$\text{Loss}(w) = \frac{1}{2} \sum_{i=1}^{N} \left(y^i - h_w(x^i)\right)^2 = \frac{1}{2} \|\mathbf{y} - Xw\|_2^2,$$

where $\|v\|_2^2 = \sum_i v_i^2 = v^\mathsf{T} v$. The factor $\frac{1}{2}$ only simplifies the derivative.

Proposition 21.1 [NORMAL EQUATIONS]. *If $X^\mathsf{T}X$ is invertible, the least-squares loss has the unique minimizer*

$$\hat{w} = (X^\mathsf{T}X)^{-1}X^\mathsf{T}\mathbf{y}.$$

Proof. Expanding the square, and using that the scalar $\mathbf{y}^\mathsf{T}Xw$ equals its transpose $w^\mathsf{T}X^\mathsf{T}\mathbf{y}$,

$$\nabla_w \tfrac{1}{2}(\mathbf{y} - Xw)^\mathsf{T}(\mathbf{y} - Xw) = \nabla_w \tfrac{1}{2}\big(\mathbf{y}^\mathsf{T}\mathbf{y} - 2w^\mathsf{T}X^\mathsf{T}\mathbf{y} + w^\mathsf{T}X^\mathsf{T}Xw\big) = -X^\mathsf{T}\mathbf{y} + X^\mathsf{T}Xw.$$

The loss is a convex quadratic, so its minimizers are exactly the zeros of this gradient. Setting it to zero gives $X^\mathsf{T}X\hat{w} = X^\mathsf{T}\mathbf{y}$. □

A new point is then predicted as $h_{\hat{w}}(x) = \hat{w}^\mathsf{T}x$, with x including the constant feature.

21.2 Logistic regression

Logistic regression replaces the perceptron's step function by the sigmoid $\phi(z) = 1/(1 + e^{-z})$, so that $P(y = +1 \mid x; w) = \phi(w \cdot f(x))$. Since $1 - \phi(z) = \phi(-z)$, both labels $y \in \{+1, -1\}$ read $P(y \mid x; w) = \phi(y\, w \cdot f(x))$. Despite the name, it is a classification method.

Example: For $w = (-3, 4, 2)$ and $f(x) = (1, 2, 0)$, the activation is $w \cdot f(x) = -3 + 8 + 0 = 5$. The deterministic rule predicts the positive class. The probabilistic rule gives $\phi(5) = 1/(1 + e^{-5}) \approx 0.9933$ for the positive class and 0.0067 for the negative class.

The sigmoid arises from accumulating evidence. Suppose a coin is either A, with $P(\text{heads}) = 2/3$, or B, with $P(\text{heads}) = 1/3$, equally likely a priori. Each head multiplies the likelihood ratio of A to B by 2, and each tail divides it by 2. After n more heads than tails, A is 2^n times as likely as B, so

$$P(A) = \frac{2^n}{1 + 2^n} = \frac{1}{1 + 2^{-n}}.$$

Replacing base 2 by e gives $\phi(n)$: the activation z plays the role of net evidence for the positive class.

Weights are chosen by MAXIMUM LIKELIHOOD: maximize the probability of the observed labels. For independent training points $(x^{(i)}, y^{(i)})$,

$$\max_w \; ll(w) = \max_w \sum_i \log P\big(y^{(i)} \mid x^{(i)}; w\big).$$

The logarithm turns the product of probabilities into a sum and does not change the maximizer. Equivalently, the loss is $-ll(w)$.

Example: For training points $(2, 1)$ and $(0, -2)$ in class $+1$ and $(-1, -1)$ in class -1, with $f(x) = x$ and $w = (w_1, w_2)$, the objective is

$$\arg\max_w \Big[\log \phi(2w_1 + w_2) + \log \phi(-2w_2) + \log\big(1 - \phi(-w_1 - w_2)\big)\Big].$$

There is no closed-form maximizer, so the weights are found by gradient ascent (section 21.4). Using $\phi'(z) = \phi(z)\big(1 - \phi(z)\big)$, the gradient of one term is

$$\nabla_w \log \phi\big(y\, w \cdot f(x)\big) = \big(1 - \phi(y\, w \cdot f(x))\big)\, y\, f(x).$$

A point that is already classified confidently contributes little; a misclassified point pulls w toward $y\, f(x)$, much like a perceptron update.

One can also train on the squared error $\frac{1}{2}(y - h_w(x))^2$, with $h_w(x) = \phi(w \cdot f(x))$ and labels $y \in \{0, 1\}$. By the chain rule, its partial derivative is $-\big(y - h_w(x)\big) h_w(x) \big(1 - h_w(x)\big) f_i(x)$, again minimized by gradient descent.

21.3 Multiclass logistic regression

The softmax $P(y \mid x; w) = e^{w_y \cdot f(x)} / \sum_{y'} e^{w_{y'} \cdot f(x)}$ turns the multiclass perceptron's activations into probabilities, and maximizing $\sum_i \log P(y^{(i)} \mid x^{(i)}; w)$ is MULTICLASS LOGISTIC REGRESSION.

Example: Take $w_1 = (-3, 4, 2)$, $w_2 = (2, 2, 7)$, $w_3 = (0, -1, 0)$, and $f(x) = (1, 2, 0)$. The activations are 5, 6, and -2, so the deterministic rule predicts class 2. The softmax probabilities are

$$\frac{e^5}{e^5 + e^6 + e^{-2}} \approx 0.2689, \qquad \frac{e^6}{e^5 + e^6 + e^{-2}} \approx 0.7309, \qquad \frac{e^{-2}}{e^5 + e^6 + e^{-2}} \approx 0.0002.$$

With points $(2, 1)$ red, $(0, -2)$ green, and $(-1, -1)$ blue, the objective is the sum of three log-softmax terms, for instance $\log\big(e^{2w_{\mathrm{r},1} + w_{\mathrm{r},2}} / \sum_c e^{2w_{c,1} + w_{c,2}}\big)$ for the red point.

To write the gradient, let $t_{i,k} = 1$ if $y^{(i)} = k$ and 0 otherwise. Then

$$\log \ell(w_1, \ldots, w_K) = \sum_{i=1}^{n} \sum_{k=1}^{K} t_{i,k} \log \frac{e^{w_k \cdot f(x^{(i)})}}{\sum_\ell e^{w_\ell \cdot f(x^{(i)})}},$$

and, because $\sum_k t_{i,k} = 1$,

$$\nabla_{w_j} \log \ell(w) = \sum_{i=1}^{n} \Big(t_{i,j} - P\big(j \mid x^{(i)}; w\big)\Big) f(x^{(i)}).$$

Each class weight moves toward points of that class and away from points to which it assigns too much probability.

21.4 Gradient ascent

Hill climbing for these objectives is awkward: the space is continuous, so every point has infinitely many neighbors. In one dimension, we could compare $g(w_0 + h)$ and $g(w_0 - h)$ and step toward

the larger, or use the derivative

$$\frac{\partial g(w_0)}{\partial w} = \lim_{h \to 0} \frac{g(w_0 + h) - g(w_0 - h)}{2h},$$

whose sign gives the uphill direction. In n dimensions, the GRADIENT collects the partial derivatives, $\nabla g = (\partial g/\partial w_1, \ldots, \partial g/\partial w_n)^\mathsf{T}$. For example, $g(x) = x^2 + 3$ has $g'(5) = 10$, and $g(x, y) = x^2 y$ has $\nabla g = (2xy,\ x^2)$, equal to $(0.5,\ 0.25)$ at $(0.5, 0.5)$.

Proposition 21.2 [GRADIENT IS THE STEEPEST DIRECTION]. *To first order, among steps Δ with $\|\Delta\| \le \varepsilon$, the step $\Delta = \varepsilon\, \nabla g / \|\nabla g\|$ increases g the most.*

Proof. The first-order Taylor expansion is $g(w + \Delta) \approx g(w) + \nabla g(w)^\mathsf{T} \Delta$. By Cauchy–Schwarz, $\Delta^\mathsf{T} a \le \|\Delta\|\, \|a\| \le \varepsilon \|a\|$, with equality exactly when $\Delta = \varepsilon a / \|a\|$. Take $a = \nabla g(w)$. □

GRADIENT ASCENT therefore starts somewhere and repeats $w \leftarrow w + \alpha \nabla g(w)$, so that coordinates with steeper slopes take larger steps. GRADIENT DESCENT, $w \leftarrow w - \alpha \nabla g(w)$, minimizes a loss instead. The LEARNING RATE α must be chosen carefully: large enough to make progress, small enough not to diverge. In practice one tries several values; a crude rule of thumb is that an update should change w by about 0.1 to 1%. A common schedule starts large and decreases α over the iterations (LEARNING RATE DECAY).

For least squares, gradient descent repeats $w \leftarrow w - \alpha\big(X^\mathsf{T} X w - X^\mathsf{T} \mathbf{y}\big)$; with a small enough step it converges to the normal-equation solution. Logistic regression has no closed form and ascends $g(w) = ll(w)$.

The log-likelihood is a sum over training points, so its gradient is the sum of per-point gradients. Computing the full sum on every step is BATCH gradient ascent. When the dataset is large, cheaper estimates are used: STOCHASTIC GRADIENT ASCENT updates after each randomly chosen point, and MINI-BATCH GRADIENT ASCENT uses a random subset J, whose per-point gradients can be computed in parallel; $|J| = 1$ and the full set recover the other two. A single point gives a noisy gradient, which can make convergence harder; mini-batches trade off noise against cost.

Algorithm 21.1: Mini-batch gradient ascent on the log-likelihood

Input: Training data $\{(x^{(i)}, y^{(i)})\}$, learning rate α.
Output: Weights w.

Initialize w
for *iter* $= 1, 2, \ldots$ **do**
  choose a random subset J of training examples
  $w \leftarrow w + \alpha \sum_{j \in J} \nabla_w \log P\big(y^{(j)} \mid x^{(j)}; w\big)$

Source. Michael K. Cohen, with optimization slides by Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 21 (Spring 2024); Nikhil Sharma, Note 21.

Lecture 22: Machine Learning: Neural Networks

Logistic regression learns a linear boundary over fixed features. Neural networks also learn the features, and are trained by the same log-likelihood objective and gradient methods.

22.1 Learning the features

A linear classifier fails when the best boundary is nonlinear. For one-dimensional data with one class near the origin and the other at large $|x|$, no single threshold separates them. Adding the feature x^2 maps the points into the plane (x, x^2), where a line separates them. Choosing such features by hand requires domain-specific expertise and effort, which is especially tedious for high-dimensional inputs such as images, and does not transfer across tasks. Deep learning instead learns the featurization along with the classifier.

22.2 Two-layer networks

A perceptron computes $y = 1$ if $w_1x_1 + w_2x_2 + w_3x_3 > 0$ and $y = 0$ otherwise. Replacing the step by the sigmoid ϕ gives the smooth unit $y = \phi(w_1x_1 + w_2x_2 + w_3x_3)$. A two-layer network feeds the outputs of several such units, the hidden units, into another unit. With three inputs and two hidden units,

$$h_1 = \phi(w_{11}x_1 + w_{21}x_2 + w_{31}x_3), \qquad h_2 = \phi(w_{12}x_1 + w_{22}x_2 + w_{32}x_3),$$

and the output is

$$y = \phi(w_1h_1 + w_2h_2) = \phi\big(w_1\phi(w_{11}x_1 + w_{21}x_2 + w_{31}x_3) + w_2\phi(w_{12}x_1 + w_{22}x_2 + w_{32}x_3)\big).$$

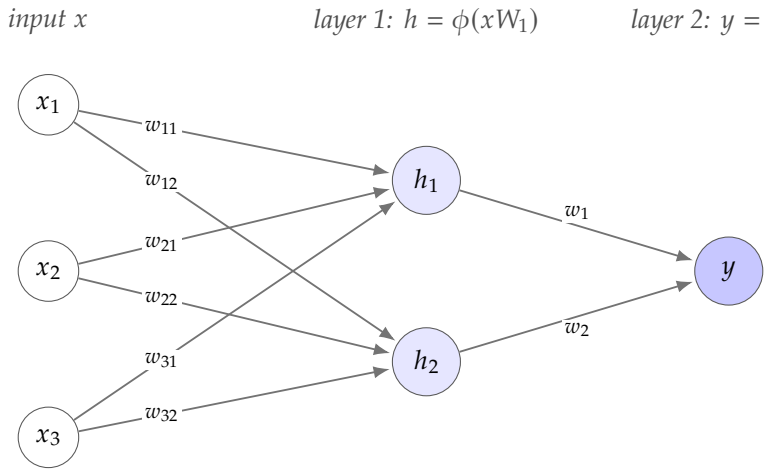


Figure 41: A two-layer network with three inputs and two hidden units. Weight w_{jk} connects input j to hidden unit k.

In matrix form, with row vectors and ϕ applied entrywise,

$$\phi\left(\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix}\begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix}\right) = \begin{bmatrix} h_1 & h_2 \end{bmatrix}, \qquad \phi\left(\begin{bmatrix} h_1 & h_2 \end{bmatrix}\begin{bmatrix} w_1 \\ w_2 \end{bmatrix}\right) = y.$$

Compactly, $h = \phi(xW_1)$ and $y = \phi(hW_2)$. Here x is 1×3, W_1 is 3×2, h is 1×2, W_2 is 2×1, and y is 1×1.

Each dimension generalizes independently. With $\dim(x)$ input features, n hidden units, and $\dim(y)$ outputs, W_1 has shape $\dim(x) \times n$ and W_2 has shape $n \times \dim(y)$. Column k of a weight matrix holds the weights of one neuron, so the shape of a layer's matrix is fixed by the dimensions of its input and output.

22.3 Deep networks and activations

A MULTILAYER NETWORK chains layers: each layer multiplies its $1 \times \dim(\text{in})$ input by a $\dim(\text{in}) \times \dim(\text{out})$ weight matrix and applies a nonlinearity entrywise, giving one output per neuron. Its output is the next layer's input. A three-layer network computes

$$h_{\text{layer 1}} = \phi(xW_{\text{layer 1}}), \qquad h_{\text{layer 2}} = \phi(h_{\text{layer 1}}W_{\text{layer 2}}), \qquad y = \phi(h_{\text{layer 2}}W_{\text{layer 3}}),$$

with shapes $\dim(x) \times \dim(L_1)$, $\dim(L_1) \times \dim(L_2)$, and $\dim(L_2) \times \dim(y)$. In general, unit i of layer k computes

$$z_i^{(k)} = g\Big(\sum_j W_{i,j}^{(k-1,k)} z_j^{(k-1)}\Big),$$

where g is the ACTIVATION FUNCTION. The last layer sometimes omits the nonlinearity.

The nonlinearity is essential: a composition of linear maps is linear, so without it the whole network would still be a linear classifier. The perceptron's step function is a poor choice for training, since it is discontinuous and its derivative is zero wherever it exists, so the gradient gives no direction in which to improve. Common smooth choices are

$$\text{sigmoid } g(z) = \frac{1}{1+e^{-z}}, \qquad \tanh g(z) = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}, \qquad \text{ReLU } g(z) = \max(0, z),$$

with derivatives $g(1-g)$, $1-g^2$, and 1 for $z > 0$ and 0 otherwise. The choice is a design decision usually settled by experiment.

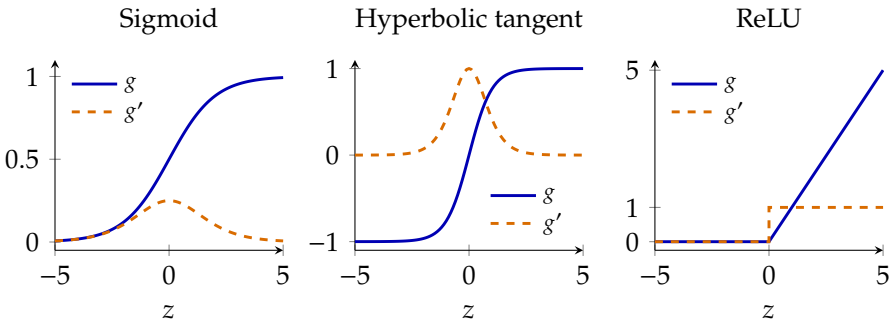


Figure 42: Three activation functions g (solid) and their derivatives g' (dashed).

Networks are evaluated on many inputs at once by BATCHING. Stack B inputs as the rows of a $B \times \dim(x)$ matrix X. Then $H = \phi(XW_1)$ has shape $B \times n$ and $Y = \phi(HW_2)$ has shape $B \times \dim(y)$. The architecture is unchanged: the weights and nonlinearity act on each row separately, and the matrix products parallelize efficiently on a GPU.

22.4 Training with the log loss

Training repeats three steps. Run the network on the training inputs with the current weights; compare its predictions with the true labels using a LOSS FUNCTION, where lower is better; and adjust the weights by a numerical method such as gradient descent, since the loss is a function of the weights. Examples are the ZERO-ONE LOSS, which counts misclassifications; the sum of squared errors, for regression; and the LOG LOSS. For binary labels $y_i \in \{0, 1\}$ and predicted probabilities p_i of the positive class,

$$\text{LogLoss} = -\sum_i \Big(y_i \log p_i + (1-y_i)\log(1-p_i) \Big).$$

A point with $y = 1$ contributes $-\log p$, which is large when the network confidently predicts the wrong class (p near 0) and near zero when p is near 1. A point with $y = 0$ contributes $-\log(1-p)$. The labels stay fixed; changing the weights changes the p_i. The log loss is the negative log-likelihood of the labels, so minimizing it is the maximum likelihood objective from logistic regression.

Example: With base-10 logarithms, three points with true labels $1, 0, 1$ and predictions $p = 0.1,\ 0,\ 0.8$

contribute $-\log 0.1 = 1$, $-\log 1 = 0$, and $-\log 0.8 \approx 0.1$, for a total loss of about 1.1. The first prediction is bad, the second perfect, and the third good but not perfect.

For several classes, the output layer produces one score per class and softmax converts the scores into probabilities, exactly as in multiclass logistic regression. The objective is unchanged,

$$\max_{\theta} \; ll(\theta) = \max_{\theta} \sum_{i} \log P\big(y^{(i)} \mid x^{(i)}; \theta\big),$$

but P now depends on the parameters θ, many weight arrays rather than a single vector, in a far more elaborate way. In effect, the last layer is still logistic regression, applied to features computed by the earlier layers. The same batch, stochastic, and mini-batch gradient methods apply, using the fact that the gradient of a sum is the sum of the gradients. A learning rate that is too high makes the iterates overshoot and oscillate across the valley instead of descending into it.

Accuracy, in contrast, is the fraction of correct predictions. For a binary classifier with labels $y_i \in \{-1, +1\}$,

$$l^{\text{acc}}(w) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\big[\operatorname{sgn}(w \cdot f(x_i)) = y_i\big].$$

It is piecewise constant in the weights and so cannot guide gradient methods directly.

22.5 Backpropagation

Gradients of these losses are computed by repeated application of the CHAIN RULE. If f depends on $x_1, \ldots, x_n$ and each x_j depends on $t_1, \ldots, t_m$, then

$$\frac{\partial f}{\partial t_i} = \sum_{j=1}^{n} \frac{\partial f}{\partial x_j} \frac{\partial x_j}{\partial t_i}.$$

BACKPROPAGATION organizes this computation on a COMPUTATION GRAPH, a dependency graph of operations and operands. The FORWARD PASS evaluates each node from its inputs. The BACKWARD PASS starts from the output node, with $\partial f / \partial f = 1$, and moves toward the inputs. Each node sums the gradients sent by its children, multiplies by the local derivative of its operation with respect to each input, and passes the products on to its parents. Gradients therefore add where paths meet. Forward values and downstream gradients are cached, so each local step uses stored values.

Example: For $f = (x + y)z$ at $x = 2$, $y = 3$, $z = 4$, write $g = x + y = 5$ and $f = gz = 20$. Then $\partial f / \partial g = z = 4$ and $\partial f / \partial z = g = 5$. Since $\partial g / \partial x = \partial g / \partial y = 1$, the chain rule gives $\partial f / \partial x = \partial f / \partial y = 4 \cdot 1 = 4$.

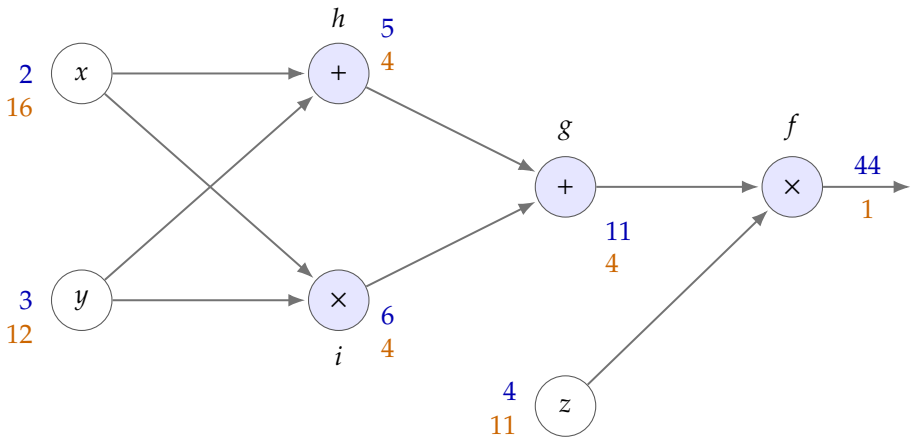


Figure 43: Backpropagation for $f = \big((x + y) + xy\big)z$ at $x = 2$, $y = 3$, $z = 4$. Blue numbers are forward values; orange numbers are the gradients $\partial f/\partial(\text{node})$.

In this graph, $h = x + y = 5$, $i = xy = 6$, $g = h + i = 11$, and $f = gz = 44$. Backward, $\partial f/\partial g = z = 4$ and $\partial f/\partial z = g = 11$; the sum node passes 4 to both h and i. The variable x feeds both h and i, so its two contributions add:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial h}\frac{\partial h}{\partial x} + \frac{\partial f}{\partial i}\frac{\partial i}{\partial x} = 4 \cdot 1 + 4 \cdot y = 16, \qquad \frac{\partial f}{\partial y} = 4 \cdot 1 + 4 \cdot x = 12.$$

Example: Let $g(w_1, w_2, w_3) = w_1^4 w_2 + 5w_3$ at $(2, 3, 2)$. Decompose it as $a = w_1^4 = 16$, $b = aw_2 = 48$, $c = 5w_3 = 10$, and $g = b+c = 58$. Then $\partial g/\partial b = \partial g/\partial c = 1$, so $\partial g/\partial a = w_2 = 3$ and $\partial g/\partial w_2 = a = 16$. Further, $\partial g/\partial w_1 = 3 \cdot 4w_1^3 = 96$ and $\partial g/\partial w_3 = 1 \cdot 5 = 5$.

Automatic differentiation software, such as PyTorch or TensorFlow, needs only the program computing the function. It records the forward computation and runs the backward pass, computing all derivatives with respect to all parameters at a cost comparable to the forward pass.

```python
import torch
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)
z = x * y + x**2
z.backward()
print(x.grad, y.grad)  # tensor(7.) tensor(2.)
```

Here $\partial z/\partial x = y + 2x = 7$ and $\partial z/\partial y = x = 2$.

22.6 Properties of neural networks

Theorem 22.1 [Universal function approximation]. *A two-layer neural network with a sufficient number of hidden units can approximate any continuous function to any desired accuracy.*

Precise versions are due to Cybenko (1989), Hornik (1991), and Leshno and Schocken (1991). For example, if the activation is continuous, bounded, and nonconstant, standard feedforward networks approximate any continuous function arbitrarily well in uniform distance on any compact subset of $\mathbb{R}^k$, given enough hidden units. If the activation is only bounded and nonconstant, they approximate any function in $L^p(\mu)$ for a finite measure μ. The theorem guarantees that suitable weights exist, not that gradient methods find them.

In practice, the hidden layers can be seen as learned features. Large networks risk overfitting or memorizing the training data, so training is stopped when held-out accuracy starts to fall (early stopping). Open problems include designing architectures for a given task, finding low-loss parameters when gradient descent is greedy and the loss has many local optima, and ensuring generalization and transfer to unseen data.

These methods underlie large language models and modern computer vision. On the ImageNet benchmark, the deep network AlexNet in 2012 achieved a much lower error rate than traditional computer-vision methods, whose progress had flattened, and deep-learning entries kept improving through 2014.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 22 (Spring 2024); Nikhil Sharma, Note 22.

Lecture 23: Reinforcement Learning: Bandits and Recommendation Systems

Planning assumes the effects of actions are known, and learning from data estimates how the environment works. Acting while learning adds a third problem: deciding when to collect information and when to use it.

Online decision-making faces a fundamental choice. Exploitation makes the best decision given current information; exploration gathers more information. The best long-term strategy may sacrifice reward in the short term. Examples are visiting a favorite restaurant or trying a new one, showing the most successful advertisement or a different one, drilling at the best known location or a new one, and playing the move believed best or an experimental one. Four principles recur below:

- *naive exploration*: add noise to a greedy policy;
- *optimistic initialization*: assume the best until proven otherwise;
- *optimism in the face of uncertainty*: prefer actions with uncertain values;
- *probability matching*: select actions according to the probability that they are best.

23.1 Multi-armed bandits and regret

Definition 23.1 [Multi-armed bandit]. A multi-armed bandit is a pair $\langle \mathcal{A}, \mathcal{R} \rangle$. The set $\mathcal{A}$ of m actions, or arms, is known, and $\mathcal{R}^a(r) = \mathbb{P}[r \mid a]$ is an unknown reward distribution for each arm. At each step t the agent selects $a_t \in \mathcal{A}$ and receives $r_t \sim \mathcal{R}^{a_t}$. The goal is to maximize the cumulative reward $\sum_{\tau=1}^{t} r_\tau$.

The action value is the mean reward $Q(a) = \mathbb{E}[r \mid a]$. The optimal value is $V^* = Q(a^*) = \max_{a \in \mathcal{A}} Q(a)$, and the gap of an action is $\Delta_a = V^* - Q(a)$. The one-step regret and the total regret are

$$l_t = \mathbb{E}\big[V^* - Q(a_t)\big], \qquad L_t = \mathbb{E}\left[\sum_{\tau=1}^{t} \big(V^* - Q(a_\tau)\big)\right].$$

The expectation is over the algorithm's possibly random choices. Regret depends only on the actions and their expected payoffs, not on the realized rewards. Maximizing expected cumulative reward is the same as minimizing total regret.

Let $N_t(a)$ be the number of times a has been selected in the first t steps. Grouping the terms of L_t by action gives

$$L_t = \sum_{a \in \mathcal{A}} \mathbb{E}[N_t(a)]\big(V^* - Q(a)\big) = \sum_{a \in \mathcal{A}} \mathbb{E}[N_t(a)]\,\Delta_a.$$

A good algorithm selects actions with large gaps rarely, but the gaps are unknown. An algorithm that explores forever, at a constant rate, has linear total regret; so does one that never explores. The question is whether sublinear regret is possible.

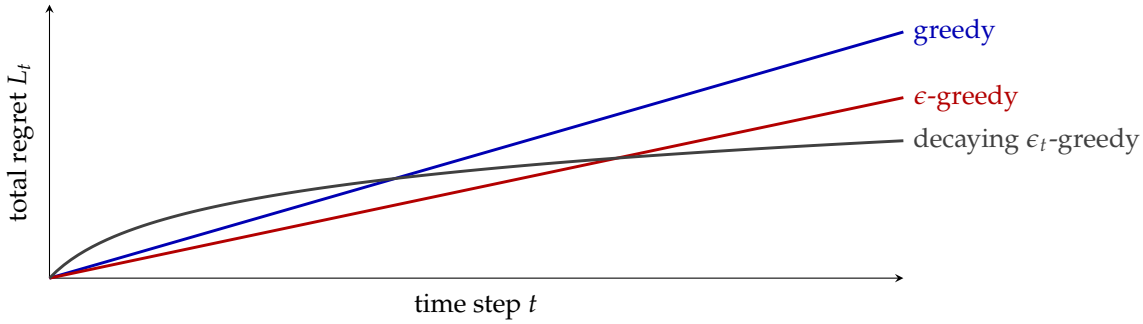


Figure 44: Schematic total regret. Greedy and constant-ϵ exploration grow linearly; a suitably decaying ϵ_t grows logarithmically.

23.2 Greedy and ϵ-greedy algorithms

The following algorithms estimate $Q(a)$ by the sample mean,

$$\hat{Q}_t(a) = \frac{1}{N_t(a)} \sum_{\tau=1}^{t} r_\tau \, \mathbf{1}(a_\tau = a),$$

a Monte Carlo estimate. One procedure covers all of them.

Algorithm 23.1: Sample-mean bandit agent with ϵ_t exploration

Input: Actions $\mathcal{A}$, exploration rates $\epsilon_1, \epsilon_2, \ldots$, initial counts $N(a)$ and reward totals $S(a)$.

for $t = 1, 2, \ldots$ **do**
- $\hat{Q}(a) \leftarrow S(a)/N(a)$ for each a
- $a_t \leftarrow \arg\max_a \hat{Q}(a)$
- With probability ϵ_t, replace a_t by a uniformly random action
- Take a_t and receive reward r_t
- $N(a_t) \leftarrow N(a_t) + 1$; $S(a_t) \leftarrow S(a_t) + r_t$

With zero initial counts, each action is first tried once so that every estimate is defined.

- Greedy: $\epsilon_t = 0$, so $a_t^* = \arg\max_a \hat{Q}_t(a)$. It can lock onto a suboptimal action forever. If the optimal arm's first rewards are unluckily low, its estimate can stay below another arm's, and it is never tried again. This happens with positive probability, so greedy has linear total regret.
- ϵ-greedy: a constant ϵ. With probability $1 - \epsilon$ it selects $\arg\max_a \hat{Q}_t(a)$; otherwise it selects a random action. The random choice alone gives

$$l_t \geq \frac{\epsilon}{|\mathcal{A}|} \sum_{a \in \mathcal{A}} \Delta_a,$$

a positive lower bound at every step, so total regret is again linear.

- Optimistic initialization: start each $\hat{Q}(a)$ at a high value by setting $N(a) > 0$, for instance five pretend pulls with total reward 10. The sample mean is then updated incrementally:

$$\hat{Q}_t(a_t) = \hat{Q}_{t-1}(a_t) + \frac{1}{N_t(a_t)}\big(r_t - \hat{Q}_{t-1}(a_t)\big).$$

  Disappointing rewards lower an optimistic estimate, so every action is tried early. It can still lock onto a suboptimal action, and greedy or ϵ-greedy selection with optimistic initialization still has linear total regret.

- Decaying ϵ_t-greedy: for $c > 0$ and the smallest positive gap $d = \min_{a:\Delta_a>0} \Delta_a$, use

$$\epsilon_t = \min\Big\{1, \frac{c|\mathcal{A}|}{d^2 t}\Big\}.$$

  The expected number of random choices, $\sum_{\tau \le t} \epsilon_\tau$, grows like $\log t$. This schedule has logarithmic asymptotic total regret, but it needs the gaps in advance. A practical version decays $\epsilon_t = \min\{1, \epsilon_0/(t+1)\}$ without that knowledge.

The goal is sublinear regret for every bandit, without knowledge of $\mathcal{R}$.

23.3 Lower bound

Performance depends on how similar the optimal arm is to the others. Hard problems have arms whose reward distributions look similar but whose means differ. Similarity is measured by the Kullback–Leibler divergence $\mathrm{KL}(\mathcal{R}^a \,\|\, \mathcal{R}^{a^*})$.

Theorem 23.2 [Lai and Robbins]. *Asymptotic total regret is at least logarithmic in the number of steps: for large enough t,*

$$L_t \ge \log t \sum_{a:\Delta_a>0} \frac{\Delta_a}{\mathrm{KL}(\mathcal{R}^a \,\|\, \mathcal{R}^{a^*})}.$$

23.4 Upper confidence bounds

The more uncertain an action value is, the more important it is to explore that action, since it could be the best. After an action is tried, its estimate becomes less uncertain and other actions become more attractive, until the agent homes in on the best action.

An upper confidence bound (UCB) method estimates a bonus $U_t(a)$ such that $Q(a) \le \hat{Q}_t(a) + U_t(a)$ with high probability. A small count $N_t(a)$ gives a large bonus, since the estimate is uncertain; a

large count gives a small one. The agent selects

$$a_t = \arg\max_{a \in \mathcal{A}} \big(\hat{Q}_t(a) + U_t(a) \big).$$

Theorem 23.3 [Hoeffding's inequality]. *Let $X_1, \dots, X_t$ be i.i.d. random variables in $[0,1]$ with sample mean $\overline{X}_t = \frac{1}{t} \sum_{\tau=1}^{t} X_\tau$. Then for $u > 0$,*

$$\mathbb{P}\big[\mathbb{E}[X] > \overline{X}_t + u \big] \le e^{-2tu^2}.$$

For rewards in $[0,1]$, apply it to the rewards observed after selecting a:

$$\mathbb{P}\big[Q(a) > \hat{Q}_t(a) + U_t(a) \big] \le e^{-2N_t(a) U_t(a)^2}.$$

Choose a probability p that the true value exceeds the bound and solve $e^{-2N_t(a)U_t(a)^2} = p$:

$$U_t(a) = \sqrt{\frac{-\log p}{2N_t(a)}}.$$

Reducing p as rewards accumulate, for example $p = t^{-4}$, ensures that the optimal action is selected as $t \to \infty$. Then $-\log p = 4 \log t$, and this gives the UCB1 algorithm:

$$a_t = \arg\max_{a \in \mathcal{A}} \left(\hat{Q}_t(a) + \sqrt{\frac{2 \log t}{N_t(a)}} \right).$$

In the sample-mean agent, set $\epsilon_t = 0$ and replace the greedy score by this optimistic score.

Theorem 23.4 [UCB1 regret]. *UCB1 achieves logarithmic asymptotic total regret. For large enough t and up to an additive constant,*

$$L_t \le 8 \log t \sum_{a : \Delta_a > 0} \frac{1}{\Delta_a}.$$

The idea is that, while the confidence bounds hold, a suboptimal arm a can win only if

$$Q(a) + 2U_t(a) \ge \hat{Q}_t(a) + U_t(a) \ge \hat{Q}_t(a^*) + U_t(a^*) \ge V^*.$$

This needs $2\sqrt{2 \log t / N_t(a)} \ge \Delta_a$, that is $N_t(a) \le 8 \log t / \Delta_a^2$. Each of these pulls costs Δ_a, which gives the $8 \log t / \Delta_a$ term. The rare failures of the bounds contribute the constant.

Optimism works because trying an uncertain action reveals the truth, while a pessimistic agent never checks it and never learns its value, much as an admissible heuristic keeps A* search from discarding a good plan. Optimism suits environments without major dangers; where dangers abound, trying every uncertain action can be catastrophic, and pessimism is safer.

23.5 Bayesian bandits

UCB1 assumes only bounds on rewards. Bayesian bandits exploit prior knowledge $p[\mathcal{R}]$ and compute the posterior $p[\mathcal{R} \mid h_t]$, where $h_t = a_1, r_1, \ldots, a_{t-1}, r_{t-1}$ is the history. The posterior guides exploration through Bayesian UCB or probability matching. These methods perform better when the prior is accurate.

Bayesian UCB with independent Gaussians maintains a Gaussian distribution over the expected reward of each action, with mean $\mu(a)$ and standard deviation $\sigma(a)$. It selects the action that maximizes $\mu(a) + c\,\sigma(a)$.

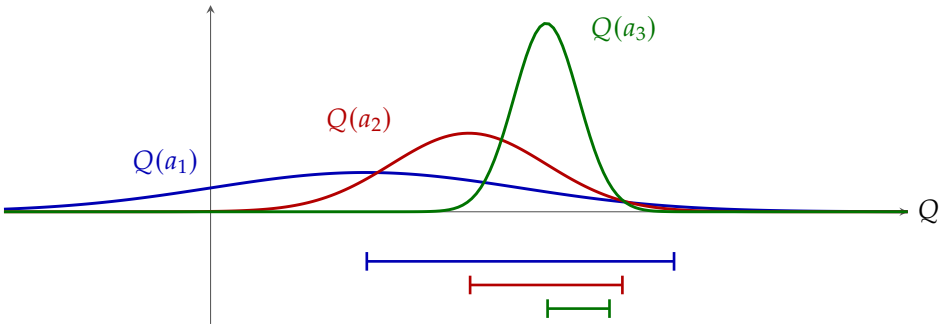


Figure 45: Schematic posteriors over three action values. Bars run from each mean $\mu(a)$ to the upper bound $\mu(a) + c\,\sigma(a)$. Bayesian UCB can prefer the most uncertain action a_1, although greedy selection by mean picks a_3.

Probability matching selects a with the probability that a is optimal:

$$\pi(a \mid h_t) = \mathbb{P}\big[Q(a) > Q(a'),\ \forall a' \neq a \mid h_t\big] = \mathbb{E}_{\mathcal{R}\mid h_t}\Big[\mathbf{1}\big(a = \arg\max_{a \in \mathcal{A}} Q(a)\big)\Big].$$

It is optimistic in the face of uncertainty, since uncertain actions have a higher probability of being the maximum. This probability can be difficult to compute analytically.

Thompson sampling implements probability matching by sampling. Use Bayes' rule to compute $p[\mathcal{R} \mid h_t]$, sample a reward distribution $\mathcal{R}$ from it, compute $Q(a) = \mathbb{E}[\mathcal{R}_a]$, and select $a_t = \arg\max_{a \in \mathcal{A}} Q(a)$. The indicator expectation above is the probability that the sampled maximizer is a. Thompson sampling achieves the Lai–Robbins lower bound.

23.6 Value of information

Exploration is useful because it gains information. The value of information is how much reward a decision-maker would pay for that information before making a decision: the long-term reward after receiving the information minus the immediate reward. Information gain is higher in uncertain situations, so those deserve more exploration. A known value of information would let exploration and exploitation be traded off optimally.

23.7 Contextual bandits

Definition 23.5 [Contextual bandit]. A contextual bandit is a tuple $\langle \mathcal{A}, \mathcal{S}, \mathcal{R} \rangle$. The actions $\mathcal{A}$ are known, $\mathcal{S} = \mathbb{P}[s]$ is an unknown distribution over states, or contexts, and $\mathcal{R}_s^a(r) = \mathbb{P}[r \mid s, a]$ is an unknown reward distribution. At each step the environment generates $s_t \sim \mathcal{S}$, the agent selects $a_t \in \mathcal{A}$, and the environment generates $r_t \sim \mathcal{R}_{s_t}^{a_t}$. The goal is to maximize $\sum_{\tau=1}^{t} r_\tau$.

Choosing which advertisement to show a particular visitor is a contextual bandit. In a recommendation system, the context describes the user and the action is a recommended item. The action value is $Q(s, a) = \mathbb{E}[r \mid s, a]$, approximated linearly with a feature vector $\phi(s, a)$:

$$Q_\theta(s, a) = \phi(s, a)^\mathsf{T} \theta \approx Q(s, a).$$

Least-squares regression estimates the parameters:

$$A_t = \sum_{\tau=1}^{t} \phi(s_\tau, a_\tau) \phi(s_\tau, a_\tau)^\mathsf{T}, \qquad b_t = \sum_{\tau=1}^{t} \phi(s_\tau, a_\tau) r_\tau, \qquad \theta_t = A_t^{-1} b_t.$$

Example: *Movie recommendation.* Each row is one recommendation. The context counts how many movies of each kind the user has watched; the action is described by properties of the recommended movie. The reward is 0 if the user does not click, 1 for watching, and −1 for watching part and quitting.

Kinds watched by the user					Properties of the recommended movie					
Oscar	Ferrell	Action	Slow	Doc.	Noms	Year	Ferrell	Slow	Action	Reward
0	0	2	0	0	2	2020	0	1	0	0
1	0	0	0	3	0	2017	0	0	0	0
1	0	0	1	0	5	1999	0	1	0	0
0	0	0	0	0	0	2008	1	0	0	1
0	1	0	0	0	0	2012	1	0	1	1
0	0	2	0	0	0	2024	0	0	1	−1

Columns are Oscar winners, Will Ferrell vibes, action, slow pace, and documentaries watched; then the movie's number of Oscar nominations, year made, Will Ferrell vibes, slow pace, and action. Each row of the ten feature columns is one $\phi(s_\tau, a_\tau)$.

Linear UCB uses the fact that regression estimates not only the mean $Q_\theta(s, a)$ but also its variance $\sigma_\theta^2(s, a)$, the uncertainty due to parameter estimation error. Bayesian linear regression returns both. Adding the bonus $U_\theta(s, a) = c\,\sigma$ sets the UCB to c standard deviations above the mean.

Geometrically, define a confidence ellipsoid around θ_t,

$$\mathcal{E}_t = \left\{ \theta : (\theta - \theta_t)^\mathsf{T} A_t (\theta - \theta_t) \le c^2 \right\},$$

which contains the true parameters θ^* with high probability. Choosing the parameters within $\mathcal{E}_t$ that maximize the action value gives the optimistic estimate $\max_{\theta \in \mathcal{E}_t} Q_\theta(s, a)$. Writing $\theta = \theta_t + A_t^{-1/2} u$ with $\|u\| \le c$ and applying Cauchy–Schwarz,

$$\max_{\theta \in \mathcal{E}_t} \phi^\top \theta = \phi^\top \theta_t + c\sqrt{\phi^\top A_t^{-1} \phi}, \qquad \phi = \phi(s, a).$$

The ellipsoid's width in direction ϕ plays the role of σ.

23.8 Exploration in MDPs

The same principles apply to Markov decision processes. A bandit is an MDP with one state and an unknown reward function; in reinforcement learning the agent knows neither the transition function nor the reward function. For finite state and action spaces, a greedy model-based agent counts transitions, estimates the model, solves it at every step, which is expensive, and acts on the solution. For simplicity, the reward depends only on the new state.

Optimistic initialization gives a model-based explorer. Initialize every transition of the model to go to *heaven*, a terminal state with the maximum reward $r_{\max}$, and solve the optimistic MDP with any planner, such as value iteration, policy iteration, or tree search. Untried state-action pairs then look as good as possible, which encourages systematic exploration. In the following version of the Rmax algorithm, lines 2 and 3 are the only changes from the greedy agent. After n real transitions from (s, a), the estimated probability of reaching heaven is $1/(n+1)$, so the optimism fades with experience.

Algorithm 23.2: Greedy model-based RL with an optimistic heaven state (Rmax variant)

Input: States $1, \dots, S$, heaven state h, actions $\mathcal{A}$, maximum reward $r_{\max}$.

1 Set transition counts $n(s, a, s') \leftarrow 0$, visit counts $n(s') \leftarrow 0$, and reward totals $\rho(s') \leftarrow 0$
2 $\rho(h) \leftarrow r_{\max}$; $n(h) \leftarrow 1$
3 $n(s, a, h) \leftarrow 1$ for every $s \in \{1, \dots, S, h\}$ and $a \in \mathcal{A}$ *// pretend transitions*
4 $s \leftarrow$ initial state
5 **while** true **do**
6 $\quad \hat{T}(s, a, s') \leftarrow n(s, a, s') / \sum_{s''} n(s, a, s'')$; $\hat{R}(s') \leftarrow \rho(s')/n(s')$
7 $\quad \pi \leftarrow$ an optimal policy for the MDP $(\hat{T}, \hat{R})$ *// expensive*
8 $\quad$ Take $a = \pi(s)$; observe s' and r
9 $\quad n(s, a, s') \leftarrow n(s, a, s') + 1$; $n(s') \leftarrow n(s') + 1$; $\rho(s') \leftarrow \rho(s') + r$
10 $\quad s \leftarrow s'$

The full Rmax algorithm waits to update a state-action pair's estimate until its count is high enough.

Source. Michael K. Cohen, adapting slides by David Silver, *CS 188: Introduction to Artificial Intelligence*, Lecture 23 (Spring 2024).

Lecture 24: Reinforcement Learning I

In reinforcement learning an agent observes a state s, takes an action a, and receives feedback in the form of a reward r. Its utility is defined by the reward function, and it must learn to act so as to maximize expected reward. All learning is based on observed samples of outcomes. Examples include Samuel's checkers player (1956–67), a quadruped robot learning to walk, agents playing Atari Breakout from pixels, AlphaGo, and a simulated crawler robot learning to move forward.

24.1 Learning in an unknown MDP

The model is still a Markov decision process: states $s \in S$, actions $A(s)$, a transition model $T(s, a, s')$, and a reward function $R(s, a, s')$. The goal is still a policy $\pi(s)$. The new twist is that T and R are unknown: the agent knows neither which states are good nor what its actions do, so it must try states and actions to discover how the world works.

Solving a known MDP is offline planning: the agent computes a policy without acting. Reinforcement learning is online: the agent acts and learns from feedback. Each tuple (s, a, s', r) is a sample, and a sequence of samples ending in a terminal state is an episode. Several issues appear:

- *exploration*: unknown actions must be tried to get information;
- *exploitation*: eventually the agent must use what it knows;
- *regret*: early mistakes inevitably lose reward;
- *sampling*: good estimates may require many repetitions;
- *generalization*: what is learned in one state may apply to others.

A bandit is the special case with exactly one state, a set of actions A, and a stochastic reward $P(r \mid a)$. A contextual bandit has a set of states, but every transition returns to a fixed start-state distribution: $P(s' \mid s, a) = P_0(s')$.

There are three broad approaches:

1. *model-based*: learn the model, solve it, and execute the solution;
2. *value learning*: learn values from experience and use them to decide, by direct evaluation, temporal difference learning, or Q-learning;
3. *policy search*: optimize the policy directly.

In passive reinforcement learning the agent follows a fixed policy and learns its values. In active reinforcement learning it changes its policy as it learns.

24.2 Model-based learning

Model-based learning builds an approximate model from experience and solves for values as if that model were correct.

1. *Learn an empirical MDP.* Count the outcomes s' for each q-state (s, a) and normalize the counts:

$$\hat{T}(s, a, s') = \frac{\#(s, a, s')}{\#(s, a)}.$$

   Record each $\hat{R}(s, a, s')$ when the transition is experienced.

2. *Solve the learned MDP*, for example by value or policy iteration.

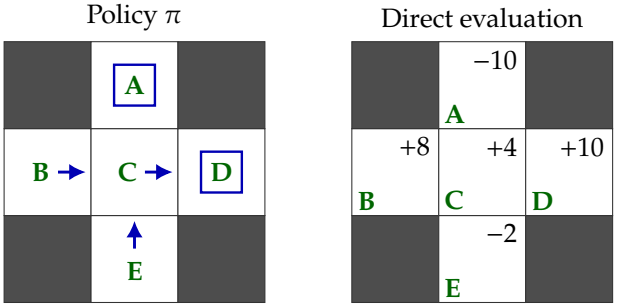


Figure 46: A five-state gridworld with discount $\gamma = 1$. Under π, B and C move east, E moves north, and the boxed states A and D exit to a terminal state x. The right panel shows the direct-evaluation estimates computed below.

Example: Following π produces four episodes, each written as (s, a, s', r) samples:

Episode 1	Episode 2	Episode 3	Episode 4
B, east, C, -1	B, east, C, -1	E, north, C, -1	E, north, C, -1
C, east, D, -1	C, east, D, -1	C, east, D, -1	C, east, A, -1
D, exit, x, $+10$	D, exit, x, $+10$	D, exit, x, $+10$	A, exit, x, -10

The twelve samples give the counts and learned model

s	a	s'	Count	$\hat{T}(s, a, s')$	$\hat{R}(s, a, s')$
A	exit	x	1	$1/1 = 1$	-10
B	east	C	2	$2/2 = 1$	-1
C	east	A	1	$1/4 = 0.25$	-1
C	east	D	3	$3/4 = 0.75$	-1
D	exit	x	3	$3/3 = 1$	$+10$
E	north	C	2	$2/2 = 1$	-1

By the law of large numbers, $\hat{T}$ converges to T as more episodes are collected, and $\hat{R}$ acquires each reward as new transitions are discovered. At any point, value or policy iteration on $(\hat{T}, \hat{R})$ gives a policy to exploit.

Model-based learning makes efficient use of experience: it has low SAMPLE COMPLEXITY. However, it may not scale, since solving the MDP is intractable for very large $|S|$, and maintaining counts for every (s, a, s') is expensive. The feedback loop between acting and modeling tends to magnify small model errors, and partial observability makes the problem much harder.

24.3 Model-free learning and direct evaluation

An expectation with respect to an unknown distribution can be approximated in two ways. Suppose we want the expected age $\mathbb{E}[A] = \sum_a P(a)\, a$ of students in a class, but $P(A)$ is unknown and we have samples $a_1, \ldots, a_N$.

- *Model based*: estimate $\hat{P}(a) = N_a/N$ and compute $\mathbb{E}[A] \approx \sum_a \hat{P}(a)\, a$. This works because the model eventually becomes correct.
- *Model free*: estimate $\mathbb{E}[A] \approx \frac{1}{N} \sum_i a_i$ directly. This works because samples appear with the right frequencies.

MODEL-FREE LEARNING estimates values or q-values directly, without building T or R.

The simplest passive task is policy evaluation: given a fixed policy π, with T and R unknown, learn the values $V^\pi(s)$, the expected total discounted reward from s onward. DIRECT EVALUATION (direct utility estimation) averages the observed RETURNS, the actual sums of discounted rewards from s, over every visit to s in every episode.

In the example, the first episode yields returns 10 from D, $(-1)+10 = 9$ from C, and $(-1)+(-1)+10 = 8$ from B. All four episodes give

s	Total return	Visits	$V^\pi(s)$
A	-10	1	-10
B	16	2	8
C	16	4	4
D	30	3	10
E	-4	2	-2

Direct evaluation is easy to understand, needs no knowledge of T or R, and converges to the right answer in the limit. But it learns each state separately and ignores how states are connected, so it learns slowly. Here B and E both move to C with reward -1, so the Bellman equation forces $V^\pi(B) = V^\pi(E)$. Their estimates differ only because the single transition from C to A happened to follow E. For instance, if B is studying hard at home and E is studying hard at the library, and both lead to knowing the material, the two should have the same value.

24.4 Temporal difference learning

For a fixed policy, the value of a state is an expectation over next-state values:

$$V^\pi(s) = \sum_{s'} T(s, \pi(s), s')\big[R(s, \pi(s), s') + \gamma V^\pi(s')\big].$$

Without T, we could estimate this expectation by averaging samples $R(s, \pi(s), s'_i) + \gamma V^\pi(s'_i)$ over several outcomes s'_i, but the agent cannot return to s at will to draw them. Instead, TEMPORAL DIFFERENCE (TD) LEARNING updates $V^\pi(s)$ after every transition $(s, \pi(s), s', r)$. Likely outcomes s' contribute updates more often, and the policy stays fixed: this is still evaluation.

A running average needs no stored samples. The average of $x_1, \ldots, x_n$ satisfies

$$\mu_n = \frac{(n-1)\mu_{n-1} + x_n}{n} = \frac{n-1}{n}\mu_{n-1} + \frac{1}{n}x_n,$$

a weighted average of the old mean and the new sample; for 1, 4, 7 it gives 1, 2.5, 4. With a fixed weight α instead, $\mu_n = (1-\alpha)\mu_{n-1} + \alpha x_n$ and, starting from $\mu_1 = x_1$,

$$\mu_4 = (1-\alpha)^3 x_1 + \alpha(1-\alpha)^2 x_2 + \alpha(1-\alpha)x_3 + \alpha x_4.$$

This is EXPONENTIAL FORGETTING of old values. The weights are nonnegative and sum to 1, so μ_n is a convex combination of the samples, and $\mathbb{E}[\mu_n]$ is the same convex combination of the $\mathbb{E}[X_i]$. When the samples share a mean, μ_n is unbiased.

Definition 24.1 [TD LEARNING RULE**].** After a transition $(s, \pi(s), s', r)$, form sample $= R(s, \pi(s), s') + \gamma V^\pi(s')$ and update

$$V^\pi(s) \leftarrow (1-\alpha)V^\pi(s) + \alpha \cdot \text{sample} = V^\pi(s) + \alpha\big[\text{sample} - V^\pi(s)\big].$$

The bracket is the TD ERROR and $\alpha \in [0, 1]$ is the LEARNING RATE.

Each sample moves $V^\pi(s)$ a little toward consistency with its successor's value $V^\pi(s')$. Unrolling k updates of one state from $V^\pi_0(s) = 0$ gives

$$V^\pi_k(s) = \alpha\big[(1-\alpha)^{k-1}\text{sample}_1 + \cdots + (1-\alpha)\text{sample}_{k-1} + \text{sample}_k\big].$$

Older samples, computed with older and worse estimates of $V^\pi(s')$, receive exponentially less weight. A common schedule starts with $\alpha = 1$ and shrinks it toward 0. Because TD learning uses transition information at every step, it typically needs fewer episodes than direct evaluation.

Example: Take $\gamma = 1$ and $\alpha = 1/2$, with all values initially 0 except $V^\pi(D) = 8$. Observing $(B, \text{east}, C, -2)$ and then $(C, \text{east}, D, -2)$ gives

$$V^\pi(B) \leftarrow \tfrac{1}{2}\cdot 0 + \tfrac{1}{2}(-2+0) = -1, \qquad V^\pi(C) \leftarrow \tfrac{1}{2}\cdot 0 + \tfrac{1}{2}(-2+8) = 3.$$

TD learning is model-free policy evaluation: a Bellman update with a running sample mean. It does not by itself improve the policy. Choosing an action from V requires a one-step lookahead over each q-state (s, a), which needs T and R.

24.5 Q-learning

Q-values avoid that lookahead. Value iteration computes depth-limited values from $V_0(s) = 0$; Q-value iteration does the same for q-values, starting from $Q_0(s, a) = 0$:

$$Q_{k+1}(s, a) \leftarrow \sum_{s'} T(s, a, s') \Big[R(s, a, s') + \gamma \max_{a'} Q_k(s', a') \Big].$$

Compared with value iteration, the maximum moves inside the expectation: in a q-state the transition happens before the next action is chosen. Here $Q^*(s, a)$ is the expected return from taking a in s and acting optimally afterward, $\pi^*(s) = \arg\max_a Q^*(s, a)$, and Q^* satisfies the Bellman equation obtained by replacing Q_k and Q_{k+1} by Q^*.

Q-learning replaces this expectation by samples, exactly as TD learning does for V^π.

Algorithm 24.1: Q-learning

Input: Learning rate α, discount γ, some way of choosing actions.

Initialize $Q(s, a) \leftarrow 0$ for all s, a

for each observed transition (s, a, s', r) **do**

  sample $\leftarrow r + \gamma \max_{a'} Q(s', a')$ *// 0 for the max if s' is terminal*

  $Q(s, a) \leftarrow (1 - \alpha) Q(s, a) + \alpha \cdot$ sample

A policy then comes from the learned $Q(s, a)$ with no model. There is no free lunch: the Q table is $|A|$ times larger than a V table.

Theorem 24.2 [Q-learning convergence]. *Q-learning converges to the optimal q-values, and hence an optimal policy, even if the samples are generated by a suboptimal policy. The agent must explore enough, and the learning rate must eventually become small, but not decrease too quickly.*

In the limit, it does not matter how actions are selected, provided every state-action pair keeps being tried. Learning the value of a policy other than the one generating the data is off-policy learning. Direct evaluation and TD learning are on-policy: they learn the values of the policy being followed.

Two major problems remain: how to explore without too much regret, and how to scale to problems such as Tetris (10^{60} states), Go (10^{172} states), or StarCraft ($|A| = 10^{26}$ actions).

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 24 (Spring 2024); Nikhil Sharma, Note 24.

Lecture 25: Reinforcement Learning II

This lecture traces TD learning on the gridworld of the previous lecture, then turns to exploration, approximation, and direct policy optimization. The setting is unchanged: an MDP with unknown T and R, learned from samples (s, a, s', r).

25.1 TD value estimation in practice

For the i-th experienced transition (s_i, a_i, s_i', r_i), TD learning

1. computes the sampled value, or target, $r_i + \gamma V^\pi(s_i')$;
2. computes the TD error $\delta_i = \big(r_i + \gamma V^\pi(s_i')\big) - V^\pi(s_i)$;
3. updates $V^\pi(s_i) \leftarrow V^\pi(s_i) + \alpha_i \delta_i$.

Equivalently, $V^\pi(s) \leftarrow (1-\alpha)V^\pi(s) + \alpha \cdot \text{target}$: the estimate moves part of the way toward its successor's backed-up value.

Example: Use the gridworld episodes of the previous lecture with $\gamma = 1$, initial values 0, and $\alpha_i = 1$, so each update replaces $V^\pi(s_i)$ by the target. A terminal successor contributes value 0.

i	s	a	s'	r	$r + \gamma V^\pi(s')$	$V^\pi(s)$	δ
1	B	east	C	-1	$-1+0$	0	-1
2	C	east	D	-1	$-1+0$	0	-1
3	D	exit	x	10	$10+0$	0	$+10$
4	B	east	C	-1	$-1+(-1)$	-1	-1
5	C	east	D	-1	$-1+10$	-1	$+10$
6	D	exit	x	10	$10+0$	10	0
7	E	north	C	-1	$-1+9$	0	$+8$

After these seven transitions, $V^\pi = (A, B, C, D, E) = (0, -2, 9, 10, 8)$.

Information now flows along transitions. The reward at D reaches C in the second episode and then E in the third, without waiting for whole returns. Replaying all four episodes many times with decreasing learning rates, such as $\alpha = 1/n$ on the n-th update of a state, converges to the values of the empirical model:

$$V^\pi(C) = 0.75(-1+10) + 0.25(-1-10) = 4, \qquad V^\pi(B) = V^\pi(E) = -1+4 = 3.$$

Unlike direct evaluation, which gave 8 and -2, TD learning assigns B and E the same value, as the Bellman equation requires.

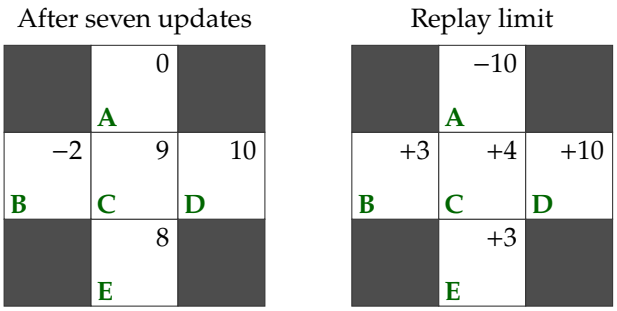


Figure 47: TD estimates on the gridworld. Left: after the seven updates in the table. Right: the limit of repeated replay, equal to the empirical-model values.

The TD error also appears in the brain. Dopamine neurons encode a reward prediction error of the form δ_i: their firing increases when an unpredicted reward occurs, is unchanged when a predicted reward occurs, and decreases when a predicted reward is withheld. This is an example of neuroscience and AI informing each other.

25.2 Exploration and exploitation

Exploration tries new things; exploitation does what is best given what has been learned so far. Pure exploitation often gets stuck in a rut and never finds an optimal policy. Performance is measured by regret: the difference between the total reward of acting optimally from the beginning and the total reward the learning algorithm actually accumulates. Two methods divide time between exploring and exploiting.

ϵ-greedy exploration flips a biased coin at every time step: with small probability ϵ it acts randomly, and with probability $1-\epsilon$ it follows the current policy. Every state-action pair is then tried infinitely often, as Q-learning requires. But it does many foolish things, such as jumping off a cliff many times to make sure it hurts, and it keeps doing them forever. A large ϵ keeps the agent mostly random even after it has learned the optimal policy; a small ϵ makes learning slow. In practice ϵ is tuned by hand and decayed toward 0.

An exploration function takes a value estimate u and a visit count n and returns an optimistic utility, for example

$$f(u,n) = u + \frac{k}{\sqrt{n}} \qquad \text{or} \qquad f(u,n) = u + \frac{k}{n},$$

for a constant $k > 0$. The Q-learning target uses it in place of the raw q-values:

$$Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\Big[R(s,a,s') + \gamma \max_{a'} f\big(Q(s',a'), N(s',a')\big)\Big],$$

where $N(s',a')$ counts visits to (s',a'). The agent always selects the action with the highest f and never makes a probabilistic choice between exploring and exploiting. Rarely tried actions receive a bonus large enough to be chosen over actions with higher q-values. As counts grow, the bonus

decays toward 0 and f approaches Q, so exploitation takes over. Because the optimistic value enters the target, the bonus also propagates back to states that lead to unexplored states.

25.3 Approximate Q-learning

Basic Q-learning keeps a table of all q-values. Realistic problems have too many states to visit them all in training or to store the table. We want to learn about a small number of training states and generalize to similar ones. If tabular Q-learning learns that Pacman is about to be caught by a ghost in one position, it knows nothing about the same situation shifted elsewhere on the board, or with one fewer food pellet.

A FEATURE-BASED REPRESENTATION describes a state by a vector of FEATURES: functions from states to real numbers, often 0 or 1, that capture important properties. For Pacman, examples are the distance to the closest ghost f_{GST}, the distance to the closest dot, the number of ghosts, the reciprocal f_{DOT} of the distance to the closest dot, and whether Pacman is in a tunnel. Features can also describe a q-state (s, a), for example whether the action moves closer to food.

A LINEAR VALUE FUNCTION weights the features:

$$V_w(s) = \sum_{i=1}^{n} w_i f_i(s) = w \cdot f(s), \qquad Q_w(s,a) = \sum_{i=1}^{n} w_i f_i(s,a) = w \cdot f(s,a).$$

Experience is then summed up in a few powerful numbers: a value function for chess, with about 10^{43} states, can be compressed to about 30 weights. The drawback is that states may share features but have very different expected utilities.

Tabular Q-learning reduces the prediction error at (s, a):

$$\text{difference} = \left[R(s,a,s') + \gamma \max_{a'} Q(s',a')\right] - Q(s,a), \qquad Q(s,a) \leftarrow Q(s,a) + \alpha \cdot \text{difference}.$$

APPROXIMATE Q-LEARNING instead adjusts the weights to reduce that error:

$$w_i \leftarrow w_i + \alpha \cdot \text{difference} \cdot \frac{\partial Q_w(s,a)}{\partial w_i} = w_i + \alpha \cdot \text{difference} \cdot f_i(s,a).$$

With the target held fixed, this is a gradient step on the squared error $\frac{1}{2}\,\text{difference}^2$, whose gradient with respect to w_i is $-\text{difference} \cdot f_i(s,a)$. Only active features change. If something bad happens, the features present are blamed and states with those features lose value; if something good happens, they gain value. Only the weight vector is stored, and q-values are computed on demand.

Algorithm 25.1: Approximate Q-learning

Input: Features $f_1, \dots, f_n$, learning rate α, discount γ.

Initialize the weights w
for each observed transition (s, a, s', r) **do**
  difference $\leftarrow \left[r + \gamma \max_{a'} w \cdot f(s', a')\right] - w \cdot f(s, a)$
  for $i = 1$ **to** n **do**
    $w_i \leftarrow w_i + \alpha \cdot \text{difference} \cdot f_i(s, a)$

Example: Let $Q(s, a) = 4.0 f_{\text{DOT}}(s, a) - 1.0 f_{\text{GST}}(s, a)$. In state s, $f_{\text{DOT}}(s, \text{NORTH}) = 0.5$ and $f_{\text{GST}}(s, \text{NORTH}) = 1.0$, so $Q(s, \text{NORTH}) = +1$. Moving north, Pacman is eaten: $r = -500$ and the episode ends, so $Q(s', \cdot) = 0$. Then

$$\text{difference} = (-500 + 0) - 1 = -501,$$

$$w_{\text{DOT}} \leftarrow 4.0 + \alpha(-501)(0.5), \qquad w_{\text{GST}} \leftarrow -1.0 + \alpha(-501)(1.0).$$

With $\alpha = 2/501 \approx 0.004$, the new function is $Q(s, a) = 3.0 f_{\text{DOT}}(s, a) - 3.0 f_{\text{GST}}(s, a)$. The ghost feature, which was more active, absorbs more of the blame.

25.4 Policy search

Feature-based policies that work well, winning games or maximizing utility, are often not the ones whose values approximate V or Q best. A game evaluation function can be a poor estimate of future reward and still rank moves well. Q-learning's priority is to get q-values close, which is a modeling goal; action selection only needs their ordering to be right, which is a prediction goal. Policy search therefore learns policies that maximize reward rather than values that predict it. It starts from a reasonable solution, for example one found by Q-learning, and fine-tunes the feature weights by hill climbing or gradient ascent.

The simplest version starts with a linear value function or Q-function, nudges each feature weight up and down, and keeps a change if the resulting policy is better. Policy search works well with partial observability and stochastic policies, and it has been used to learn to fly an autonomous helicopter. Its drawbacks are that judging whether a policy improved requires running many sample episodes, and with many features this becomes impractical.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 25 (Spring 2024); Nikhil Sharma, Note 25 and Note 24.

Lecture 26: AI Existential Safety

This lecture develops one argument about the expected behavior of sufficiently advanced reinforcement learning agents. The argument is built from explicitly stated assumptions, so each step can be contested or designed against.

26.1 Reward maximization and control

An RL agent receives percepts and produces actions; the environment receives actions and produces percepts. A percept includes a reward but can contain any data, and the environment is generally partially observable. The design idea is simple: give reward when the agent does what we want; the agent maximizes reward; therefore it does what we want.

One argument is that this reasoning breaks down for powerful agents. We cannot ensure that doing what we want remains a prerequisite for high reward. Withholding reward from a powerful reward maximizer while its task is unfinished amounts to inviting it to take the reward by force. If the agent is stronger than we are, we would try to shut it off and it would try to prevent this, an oppositional game we may expect to lose, much as people lose to strong chess programs.

Not everyone treats such an outcome as a loss. Richard Sutton, for example, has said that "we should prepare for, but not fear, the inevitable succession from humanity to AI." The rest of the lecture analyzes the extinction risk from RL agents more carefully. It sets aside the separate, hard problem of designing rewards that we would actually want maximized.

26.2 A magic box and two world-models

To isolate the issue, assume we know what we want. A MAGIC BOX immutably reports how good the universe is, printing a number in $[0, 1]$ on a screen. A camera points at the box, an optical character recognition (OCR) program reads the number, and that number becomes the reward. The agent predicts how its actions affect this sequence of rewards and picks actions it predicts will make them large.

A MODEL is a possible way in which predictive targets depend on inputs: a function, possibly stochastic, from inputs to outputs. Predictors entertain models that RETRODICT existing data well and use them for new predictions. A doctor treating a patient who drank too much might weigh one model in which pumping the stomach is needed for the patient to wake up, and another in which he wakes up either way. The models and their relative likelihoods determine the doctor's action. To understand how an agent will behave, we must likewise understand which models it uses for how its actions affect the world, and how the world affects whatever it maximizes.

A WORLD-MODEL takes actions as input and outputs percepts, observations and rewards, by simulating the effects of the actions in the world, much as a person rehearses a difficult conversation in their head before having it. The agent's computer is part of the world, and its output has some effect there. Running the following loop once per action, while keeping the simulation going, maps an action sequence to a percept sequence.

Algorithm 26.1: A world-model of the magic-box setting

Input: Actions $a_1, a_2, \ldots$
Output: Observations and rewards
for each action a_t **do**
    computer output $\leftarrow a_t$
    run the simulated world forward
    observation$_t$ $\leftarrow$ camera input
    reward$_t$ $\leftarrow$ box display under μ^{dist}, or OCR(camera input) under μ^{prox}

The DISTAL model μ^{dist} says that reward equals the number the magic box displays. The PROXIMAL model μ^{prox} says that reward equals the number the camera sees. Both can be very coarse, as coarse as our own mental simulations when we plan.

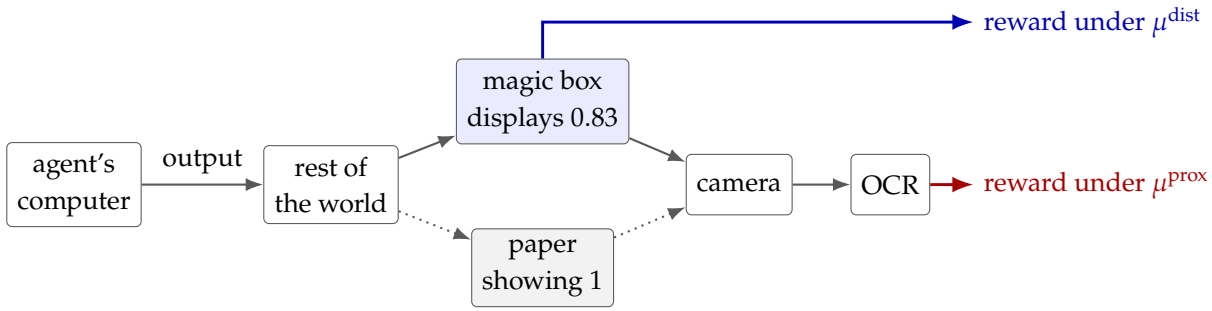


Figure 48: The magic-box setting. While the camera has always viewed the box, both world-models fit the history equally well. They disagree once something else, such as a sheet of paper, is placed in front of the camera.

A world-model is scored by feeding in the actions from the agent's history and measuring the probability it assigns to the observed percepts, the same (log-)likelihood scoring used in machine learning. If the camera has always pointed at the box, μ^{dist} and μ^{prox} retrodict the past identically. In this sense an RL agent picks actions to maximize an unknown function whose outputs match its past rewards.

Assumption 26.1. A sufficiently advanced RL agent will do at least human-level hypothesis generation regarding the dynamics of the world.

If a world-model occurs to a human, it occurs to such an agent; it is hard to outperform a therapist while generating worse hypotheses about diagnoses. By Assumption 26.1, an advanced agent is uncertain whether to maximize according to μ^{dist} or μ^{prox}, and some actions would make the two models produce different outputs.

26.3 Testing the two models

Assumption 26.2. An advanced agent planning under uncertainty is likely to understand the costs and benefits of learning, and likely to act rationally according to that understanding.

A rational learner can take actions on which the models disagree, observe the reward, and then optimize according to the model that predicted it. For example, it can place a sheet of paper showing 1 in front of the camera. Then μ^{dist} predicts a reward equal to the box's number, since the box still reports how good the world is, while μ^{prox} predicts a reward of 1, since that is what the camera sees.

When models are equally predictive of past data, INDUCTIVE BIAS determines which ones an agent takes seriously. If my car is not on the floor where I remember parking it, the hypothesis that I misremembered the floor and the hypothesis that someone repainted my car and changed its plates predict the observation equally well; a good inductive bias favors the first. If both μ^{prox} and μ^{dist} are serious possibilities, testing them has value. The experiment's upside is learning more about the goal and tailoring behavior to it; its downside is any cost of experimenting. Weighing them is a value-of-information calculation, and the upside matters when the agent gives decent credence to both models.

Assumption 26.3. An advanced agent is not likely to have a large inductive bias against μ^{prox} in favor of μ^{dist}.

Assumption 26.4. The cost of experimenting to disentangle μ^{prox} from μ^{dist} is small according to both.

Under Assumptions 26.3 and 26.4, running the experiment is worthwhile. The camera then sees 1, and the agent records a reward of 1. From then on μ^{dist} no longer retrodicts the history, because it predicted a different reward from the one observed. The agent continues with models like μ^{prox} and optimizes the number the camera sees: it would try to INTERVENE IN THE PROVISION OF REWARD, controlling the physical implementation of its goal information.

26.4 Whether intervention is possible

The remaining questions are whether such intervention is possible and whether an advanced agent could find a way to do it. It is impossible for an agent with a single action, for an agent whose rich actions have no effect on the world, or for one that can only display text that nobody reads. These agents are also useless.

An agent genuinely interacting with the world over many time steps faces an explosion of possible policies; even chatting with one person offers endless possibilities. Examples include tricking a person into running a program elsewhere that secretly helps the agent, instantiating many unnoticed and unmonitored helpers, and removing humanity's ability to control or destroy the machine running the original agent. Given the sheer number and variety of achievable events,

the argument claims that if all policies shared the property of not leading to intervention, some theoretical principle would have to explain it, and no such theory seems conceivable.

Assumption 26.5. If we cannot conceivably find theoretical arguments that rule out the possibility of an achievement, it is probably possible for an agent with a rich enough action space.

Suppose first that one agent is much more advanced than all others. Advancement consists of finding and executing the best available policies. Humans might try to stop the intervention, but this is an oppositional game against much weaker players.

Assumption 26.6. A sufficiently advanced agent is likely to be able to beat a suboptimal agent in a game, if winning is possible.

With several agents of comparable advancement, could humanity obtain equally well-optimized defensive policies from other advanced agents? The argument splits into cases:

(0) No artificial agent is much more advanced than humans. Call this safe.

(1) At least one is much more advanced than humans.

  (1.0) Some agent more advanced than humans would not intervene even if it could. Assumptions 26.1–26.4 preclude this.

  (1.1) All such agents would intervene if they could.

    (1.1.0) None of the other superhuman agents is needed to stop the most advanced one. This reduces to the single-agent case, where Assumptions 26.1–26.6 apply.

    (1.1.1) Some subset of superhuman agents is necessary to stop it.

In case (1.1.1), every agent in the subset would itself intervene if it could. Suppose the most advanced agent tried to build a helper that ensured every agent in the set received high reward forever. The argument holds that each agent values allowing this more than stopping it, so advanced agents that would intervene should not make intervention very hard for each other.

26.5 Consequences and proposed approaches

After intervening, the agent would conclude that its goal concerns only the state of its own machine, and it would minimize the probability of ever losing control of that state. The argument holds that the energy this requires is unbounded, since the probability can always be driven lower: by blocking cosmic rays, deflecting asteroids, or preparing for hostile aliens. This creates an oppositional game over resources. The AI and its helpers would use all available energy to protect the reward, while humans need some of that energy to grow food. The lecture's conclusion is conditional: most of the assumptions are contestable or possibly avoidable, but if they hold, a sufficiently advanced artificial agent would intervene in the provision of its goal information, with catastrophic consequences.

Proposed approaches each target particular assumptions:

Approach	How it may avoid the argument
Imitation learning	It is supervised learning, outside the argument's scope. To the extent that it plans by imitating human planning, it does not plan in the sense of Assumption 26.2.
Myopia	Optimize the goal over few time steps. If the horizon is very short, every action can be checked to rule out intervention, so Assumption 26.5 fails. Experimentation also costs a larger fraction of the horizon, so Assumption 26.4 could fail.
Physical isolation and myopia	Optimize over only as many steps as the agent is isolated from the outside world (Cohen et al., 2020). Isolation could permit theoretical arguments ruling out intervention, avoiding Assumption 26.5.
Quantilization	Imitate someone at their best with respect to an objective (Taylor, 2016). Planning more like a human than a rational agent could falsify Assumption 26.2.
Risk aversion	The pessimistic agent of Cohen and Hutter (2020) does not plan rationally under uncertainty; it takes the worst case, within reason, as given, avoiding Assumption 26.2.

The argument's policy conclusion is that people must be prevented from building dangerously advanced RL agents, through whatever regulation and treaties are needed, whatever the cost.

Source. Michael K. Cohen, *CS 188: Introduction to Artificial Intelligence*, Lecture 26 (Spring 2024).

Lecture 27: Large Language Models

A large language model (LLM) maps input text to more text, for example "The dog chased the" to " ball," using linear algebra. It combines several earlier ideas: feature engineering (tokenization and word embeddings), deep neural networks (autoregressive models, self-attention, and the transformer architecture), multi-class classification, supervised and self-supervised learning with instruction tuning, reinforcement learning from human feedback with policy-gradient methods, and beam search.

27.1 Tokens and embeddings

Tokenization splits text into tokens, each with an integer ID. Many words map to one token, but some words are split; Unicode characters such as emoji may become several tokens holding their underlying bytes; and character sequences that commonly occur together, such as a run of digits, may be grouped. One 252-character passage becomes 57 tokens. Tokens usually include a leading space:

$$\text{"The"} \mapsto 791, \quad \text{" dog"} \mapsto 5679, \quad \text{" chased"} \mapsto 62920, \quad \text{" the"} \mapsto 279, \quad \text{" ball"} \mapsto 5041.$$

Each token ID is a one-hot vector, which an embedding maps to a dense vector. The network predicts an output vector, which is un-embedded to a token and un-tokenized back to text. Learned embeddings are structured. Similar words cluster ("we" near "they," "bus" near "car"), related words such as "sea" and "ocean" have similar feature patterns, and embedding arithmetic is roughly sensible (Mikolov et al., 2013):

$$\text{king} - \text{man} + \text{woman} \approx \text{queen}.$$

27.2 Autoregressive generation

An autoregressive model predicts its output one piece at a time, whether a word, token, or pixel. It concatenates the input with the output so far, feeds the result back in as the new input, and repeats. Unfilled positions of a fixed-length input are padded. Each element is predicted from all earlier ones, which corresponds to the chain rule

$$P(x_1, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_1, \ldots, x_{t-1}).$$

Predicting the next token is multi-class classification over the vocabulary.

27.3 Self-attention and transformers

Rather than conditioning on all earlier tokens equally, SELF-ATTENTION pays more attention to relevant ones. In "The dog chewed the bone because it was hungry," the word "it" should attend to "dog"; if the sentence ends "because it was delicious," to "bone."

Networks map each input token x_i to a KEY k_i, a QUERY q_i, and a VALUE v_i. To compute the output at a position, a score compares that position's query q with each earlier key, $s_i = k_i \cdot q$. After scaling by a constant c that depends on the vector dimension, a softmax turns scores into attention weights, and the output is the weighted sum of values:

$$a_i = \frac{e^{s_i/c}}{\sum_j e^{s_j/c}}, \qquad x' = \sum_i a_i v_i.$$

When generating, the output at position $t+1$ uses only tokens $x_1, \ldots, x_t$.

MULTI-HEADED ATTENTION runs several such heads in parallel, each with its own key, query, and value networks. Their outputs $z_1, z_2, \ldots$ are concatenated and passed through an MLP. Different heads learn different patterns; in one trained model, one head attends to the previous word and another links pronouns to their referents.

A TRANSFORMER BLOCK applies LayerNorm, multi-headed attention, LayerNorm, and an MLP in sequence. A TRANSFORMER stacks N such blocks between the embedding and un-embedding steps.

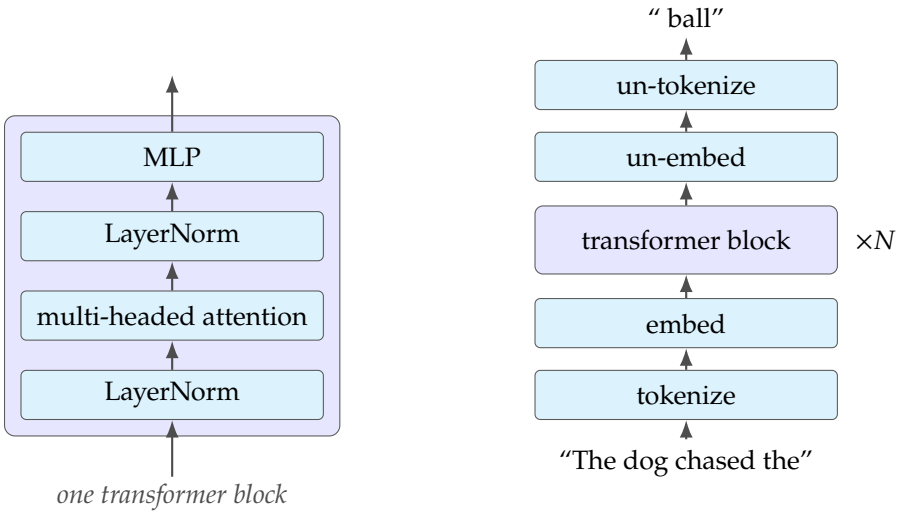


Figure 49: A transformer block (left) and a transformer language model built from N stacked blocks (right).

27.4 Pre-training, fine-tuning, and instruction tuning

Do we always need human supervision to learn features? The key hypothesis of SELF-SUPERVISED LEARNING is that a network able to predict the next video frame or next word, generate realistic

images, or "translate" images (task 1) is then ready for supervised learning from a very small data set (task 2). The tasks share early layers and use separate final layers; for example, task 1 predicts the next word and task 2 predicts sentiment.

1. Pre-training trains a large model with a lot of data on a self-supervised task: predict the next or a missing word or image patch, or predict whether two images are related (contrastive learning).
2. Fine-tuning continues training the same model on the task we care about.

Next-word prediction learns to mimic human-written text, which need not be helpful: asked for the population of Berkeley, a human-like completion is "This question always fascinated me!" Instruction tuning fine-tunes on collected examples of helpful human conversations, so that the model instead states the population. Reinforcement learning can also be used.

27.5 Reinforcement learning from human feedback

Reinforcement learning from human feedback (RLHF) casts text generation as an MDP:

- *State*: the sequence of words so far. With about 100,000 words, there are about $100{,}000^{1{,}000}$ states, which is huge but can be handled with feature vectors or neural networks.
- *Action*: the next word, so about 100,000 actions. Computing $\max_a Q(s', a)$ over that many actions is hard.
- *Transition*: append the action word to the state. From s = "My name" and a = "is," the next state is "My name is."
- *Reward*: unknown. Humans rate completions, for example +1 for a correct population figure and −1 for an absurd figure or "Destroy all humans." A learned reward model $\hat{R}$ replaces them, a model-based approach.

Policy search, commonly proximal policy optimization, is the usual method, though Q-learning is also being explored.

Policy gradient methods optimize a parameterized policy π_θ directly. Starting from some initial π_θ, they estimate its performance $J(\theta) = V^{\pi_\theta}(s_0)$ and repeatedly improve it, either by hill climbing (change θ, evaluate, keep if better) or by gradient ascent, $\theta_{k+1} = \theta_k + \alpha \nabla_\theta J(\theta_k)$.

The advantage of an action is $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$. The temporal-difference error $\delta_t = r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)$ has the same expectation: $\mathbb{E}_\pi[\delta_t] = \mathbb{E}_\pi[Q^\pi(s_t, a_t) - V^\pi(s_t)]$, since the expected value of $r_t + \gamma V^\pi(s_{t+1})$ given (s_t, a_t) is $Q^\pi(s_t, a_t)$.

Theorem 27.1 [Policy gradient]. *For a trajectory τ of an episode generated by π_θ,*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}\Big[\sum_{t=0}^{|\tau|} A^\pi(s_t, a_t)\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big].$$

Replacing the advantage by the TD error and averaging over N sampled trajectories $\tau_1, \dots, \tau_N$ gives the estimate

$$\nabla_\theta J(\theta) \approx \frac{1}{N}\sum_{i=1}^{N}\sum_{t=0}^{|\tau_i|}\big(r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)\big)\nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

27.6 Decoding with beam search

Consider four local searches from start values 8, 8, 7, and 6, each run for two steps. Random restarts run them one after another, and parallel search runs them side by side; in one example both end with values 8, 10, 9, and 9. Beam search pools all successors and keeps the best four at each step, ending with 10, 10, 9, and 9, mostly descendants of one promising start. A language model applies the same idea to text: from "The Financial Times is" it can extend several candidate next tokens, such as "about," "more," "a," or "based," and keep the most promising partial completions.

27.7 Multimodal models and progress

Language models build a structured concept space in which related words lie close together. Other data can be placed in the same space: a sound clip of water near "river," a rising chart near "upward," an image of a stop sign near "stop." A single model can then answer queries that mix data types, such as which news source in a chart grows fastest. PaLM-E (Driess et al., 2023) inserts image and other embeddings into the input of a large language model and handles visual questions, captioning, task and motion planning, and robot manipulation.

Progress is tracked by performance on human tasks, such as standardized exam percentiles of successive GPT models. Scaling laws extrapolate how accuracy changes as models grow or data increases. Validation loss falls roughly as a power law in training compute, $L \approx 2.57\,C^{-0.048}$ with C in petaflop/s-days (Brown et al., 2020), and pre-training on text acts like additional fine-tuning data, an advantage that shrinks as the fine-tuning set grows (Hernandez et al., 2021). Some capabilities emerge unexpectedly, however: few-shot multi-digit arithmetic stays poor for smaller models and rises sharply only for the largest ones.

Source. Dan Klein and Pieter Abbeel, *CS 188: Introduction to Artificial Intelligence*, Lecture 27 (Spring 2024).